

Gestão de Projeto de Software 2025/2026

Licenciatura em Engenharia Informática

João Cunha (jcunha@isec.pt)
Pedro Sá Silva (pedro.ss@isec.pt)
Gustavo Dinis (gustavo.dinis@isec.pt)

Let's Play a Game!

1. Form a team of 4 or 5 players
2. Each player should use their own smartphone
3. Scan the QR code provided and follow the on-screen instructions
4. Each one chooses a diferente role
 1. If there are 5 players, 2 of you will be programmers
5. The game includes tasks linked to specific roles, but
 1. Discuss each challenge as a team
 2. Each player must submit the answer
6. Good luck and have fun!



Software Project Management 2025/2026

João Cunha (jcunha@isec.pt)
Pedro Sá Silva (pedro.ss@isec.pt)
Gustavo Dinis (gustavo.dinis@isec.pt)



Learning Agreement

How to pass with a good grade?

“Software Project Management”
João Cunha, ISEC (jcunha@isec.pt)



Objectives

- ▶ Software project management is concerned with knowledge about the planning, organization, monitoring and control of all software life-cycle phases.
- ▶ Management is critical to ensure that:
 - ▶ software development projects are appropriate to an organization,
 - ▶ the work in different organizational units is coordinated,
 - ▶ software versions and configurations are maintained,
 - ▶ resources are available, when necessary,
 - ▶ the quality is as expected,
 - ▶ project work is divided appropriately,
 - ▶ communication is facilitated, and
 - ▶ progress is accurately charted.
- ▶ The main objective of this course is to provide knowledge, understanding and practice on software project management

Learning Outcomes

- ▶ Upon completing this course, students should be able to:
 - ▶ LO1 - Describe software development as an engineering discipline, using appropriate terminology.
 - ▶ LO2 - Develop a software project in a team, following defined processes and good software engineering practices.
 - ▶ LO3 - Plan a software development project using estimates, task scheduling, and resource allocation.
 - ▶ LO4 - Apply best practices in quality management, risk management, team management, and stakeholder management.
 - ▶ LO5 - Manage a software project by monitoring and controlling its progress, ensuring software delivery within the planned timeframes and costs, and managing stakeholder expectations.
 - ▶ LO6 - Demonstrate knowledge of software development processes and methodologies, as well as the principles and fundamentals of software process improvement.

Course Philosophy

- ▶ The course shall run around the development of a software project.
 - ▶ Following an **AGILE** methodology
- ▶ Theoretical classes shall be used for presentation and discussion of some of the subjects and study materials.
- ▶ Labs shall be used to apply the best practices in software engineering
- ▶ Labs shall also be used for software development, progress meetings with the instructor, demonstrations to the class, and clarifications about deliverables and tools.
- ▶ Students shall be assessed by their work throughout the semester.
 - ▶ In Lab classes, during ceremonies
 - ▶ In theoretical classes, responding to short quizzes and challenges

Topics

1. **Traditional and Agile Software Development Lifecycle**
 - ▶ Scrum
 - ▶ Waterfall
2. **Software Project Planning**
 - ▶ Drafting a software development plan
 - ▶ Software requirements, use cases, user stories
 - ▶ Delivery planning
 - ▶ Sprint planning
 - ▶ Task estimation and scheduling
 - ▶ Software Architecture
3. **Agile Software Project Management**
 - ▶ Requirements and backlog management
 - ▶ Risk management
 - ▶ Team management and communication with stakeholders
 - ▶ Sprints Review
3. **Quality Management**
 - ▶ Quality plan
 - ▶ Reviews
 - ▶ Tests and test plans
 - ▶ Version management
 - ▶ Process improvement
4. **Agile Software Development**
 - ▶ Scrum ceremonies, team, and artifacts
 - ▶ DevOps principles and continuous integration practices

Project outline

- ▶ The goal of the project is to build a software product within the expected scope, cost, quality and deadline.
- ▶ However, before the project begins, it must be carefully planned.
- ▶ This project shall follow a defined project management plan and quality assurance plan.
- ▶ The teams shall have 4 or 5 members.
- ▶ The project shall be **finished and presented** by week 15, i.e. **week 15-19 of December 2025**.

No extension!

Project grading policy

- ▶ Project grades are individual, and reflect both team performance and student individual contributions
- ▶ Final individual grades are based on:
 - ▶ Team grades
 - ▶ Peer assessment
 - ▶ Subjective perception of the instructor on the effective student achievements, sustained by self and peer assessment (quality of produced artifacts, participation, initiative)
- ▶ Students receive continuous feedback on their progress

Grading calculation

- ▶ **Project – 12 points**
 - ▶ Team grade = Continuous evaluation, during 7 ceremonies
 - ▶ KOM (20%) + 5 sprints (5x12%) + final presentation (20%)
 - ▶ Individual grade = Team grade x individual performance factor
- ▶ **Exam – 8 points**
 - ▶ Best of:
 - ▶ Written exam = 100% of the grade
 - ▶ Written exam (75%) + Theoretical class activities (25%)
- ▶ **Minimum of 40% for each part**

Support

- ▶ Starting point: **inforestudante.ipc.pt**
- ▶ Platform: **teams.microsoft.com**
- ▶ Development Platform: **GitLab**
- ▶ Office hours (Office or **MSTeams**)
- ▶ Communication between instructor and students, and team discussions: Posts or Chat at **MSTeams**
- ▶ Continuous feedback to students through **MSTeams**
- ▶ Individual Team Channel in **MSTeams**

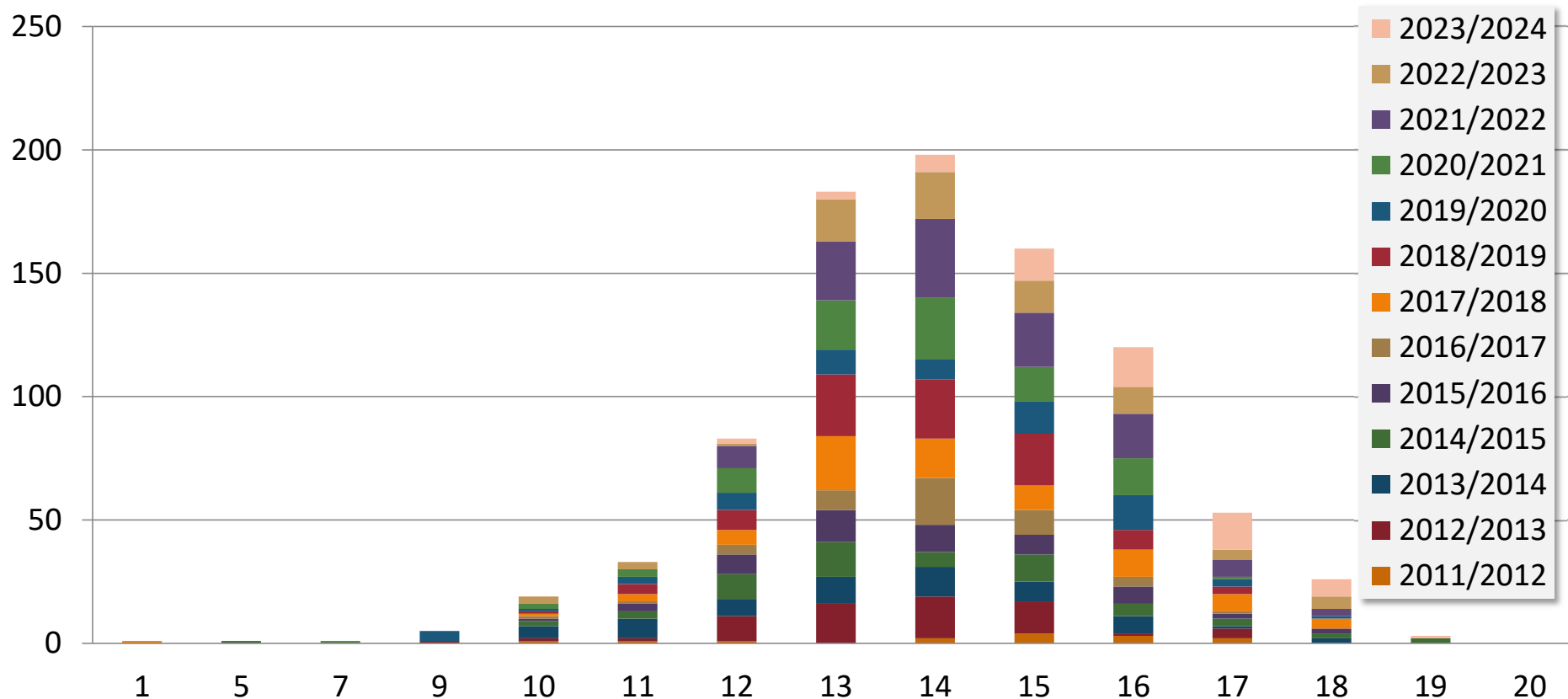
What I expect from you!

- ▶ Total working effort: 160 hours (6 ECTS)
- ▶ Regular work, per week
 - ▶ 2 hours theoretical class
 - ▶ 3 hours lab (weeks 1-3; 15)
 - ▶ 45min lab (weeks 4-14)
 - ▶ **4 hours out-of-class work for the project!**
- ▶ Non-regular work
 - ▶ 23 hours study
 - ▶ 2 hours exam

Bibliography

- ▶ Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. Scrum.org.
<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>,
<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Portuguese-European.pdf>
- ▶ Shore, J., & Warden, S. (2008). *The art of agile development*. O'Reilly Media.
- ▶ Stellman, A., & Greene, J. (2006). *Applied software project management*. O'Reilly Media.
- ▶ Sommerville, I. (2011). *Software engineering* (9th ed.). Pearson Education.
- ▶ Project Management Institute. (2017). *A guide to the project management body of knowledge (PMBOK guide) and the standard for project management* (6th ed.). Project Management Institute.

Grades 2011-2024



Project Assignment

“Software Project Management”
João Cunha, ISEC (jcunha@isec.pt)
Mickaël Santos (a21270946@isec.pt)

Short Brief (more details in the Lab class)

► Goal

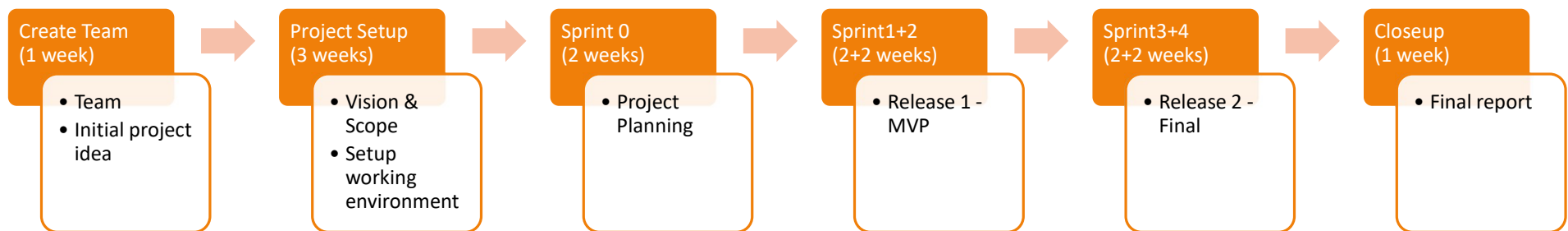
- The goal of this project is to build a software product following a well-defined development plan and quality assurance plan, within the expected scope, quality, budget and deadline.

► Theme

- The choice of the theme is up to the team; however, it must be approved by the teacher.

► Groups

- Group size shall be of 5 or 4 members, all proficient in Java programming language



1. Introduction

Why do projects fail?

“Software Project Management”

João Cunha, ISEC (jcunha@isec.pt)

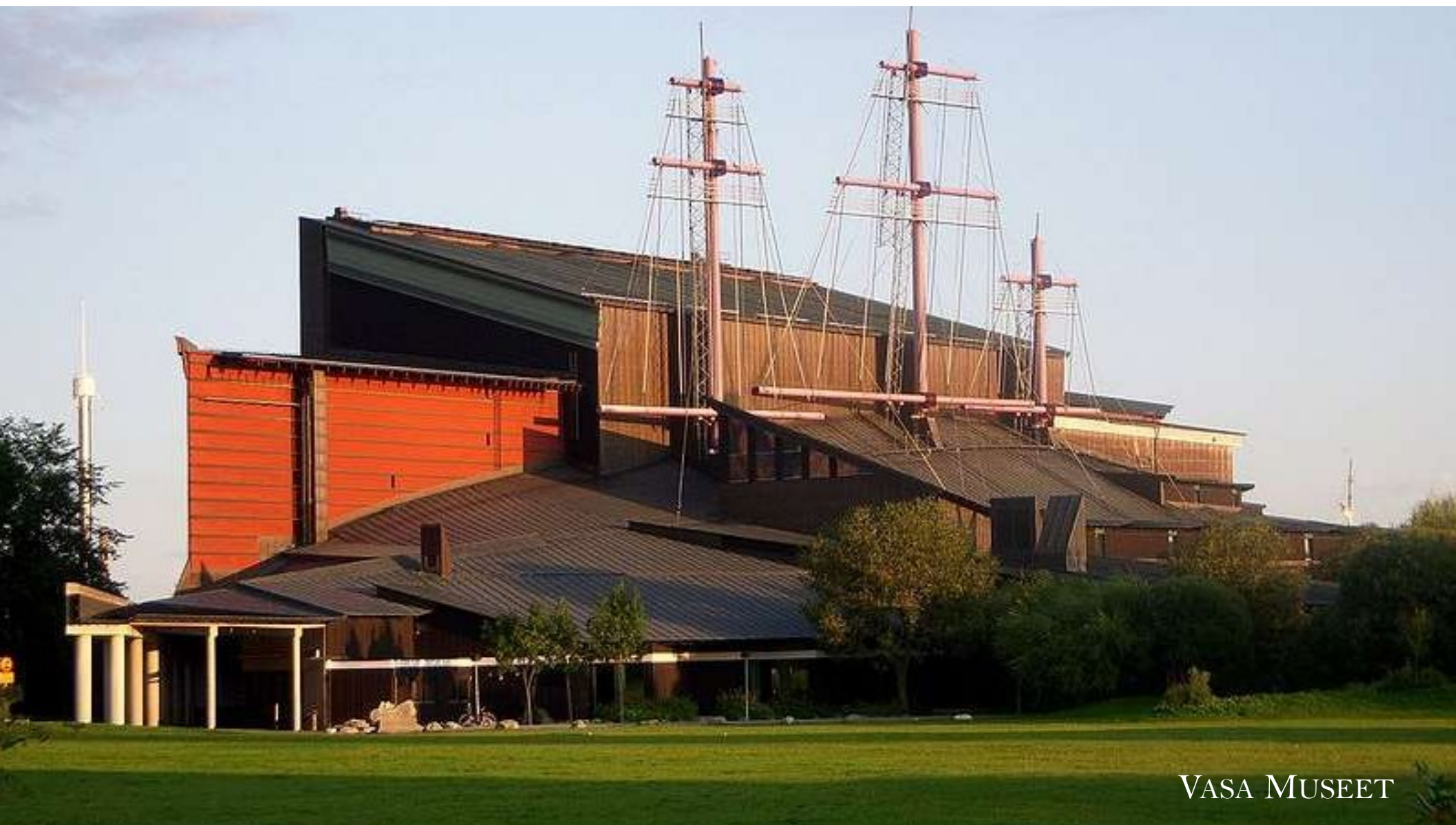
(Some slides from Andrew Stellman & Jennifer Greene,
“Applied Software Project Management”)

YEAR	COMPANY	OUTCOME (COSTS IN US \$)
2005	Hudson Bay Co. (Canada)	
2004-05	UK Ministry of Defense	
2004	Avis Rent A Car	
2004	Ford Motor Company	
2004	J. S.ainsbury plc	
2004	Hewlett-Packard	
2003-04	AT&T	
2002	McDonald's	
2002	Sydney Olympic Games	
2002	CIGNA	
2001	Nike	Supply-chain management system canceled after \$130 million is spent.
2001	Kmart	City payroll system abandoned after deployment costing \$25 million.
2000	Washburn	Administrative processing system canceled after \$12 million is spent.
1999	Unit	Tax system canceled after \$11.2 million is spent; state receives \$185 million damages.
1999	State of Mississippi	Problems with ERP system contribute to \$151 million loss.
1999	Hershey Foods Corp.	



The story of Vasa

- ▶ On 10 August 1628, the Royal Swedish Navy's newest ship set sail
- ▶ Vasa sailed about 1,300 meters and then, in a light gust of wind, capsized in Stockholm's harbor, losing 53 lives.
- ▶ The ship was Sweden's most expensive project ever.
- ▶ Sweden was at war with Poland and needed the ship for the war effort.
- ▶ A formal hearing the following month did not determine why the ship sank, and no one was blamed.
- ▶ The ship was removed from the waters, and a museum was built. The Vasa Museum is the most popular museum in Scandinavia – 45 million visitors since 1990



VASA MUSEET

What happened?

- ▶ King Gustav changed the shipbuilding orders
 - ▶ From 2 small and 2 big ships, to 4 larger ships, in reverse order, ...
- ▶ No specifications for the modified hull
 - ▶ Change from 111-feet keel to all-new 135 feet
- ▶ Shift in armament requirements
 - ▶ Upgrade to two enclosed gun decks, with heavier guns
- ▶ The shipwright died
- ▶ No way to calculate stability, stiffness, or sailing characteristics
- ▶ The prelaunch stability test... failed

Vasa... or software project problems?

1. Excessive schedule pressure
2. Changing needs
3. Lack of technical specifications
4. Lack of a documented project plan
5. Excessive and secondary innovations
6. Unclear requirements
7. Lack of scientific methods
8. Defects discovered too late
9. Unethical behavior

Why do software projects fail?

- ▶ **Jumping into coding too early**
 - ▶ Team starts programming before understanding the problem
 - ▶ Can produce software that works but solves the wrong problem
- ▶ **Underestimating the work**
 - ▶ Complex problems seem simpler than they are
 - ▶ Deadlines are often unrealistic
- ▶ **Late discovery of defects**
 - ▶ Requirements, design, or code errors found too late
 - ▶ Fixing late problems is expensive and risky
- ▶ **Poor programming habits**
 - ▶ Weak code control and testing
 - ▶ Code hard to understand or maintain
 - ▶ Little visibility on overall project health

How to Make Software Projects Succeed

- ▶ Understand the problem first
 - ▶ Define clear requirements and goals
 - ▶ Involve stakeholders early
- ▶ Plan realistically
 - ▶ Break work into manageable tasks
 - ▶ Set achievable deadlines
- ▶ Detect issues early
 - ▶ Review design and code frequently
 - ▶ Test continuously throughout development
- ▶ Follow good coding practices
 - ▶ Maintain clean, readable, and documented code
 - ▶ Use version control and code reviews
 - ▶ Monitor overall project health regularly
- ▶ Communicate effectively
 - ▶ Keep the team aligned and informed
 - ▶ Address risks and blockers immediately

Read the paper on Teams:
General->Files->External resources

Let's start by... Identifying the problem

- ▶ The business manager drives the scope of the project.
 - ▶ He should identify and talk to the main stakeholder
 - ▶ The effective way to show stakeholders that their needs are understood and that those specific needs will be addressed is with a ***vision and scope document***

Vision and Scope Document

- ▶ A typical vision and scope document follows an outline like this one:

1. Problem Statement

- a) *Project background*
- b) *Stakeholders*
- c) *Users*

2. Vision of the Solution

- a) *Vision statement*
- b) *List of features*
- c) *Scope of phased release (optional)*
- d) *Features that will not be developed*
- e) *Risks*
- f) *Assumptions*

Further reading

- ▶ Andrew Stellman and Jennifer Greene, *Applied Software Project Management*, O'Reilly, 2006. [Section 1 and 2]

Learning Agreement

- ▶ Read and confirm!

LEARNING AGREEMENT

ISEC, September 2025

*This **Learning Agreement** establishes the commitments between the **teachers** of the **Software Project Management** course and the **student**. It defines the learning outcomes, the activities to be developed, the support resources and the evaluation criteria. Its careful reading is essential for the student to understand the responsibilities he assumes and the conditions necessary to achieve exceptional results within the scope of this course.*

1 GENERAL INFORMATION

- **Course:** Software Project Management
- **Programme:** Degree in Computer Engineering (daytime and after-work), ISEC
- **Semester:** 5
- **Teaching Mode:** Face-to-face
- **Credits:** 6 ECTS
- **Working hours:** 160

TO DO for Week 2 (next Lab class) – team and project idea

1. Install Microsoft Teams

1. Add your **photo** to your profile in MTeams
2. Join the **GPS2526 Class** (code: **xsz5hv2**)
3. Explore MTeams, especially the GPS2526 Class
4. **Introduce** yourself to the class

2. Read and Accept the **Learning Agreement**

3. Enroll in your **Group**

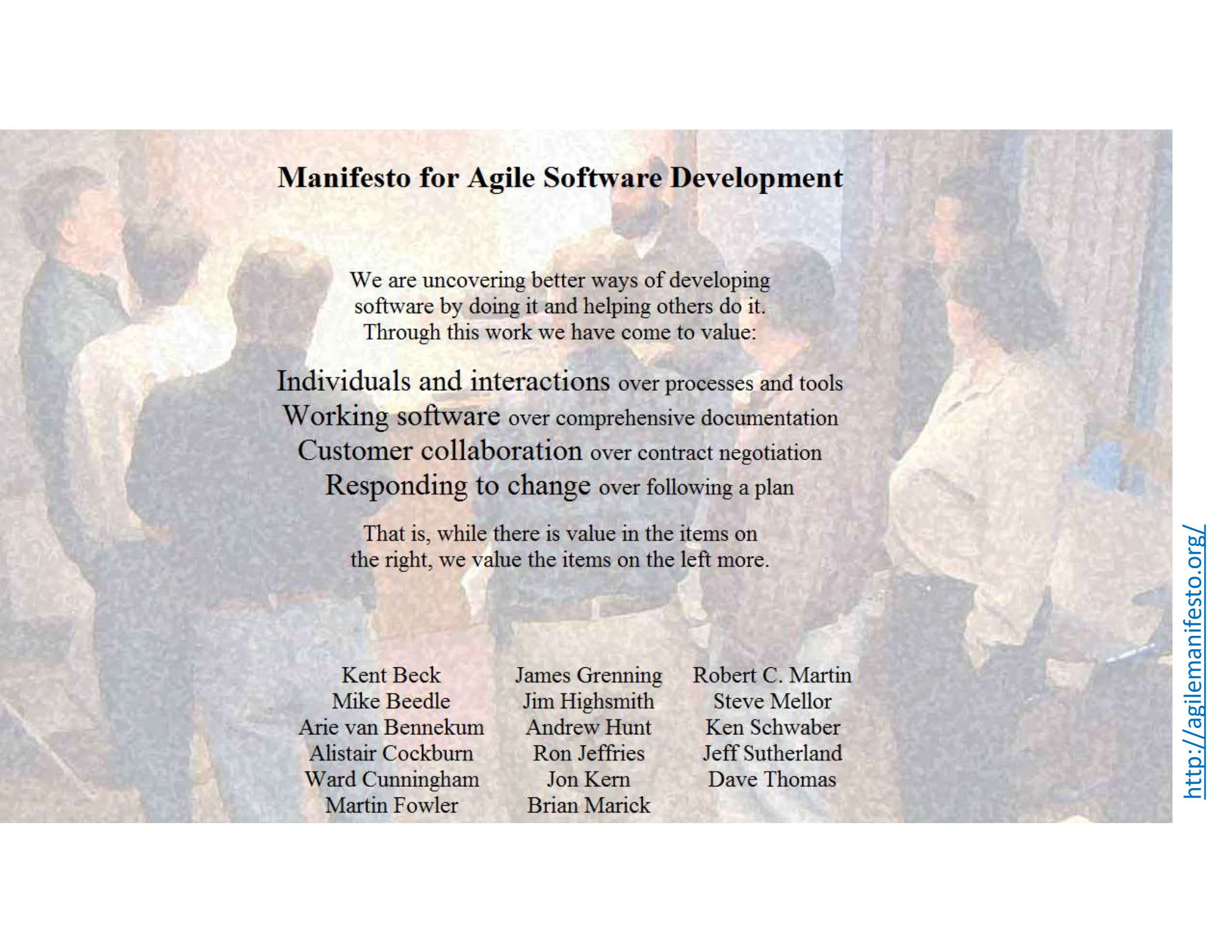
4. Discuss ideas for the project

1. Read Stellman [chapter 2] ([document on Teams](#))
2. Draft ideas on a **Vision & Scope draft** document ([template on Teams](#))

2. Agile Software Development

Embrace change

“Software Project Management”
João Cunha, ISEC (jcunha@isec.pt)



Manifesto for Agile Software Development

We are uncovering better ways of developing software by doing it and helping others do it.
Through this work we have come to value:

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

That is, while there is value in the items on the right, we value the items on the left more.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler

James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick

Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

A painting of a group of people in a meeting, with text overlaid.

Manifesto para o Desenvolvimento Ágil de Software.

Ao desenvolver e ao ajudar outros a desenvolver software,
temos vindo a descobrir melhores formas de o fazer.
Através deste processo começámos a valorizar:

Indivíduos e interacções mais do que processos e ferramentas
Software funcional mais do que documentação abrangente
Colaboração com o cliente mais do que negociação contratual
Responder à mudança mais do que seguir um plano

Ou seja, apesar de reconhecermos valor nos itens à direita,
valorizamos mais os itens à esquerda.

Kent Beck
Mike Beedle
Arie van Bennekum
Alistair Cockburn
Ward Cunningham
Martin Fowler

James Grenning
Jim Highsmith
Andrew Hunt
Ron Jeffries
Jon Kern
Brian Marick

Robert C. Martin
Steve Mellor
Ken Schwaber
Jeff Sutherland
Dave Thomas

The 4 Agile Values

- ▶ *We are uncovering better ways of developing software by doing it and helping others do it. Through this work we have come to value:*
 - ▶ **Individuals and interactions** over processes and tools
 - ▶ **Working software** over comprehensive documentation
 - ▶ **Customer collaboration** over contract negotiation
 - ▶ **Responding to change** over following a plan
-
- ▶ That is, while there is value in the items on the right, we value the items on the left more.

The 12 Agile Principles (1)

1. Our highest priority is to satisfy the customer through **early and continuous delivery** of valuable software.
2. Welcome **changing requirements**, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver **working software frequently**, from a couple of weeks to a couple of months, with a preference to the shorter time scale.
4. Business people and developers must **work together** daily throughout the project.
5. Build projects around **motivated individuals**. Give them the environment and support they need and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is **face-to-face conversation**.

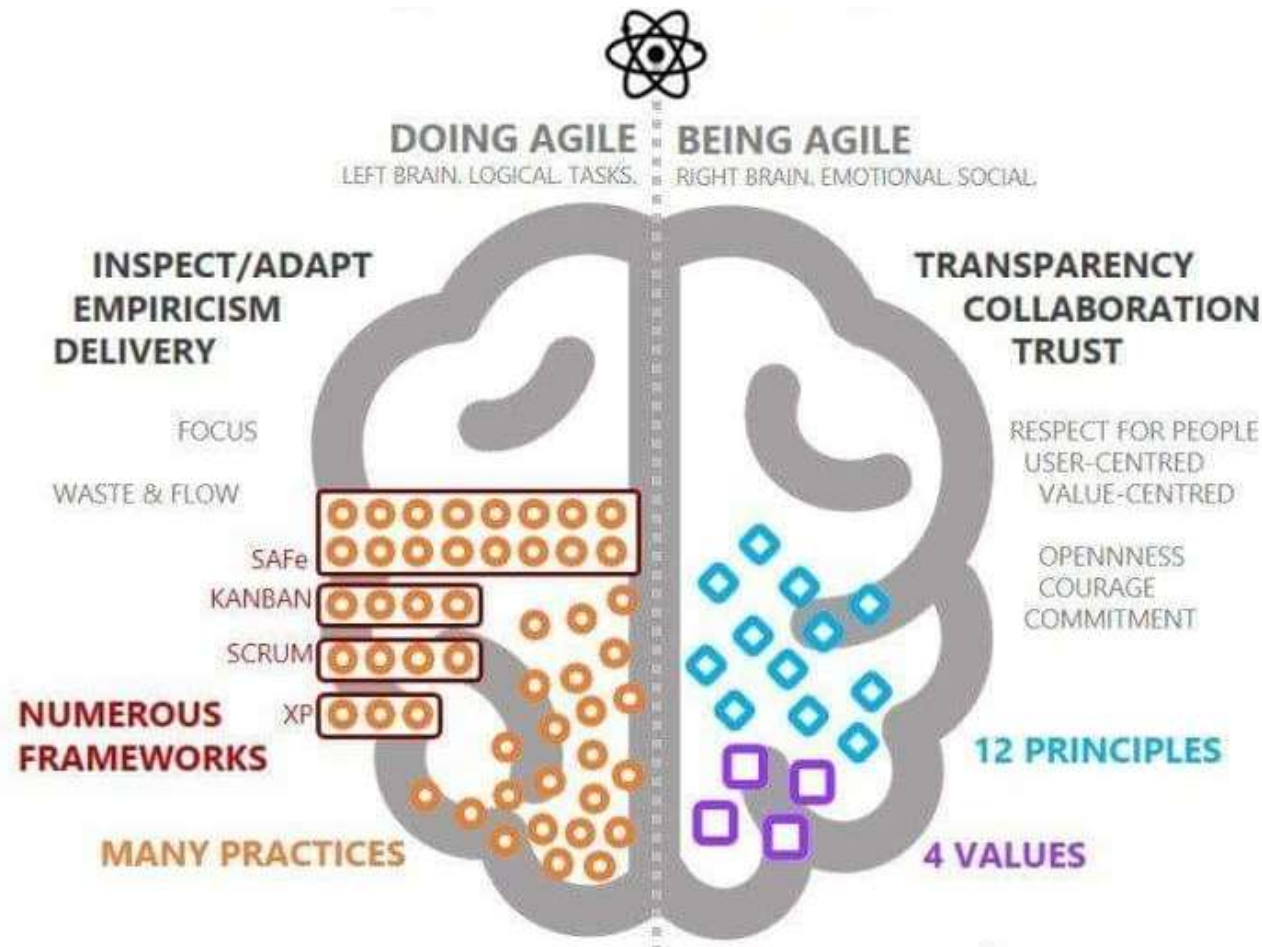
The 12 Agile principles (2)

- 7. **Working software** is the primary measure of progress.
- 8. Agile processes promote **sustainable development**. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
- 9. Continuous attention to **technical excellence** and good design enhances agility.
- 10. **Simplicity**—the art of maximizing the amount of work not done—is essential.
- 11. The best architectures, requirements, and designs emerge from **self-organizing teams**.
- 12. At regular intervals, the team reflects on **how to become more effective**, then tunes and adjusts its behavior accordingly.

What makes a method Agile?

- ▶ Short iterations
- ▶ Incremental design
- ▶ User involvement
- ▶ Minimal documentation
- ▶ Informal communication
- ▶ Adaptive planning
- ▶ Evolutionary delivery

Agile Mindset



Bibliography

- ▶ Agile Manifesto <https://agilemanifesto.org/>
- ▶ Agile Alliance <https://www.agilealliance.org/>

3. Scrum

It's about common sense

“Software Development Methodologies”
Ricardo Filipe, ISEC (ricardo.filipe@isec.pt)

SCRUM

- ▶ Scrum is an **iterative, incremental** method for managing the systems development.
- ▶ Emphasizes a set of project management **values and practices**.
- ▶ **Does not define any specific software development** technique for the implementation phase; it is a wrapper for existing engineering practices.
- ▶ Concentrates on **how the team members should function**, promoting self-directed teams and daily team measurement.
- ▶ Produces a **potentially shippable** set of functionalities at the end of every iteration.
- ▶ Has strong emphasis on **empirical** rather than defined process.

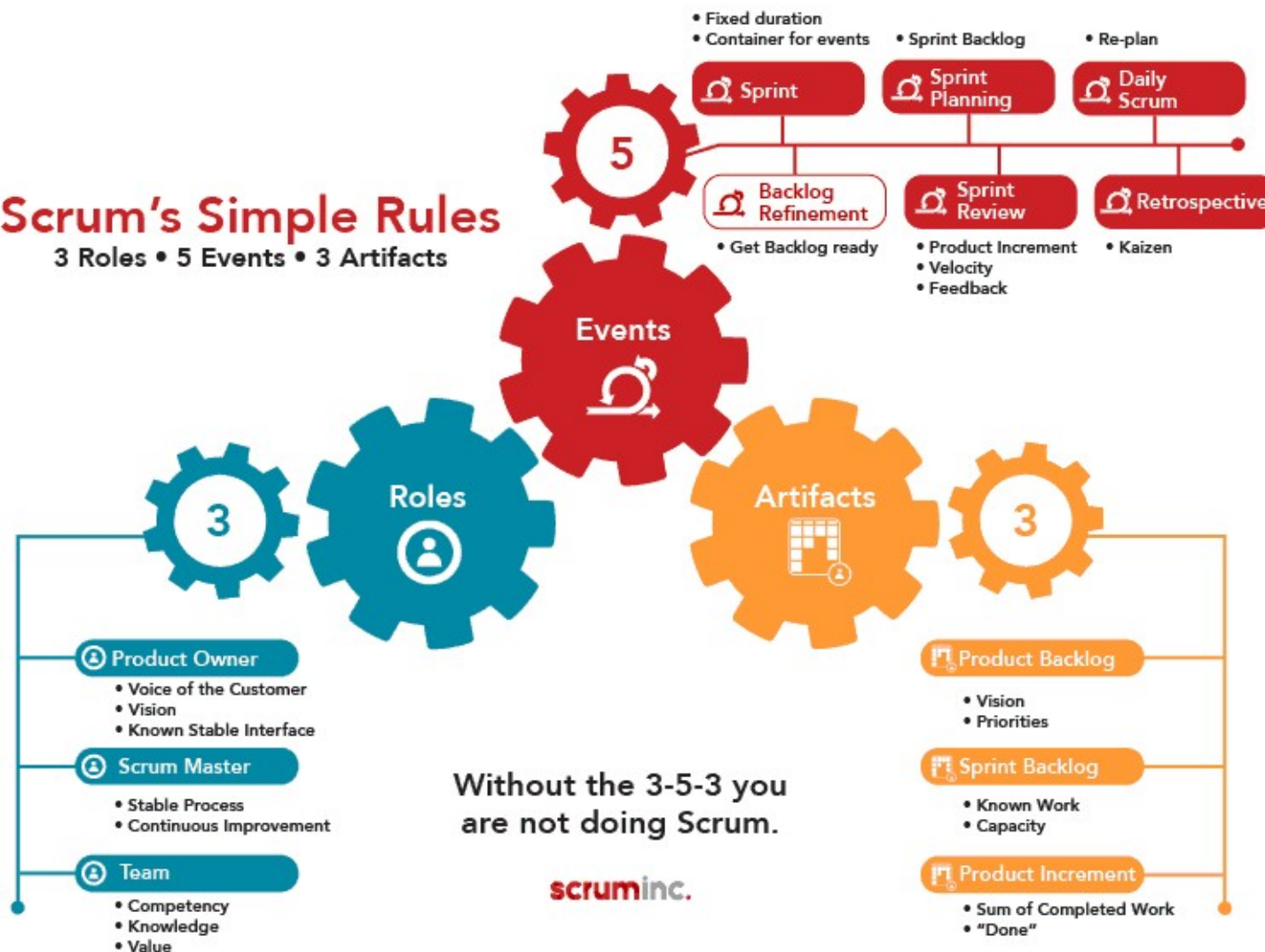
Key practices

- ▶ self-managing team
- ▶ no external addition of work to an iteration, once chosen
- ▶ daily stand-up meeting
- ▶ usually 2 to 4-week iterations
- ▶ demo to external stakeholders at end of each iteration
- ▶ client-driven adaptive planning



Scrum's Simple Rules

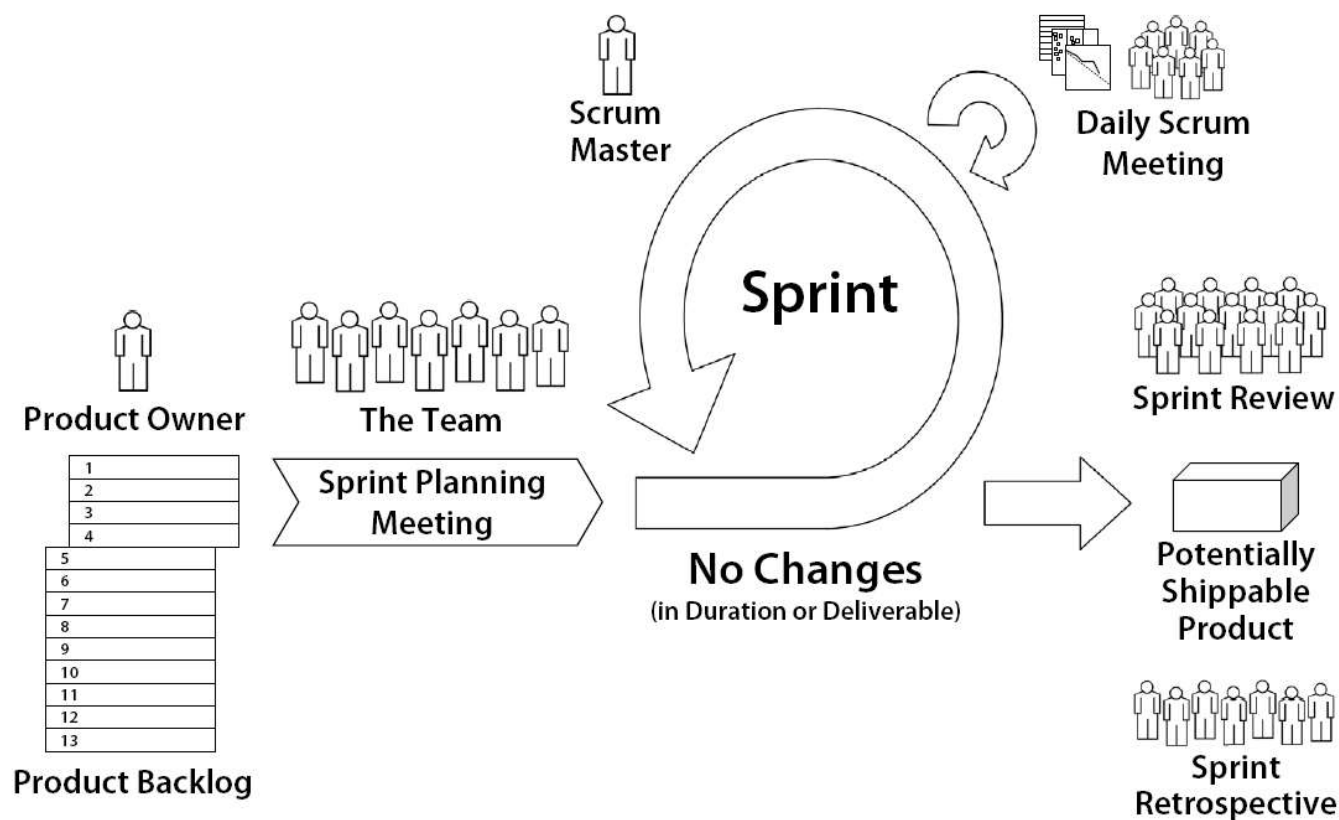
3 Roles • 5 Events • 3 Artifacts



Scrum framework

- ▶ **Roles**
 - ▶ Product Owner
 - ▶ Scrum Master
 - ▶ Team
- ▶ **Artifacts**
 - ▶ Product Backlog
 - ▶ Sprint Backlog
 - ▶ Product Increment
- ▶ **Events (Ceremonies)**
 - ▶ Sprint
 - ▶ Sprint Planning
 - ▶ Daily Scrum Meeting
 - ▶ Sprint Review
 - ▶ Sprint Retrospective
 - ▶ Backlog Refinement

Scrum Process flow



Scrum Lifecycle



Lifecycle

1. Pre-game (Sprint 0)

- ▶ Define vision and scope, prepare product backlog with estimates, run exploratory design and prototypes, prepare team, define release plan, plan next sprint, ...

2. Development

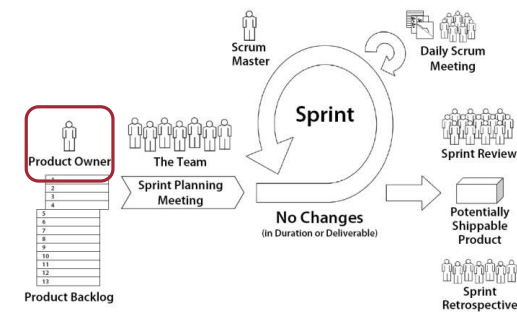
- ▶ Sprint planning, Sprint, Sprint review and retrospective

3. Release (Post-game)

- ▶ Deployment, documentation, training, ...

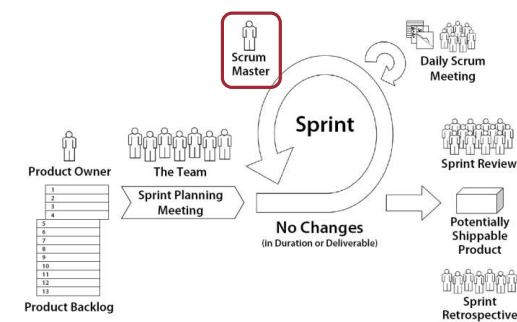
Roles: Product Owner

- ▶ Owns the vision of what should be produced
- ▶ Gets inputs from customers, end-users, team, managers, executives, industry experts, etc.
- ▶ Is responsible for creating and prioritizing the Product Backlog
- ▶ Monitors the project against the stated goals
- ▶ Only one person



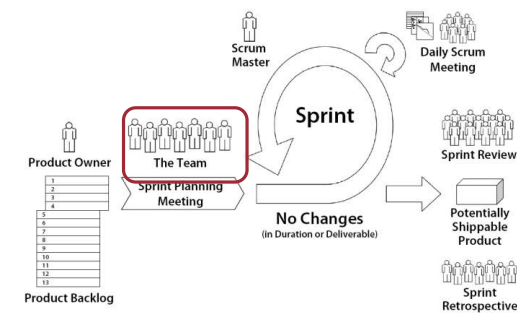
Roles: Scrum Master

- ▶ Responsible for setting the team up for success:
 - ▶ Establishes Scrum practices and rules
 - ▶ Shields the team from outside disturbances
 - ▶ Removes obstacles
- ▶ Represents management in the project
- ▶ Can be played by a team-member.
- ▶ Without a Scrum Master, the team has a high risk of failure.



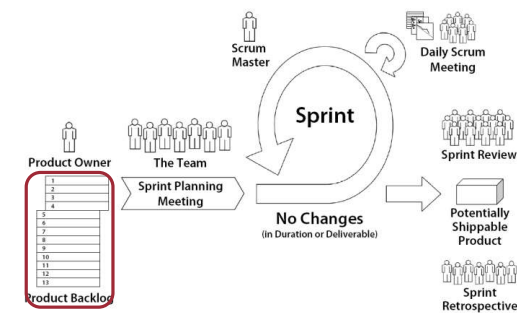
Roles: The Development Team

- ▶ Until 10 people
- ▶ Cross-functional with undifferentiated roles
 - ▶ It has all the skills to produce finished product – designers, coders, testers, etc.
 - ▶ Everyone contributes based on competency, rather than just job title
- ▶ Self-organizing and self-managing
- ▶ Responsible for making a commitment and managing itself to hit the goal
- ▶ Authority to do whatever is needed to meet the commitment



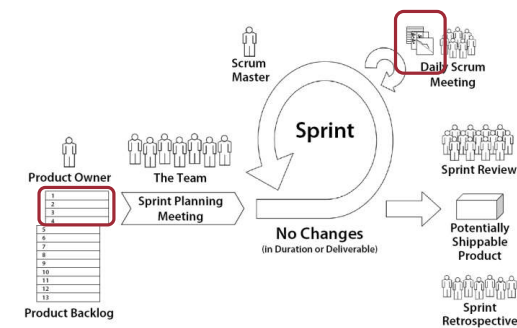
Artifacts: Product Backlog

- ▶ Prioritized list of work to be performed on a product
 - ▶ Requirements oriented or based on user stories
- ▶ Anyone can contribute to backlog items
 - ▶ Product Owner is responsible
- ▶ Product Owner is responsible for prioritization
 - ▶ Focus on business value (80% of the value comes from 20% of the features)
 - ▶ Reviewed at the start of each sprint or during Backlog Refinement



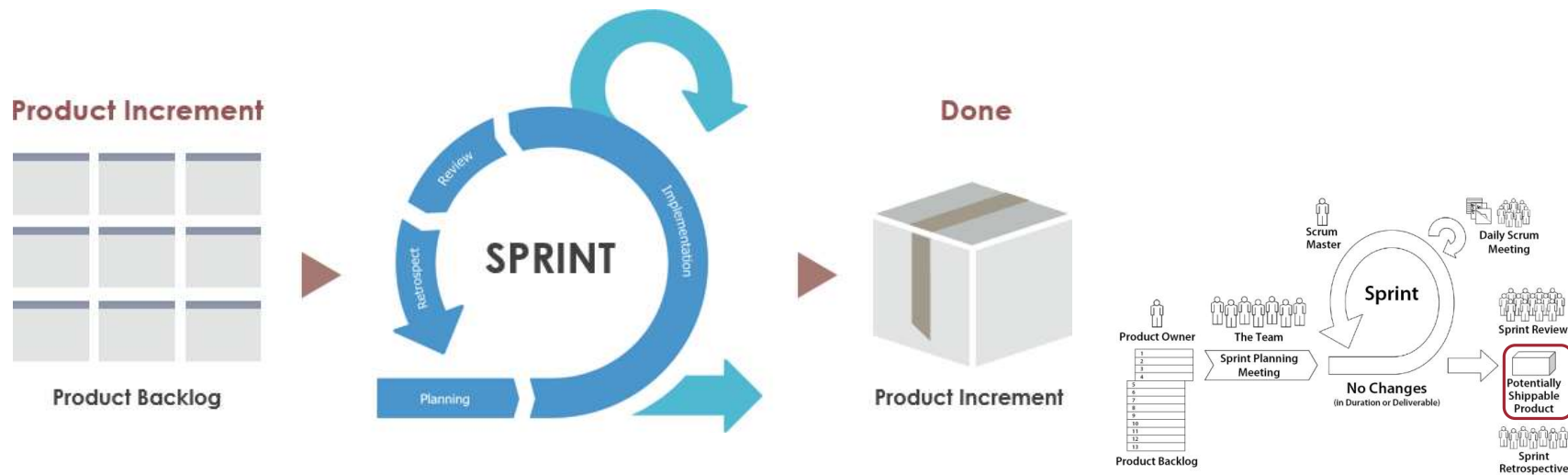
Artifacts: Sprint Backlog

- ▶ Lists of the tasks to be executed during the sprint
 - ▶ Selected during the Sprint Planning
- ▶ Status of the tasks updated daily



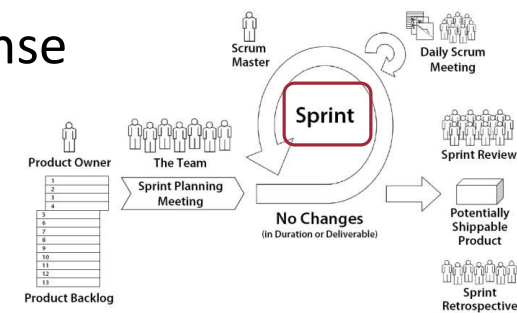
Artifacts: Product Increment

- ▶ The (working) result at the end of each sprint
- ▶ The sum of all the product backlog items completed during a sprint plus the value of the increments of all previous sprints
- ▶ The result must work (potentially shippable/releasable) – no presentations.



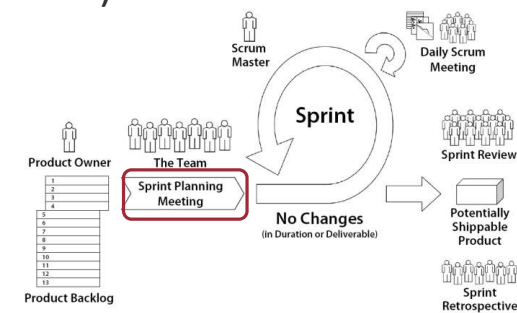
Events: Sprint

- ▶ Fixed period iteration
 - ▶ usually between 2- and 4-weeks
- ▶ Includes design, coding, testing, and documentation
- ▶ Created “done” work – According to acceptance criteria
- ▶ During the Sprint, the Scrum Team keeps committed to the goals and end-date
- ▶ The Scrum Team is fully responsible for the work
 - ▶ Only the Scrum Team can add or remove items in the Sprint backlog
- ▶ Abnormal termination of Sprint may be called by the PO when the Sprint Goal no longer makes sense
- ▶ Sprints occur one after another, without any down-time between them.



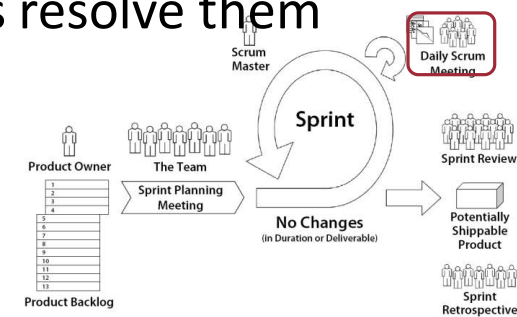
Events: Sprint Planning Meeting

- ▶ 2-part meeting to define the next Sprint
- ▶ Sprint prioritization
 - ▶ Product Owner leads, all Stakeholders are present
 - ▶ Analyze and evaluate product backlog
 - ▶ Select Sprint goal
- ▶ Sprint planning
 - ▶ Scrum Team and Product Owner (optional)
 - ▶ Decide how to achieve sprint goal (design)
 - ▶ Create Sprint Backlog (tasks) from Product Backlog (user stories)
 - ▶ Answer
 - ▶ **Why** is this Sprint valuable?
 - ▶ **What** can be Done this Sprint?
 - ▶ **How** will the chosen work get done?
 - ▶ Estimate Sprint Backlog



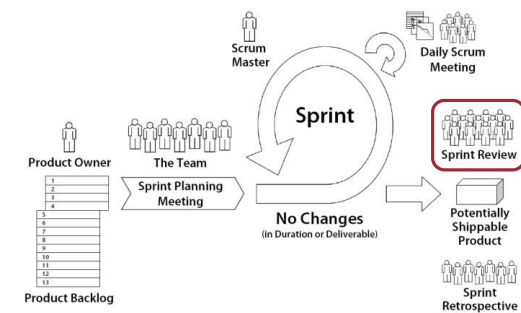
Events: Daily Scrum Meeting

- ▶ Short meeting for **Developers** to update each other on progress and problems
 - ▶ Daily, stand up, 15 minutes
 - ▶ Usually same place, same time, at the beginning of the day
- ▶ Everyone answers 3 questions (not necessarily):
 - ▶ What have you done since last daily Scrum?
 - ▶ What will you do between now and the next daily Scrum?
 - ▶ What is getting in the way (blocks) of meeting the iteration goals?
- ▶ Scrum Master notes blocks, and afterwards helps resolve them
 - ▶ Not for problem solving
 - ▶ Other follow-up meetings can be held



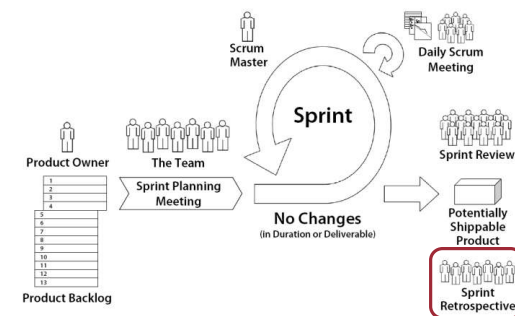
Events: Sprint Review Meeting

- ▶ At the end of the Sprint.
- ▶ Product Owner, Developers, Scrum Master, and all Stakeholders.
- ▶ Demo of what the team has produced.
- ▶ The **Product Owner** presents backlog items, gathers feedback from everyone on ways to improve what's been built
 - ▶ This feedback is incorporated into the Product Backlog
- ▶ Max 4 hours.
- ▶ Goal: Inspect product increment.



Events: Sprint Retrospective Meeting

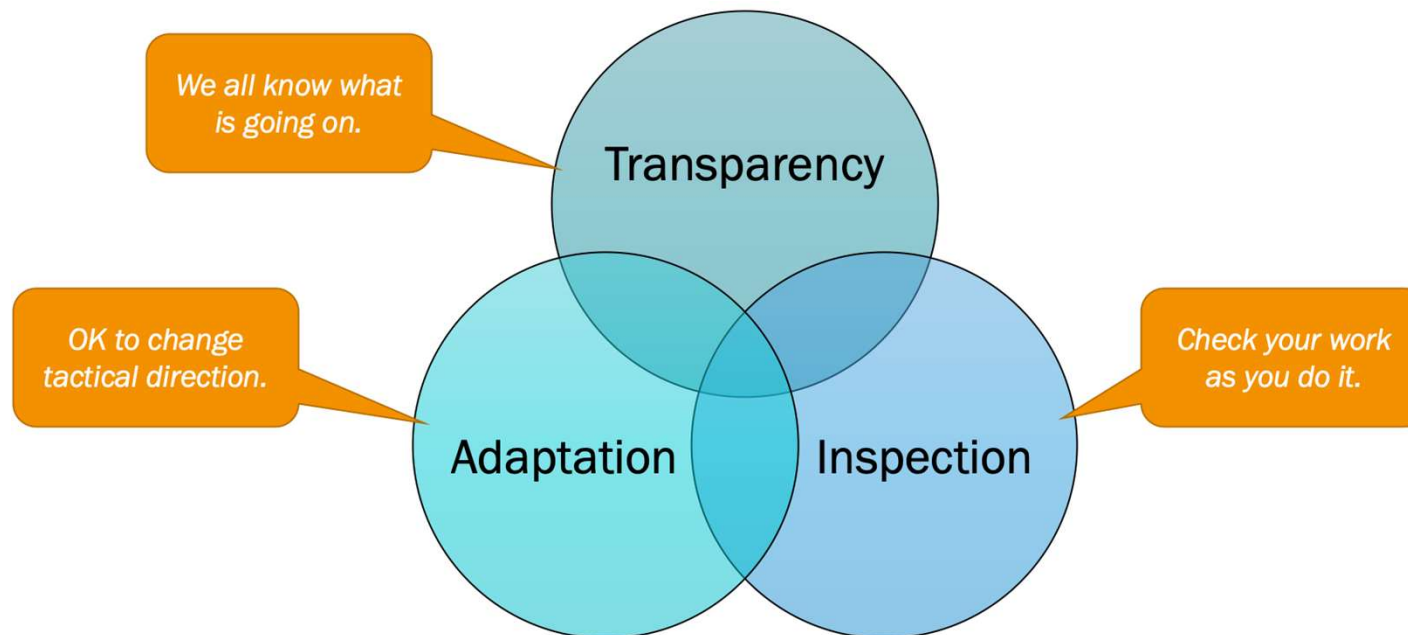
- ▶ Also at the end of the Sprint
- ▶ The Developers, Product Owner, and Scrum Master
- ▶ Goal: review the way of working and look for ways to improve their effectiveness.
- ▶ Mechanism for continuous improvement:
 - ▶ What went wrong, what went right, what can be done better
 - ▶ Which tools, processes, practices can be used to make it better, more productive and enjoyable



Events: Backlog Refinement

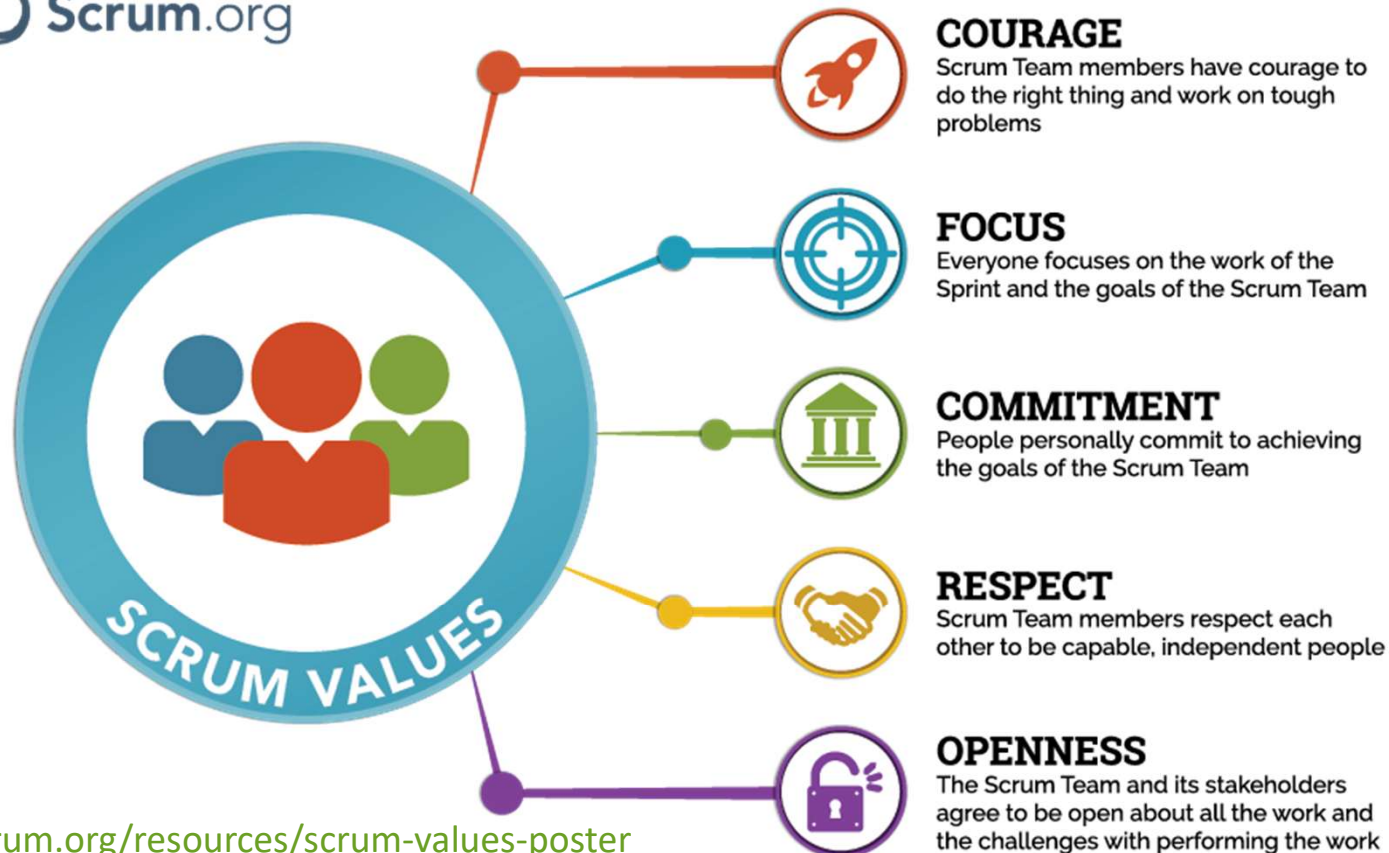
- ▶ Happens regularly, usually mid-sprint
- ▶ Make backlog items clear and easy to understand
- ▶ Break down big items into smaller and more precise ones
- ▶ Estimate size and effort
- ▶ Keep the backlog ready for the next sprint

Empirical Scrum Pillars



<https://www.scrum.org/resources/blog/three-pillars-empiricism-scrum>

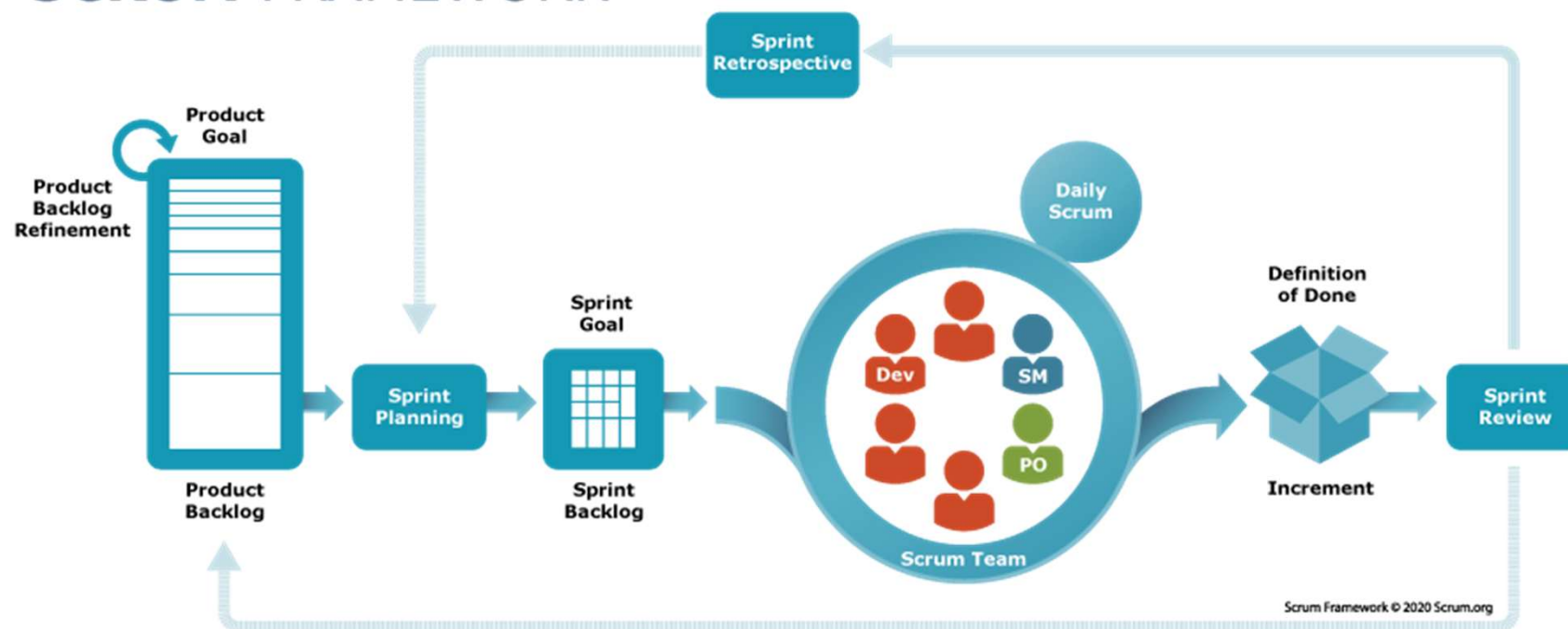
Scrum Values



<https://www.scrum.org/resources/scrum-values-poster>

Scrum Framework

SCRUM FRAMEWORK



<https://www.scrum.org/resources/scrum-framework-poster>

Bibliography

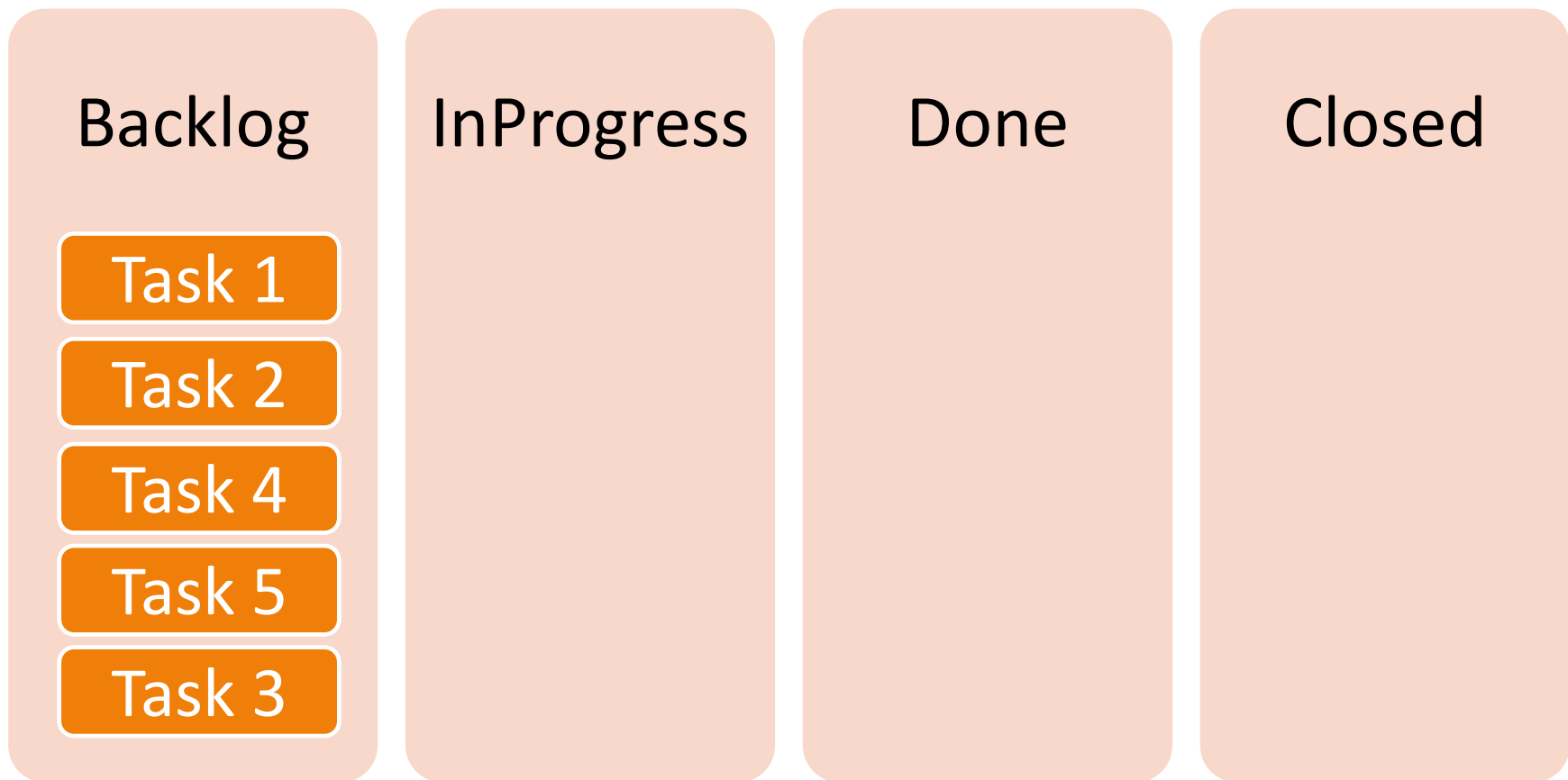
- ▶ <http://www.scrumalliance.org>
 - ▶ <https://www.scrum.org/>
 - ▶ <https://scrumguides.org/scrum-guide.html>
 - ▶ Scrum Essentials in Under 10 Minutes (<https://www.youtube.com/watch?v=RtQ3tpq-RuE>)
 - ▶ Intro to Scrum in Under 10 Minutes (<https://www.youtube.com/watch?v=XU0lIRltyFM>)
-
- ▶ Some figures from the slides were taken from these links

4. Pre-Game (Sprint 0)

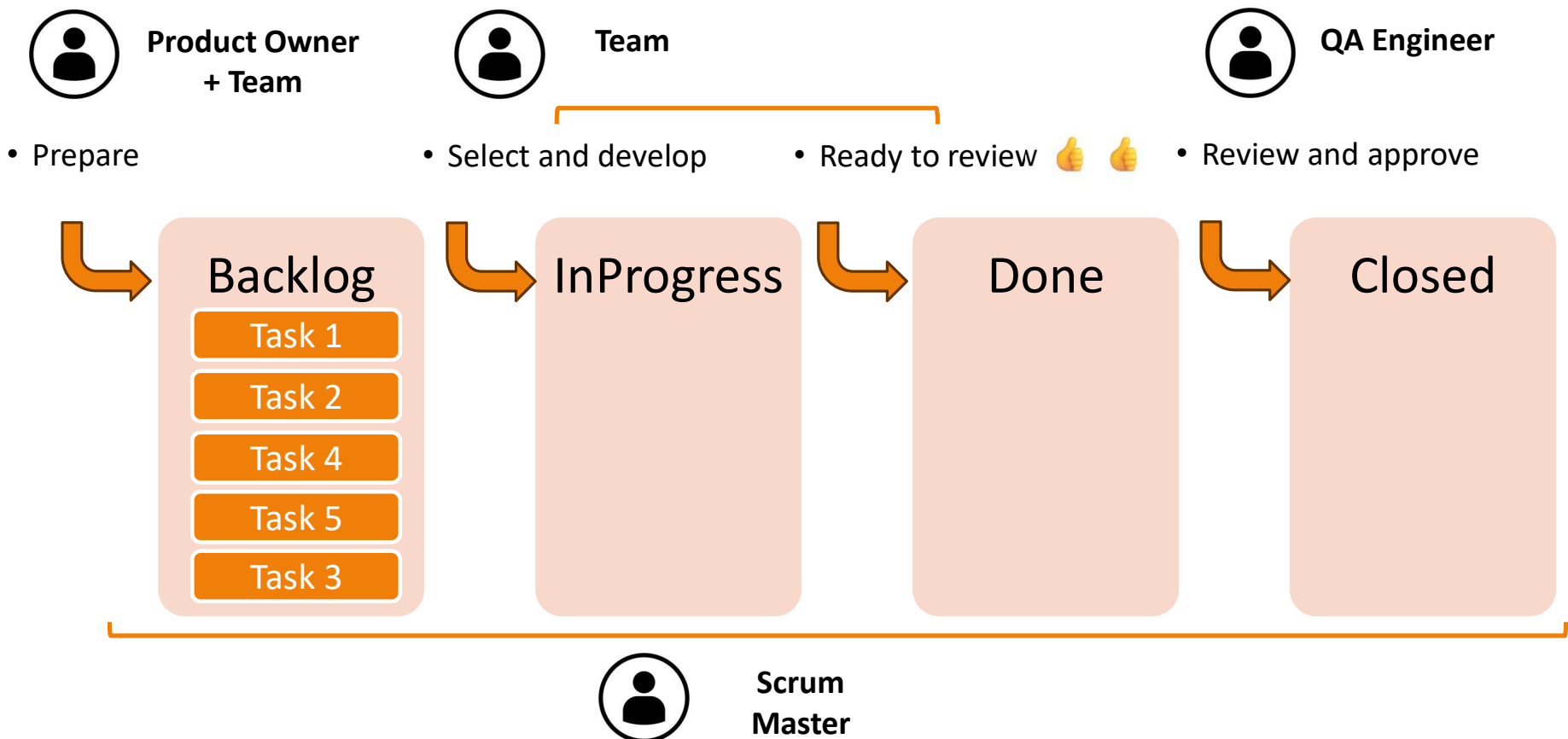
Pre-Game (or Sprint 0)

- ▶ Happens before the first real Sprint – so the first Sprint can deliver value
- ▶ Goal: prepare the team and the project
- ▶ Define vision, scope, and high-level product backlog
- ▶ Clarify architecture, conduct research and spikes
- ▶ Set up tools and environments
- ▶ Clarify roles and responsibilities
- ▶ Establish initial “Definition of Done”

Sprint 0 Backlog



Pre-Game – Sprint 0 Backlog, Roles and Responsibilities



README.md – Pre-Game section

▶ Sprint 0 Plan

- ▶ **Goal:** Setup the project foundation to start delivering value from Sprint 1
- ▶ **Dates:** XX/Oct - XX/Oct, 2 weeks
- ▶ **Team capacity:** XX hours (2 Weeks * X Team Elements * 4 Hours/Week)
- ▶ **Roles:**
 - Product Owner:** name
 - Scrum Master:** name
 - QA Engineer:** name
- ▶ **KOM (Sprint 0 Plan):** <link to file>
- ▶ **Analysis:** *short analysis of the KOM*

▶ Sprint 0 Backlog:

- Task 1 – Define Release Plan
- Task 2 – Write User Stories, estimate, prioritize
- Task 3 – Create Product Backlog (with ordered US)
- Task 4 – Establish Coding Standards
- Task 5 – Consolidate Architecture & Design
- Task 6 – Write initial DoD
- Task 7 – Plan Sprint 1
- Task 8 – Prepare Sprint 0 Review and Retrospective
- Task 9 – Create repository with “GPS Git” Workflow
- Task 10 – Create Spike for ...



5. Kick-off Meeting (KOM)

What is a KOM?

- ▶ A project kick-off meeting is the first meeting between a project manager, the team and the client of a new project
- ▶ It takes place after agreement with client and first planning activities
- ▶ Its purpose is to introduce the team, understand the project background, its goals, what needs to be done, and the general plan

KOM example typical agenda

1. Introductions – meet the team
2. Goal – problem and vision of the solution
3. Scope – what will be done
4. Quality – quality goals and how to achieve them
5. Architecture – what's the solution
6. Teamwork – how the team will work together
7. Q&A

Sprint 0 Planning Meeting

- ▶ Presented by Product Owner
- ▶ Define Sprint goal
 - ▶ **Why** is this Sprint valuable?
- ~~▶ Select items from Product Backlog~~
 - ~~▶ **What** can be Done this Sprint?~~
- ▶ Create Sprint Backlog
 - ▶ **How** will the chosen work get done?
 - ~~▶ Estimate Sprint Backlog~~

KOM Slides

KOM – Kick-off Meeting
x/October/2025

Group XX – project name

1

©2025 Group XX - project name



6. Release Plan

Product Goal

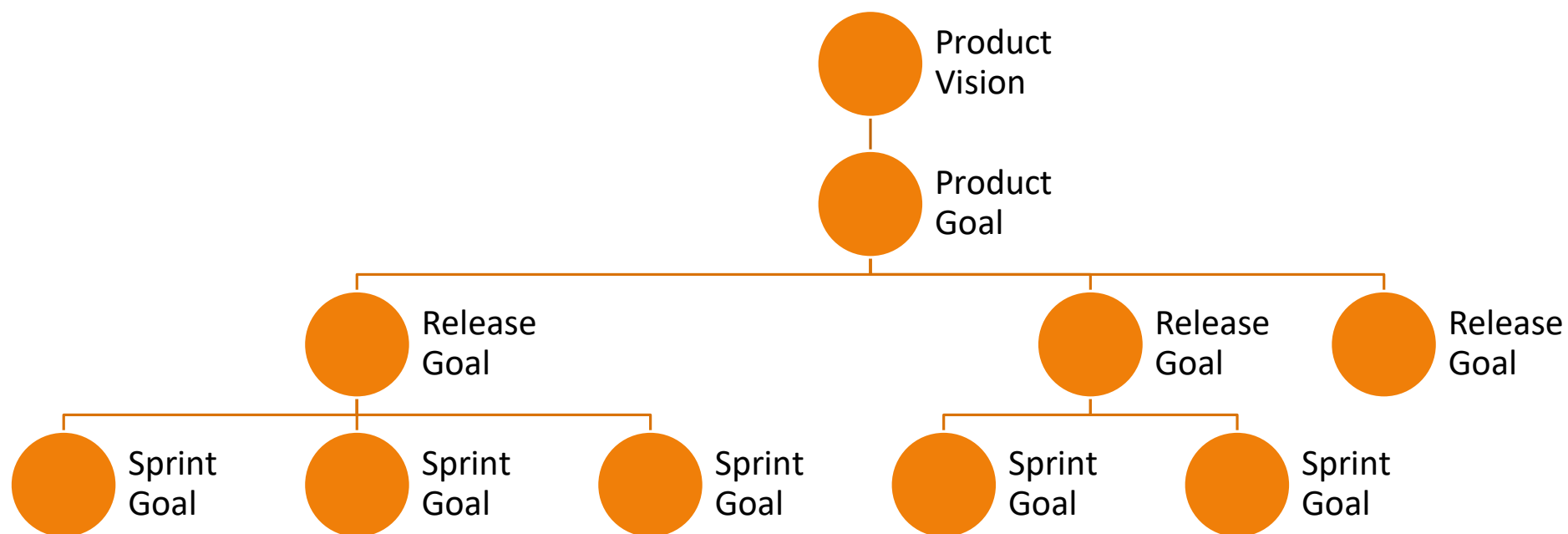
- ▶ *The Product Goal describes a future state of the product which can serve as a target for the Scrum Team to plan against.*
 - ▶ The Scrum Guide
- ▶ Single long-term objective for the product
 - ▶ The Team must fulfill (or abandon) one objective before taking on the next
- ▶ Guides prioritization of the Product Backlog
- ▶ Helps set expectations for stakeholders
- ▶ Focus on value delivery

Release Goals

- ▶ Deconstruct the product goal into smaller, achievable release goals
- ▶ A release is the delivery of a usable product increment
- ▶ Provides value and moves closer to the Product Goal
- ▶ Each release serves as a milestone on the path to the product's ultimate objective

Vision, Product Goal, Release Goal

- ▶ Vision
 - ▶ *Why are we building this?*
- ▶ Product Goal
 - ▶ *Where do we want this product to be?*
- ▶ Release Goal
 - ▶ *What do we want to achieve with this release?*



Minimum Viable Product (MVP)

- ▶ First release, to validate assumptions
- ▶ It is the simplest version of a product that is needed to build to sell it to a market
- ▶ *The version of a new product which allows a team to collect the maximum amount of validated learning about customers with the least effort*
 - ▶ Lean Startup

Examples with 2 releases

▶ **Note-Taking App**

- ▶ **Vision:** Make studying easier for students
- ▶ **Product Goal:** Let students take and organize digital notes
- ▶ **MVP:** Basic app to write and save notes
- ▶ **Final Release:** Add folders, search, sharing, and sync across devices

▶ **Food Delivery App**

- ▶ **Vision:** Help people order meals quickly
- ▶ **Product Goal:** Allow users to order food from local restaurants
- ▶ **MVP:** Simple app to order from 1–2 restaurants
- ▶ **Final Release:** Full app with many restaurants, tracking, and payment options

README.md – Release section

▶ **Product Goal:**

- ▶ Description
- ▶ <Link to Product Backlog>

▶ **Release 1**

- ▶ **Goal:** MVP - description
- ▶ **Date:** XX/Nov
- ▶ **Release:** V1.0

▶ **Release 2**

- ▶ **Goal:** description
- ▶ **Date:** XX/Dec
- ▶ **Release:** V2.0

7. Product Backlog and User Stories

Product Backlog

- ▶ Prioritized list of work to be performed to achieve the **Product Goal**
 - ▶ Requirements oriented or based on **user stories**
- ▶ Anyone can contribute to backlog items
 - ▶ Product Owner is responsible
- ▶ Product Owner is responsible for prioritization
 - ▶ Focus on business value (80% of the value comes from 20% of the features)
 - ▶ Reviewed regularly

Agile User Stories

- ▶ A user story is an informal, general explanation of a software feature written from the perspective of the end user
- ▶ Its purpose is to articulate how a software feature will provide value to the end user
- ▶ User story is a description of a specific customer requirement:
 - ▶ **RGB** format: Role; Goal; Benefit
 - ▶ **As a** <type of user> , **I want** <some goal> **so that** <some reason/benefit>

Agile User Stories

▶ **As a** <type of user> , **I want** <some goal> **so that** <some reason/benefit>

▶ Type of user:

- ▶ Who are we building this for?
- ▶ Who is the persona?

▶ Some goal:

- ▶ What is the intention?
- ▶ What are they really trying to achieve?

▶ Some reason:

- ▶ What's the overall benefit?
- ▶ What is the big problem that needs to be solved?

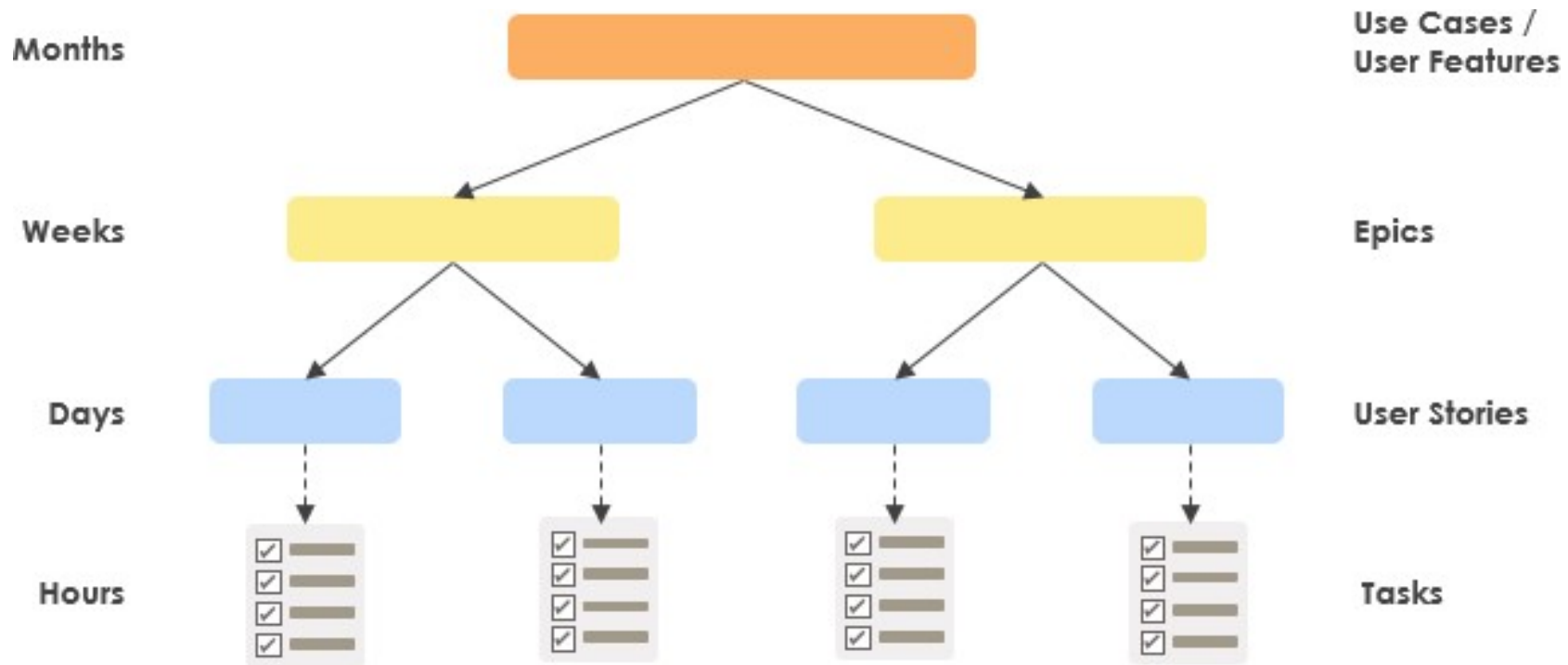
User Stories examples

- ▶ **As a student, I want to** reset my password **so that** I can access my account if I forget it.
- ▶ **As a shopper, I want to** add items to a cart **so that** I can buy multiple products at once.
- ▶ **As a project manager, I want to** see a dashboard of team tasks **so that** I can track progress easily.

INVEST – How a good user story should be

- ▶ **I** - Independent (of all others)
- ▶ **N** - Negotiable (not a specific contract for features)
- ▶ **V** - Valuable (or vertical)
- ▶ **E** - Estimable (to a good approximation)
- ▶ **S** - Small (so as to fit within an iteration)
- ▶ **T** - Testable (in principle, even if there isn't a test for it yet)

Use Cases, Epics, User Stories, Tasks



Product Backlog

- ▶ Can contain any type of work
 - ▶ user stories, tasks, bug fixes, spikes, etc.
- ▶ One single list for the product
- ▶ Starts high-level, refined continuously
 - ▶ During **Backlog Refinement** or **Sprint Review**
 - ▶ Clarified, split, estimated, re-ordered
- ▶ Ordered by priority/value
- ▶ Estimated
- ▶ Managed by the Product Owner
- ▶ Must be transparent and visible to the whole team

Product Backlog

US 1

US 5

Task2

Fix 1

US 3

Further Reading

- ▶ How to write a Good User Story
<https://www.youtube.com/watch?v=P3hbbKnzm4A>
- ▶ <https://www.atlassian.com/agile/project-management/user-stories>
- ▶ <https://www.agilealliance.org/glossary/user-stories/>



8. Estimation and Prioritization

Software Estimation

▶ What is software estimation?

- ▶ Process of predicting effort, time, or cost of software work
 - ▶ It is **not** a commitment or a plan to meet a target

▶ Why estimate?

- ▶ Supports contract negotiation
- ▶ Supports planning and scheduling
 - ▶ Helps allocate resources, set priorities and control
- ▶ Sets expectations and builds trust with stakeholders
 - ▶ Realistic expectations build trust between stakeholders and the team

▶ “... determine whether a project's targets are realistic enough to allow the project to be controlled to meet them.”

- ▶ *Source: Software Estimation – Demystifying the Black Art, 2006*

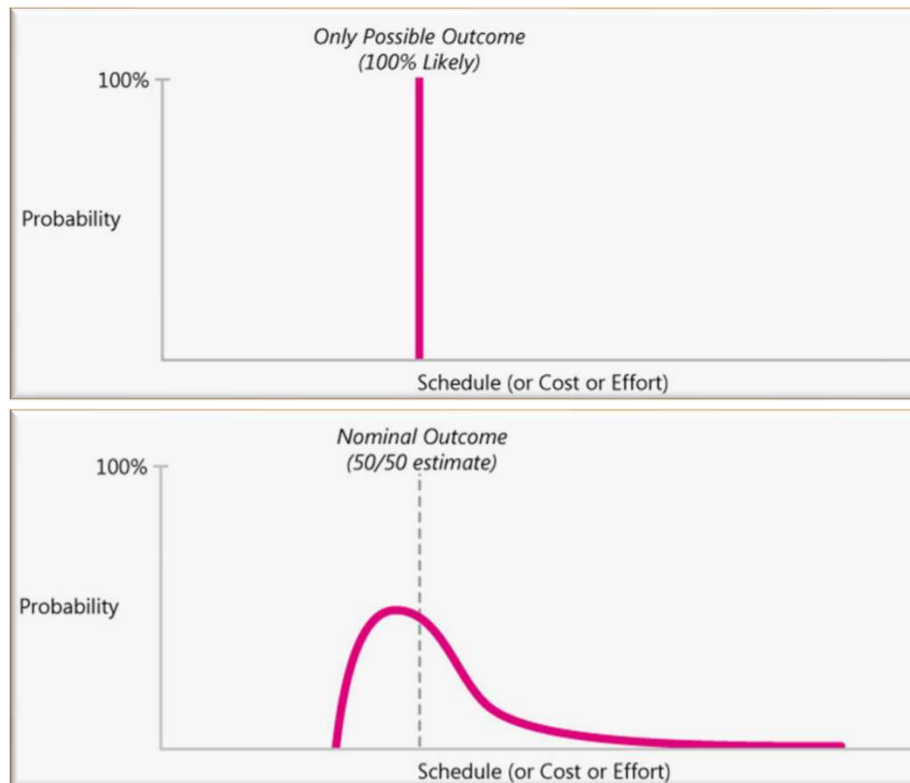
Influencing factors

- ▶ Which factors did you take in consideration?
 - ▶ Confidence level
 - ▶ Knowledge base/ historical data
 - ▶ Personal experience
- ▶ For the travelling time estimation
 - ▶ Transport (train, car, bus, ...)
 - ▶ Route (highways, secondary roads, ...)
 - ▶ Traffic hours
 - ▶ Complying with driving rules (speed limit, ...)
 - ▶ Resting/eating pauses
 - ▶ Unexpected conditions (weather, accidents, ...)
- ▶ Did you make any assumptions?
- ▶ Did you consider several options?

Assumptions Make Estimates More Accurate

- ▶ Team members use assumptions to work with incomplete information
 - ▶ Estimates rely on decisions not yet made; team members assume answers to proceed.
 - ▶ Documenting assumptions help trace down and understand inaccuracies.
 - ▶ Early assumptions align the team and help progress on key decisions affecting development.

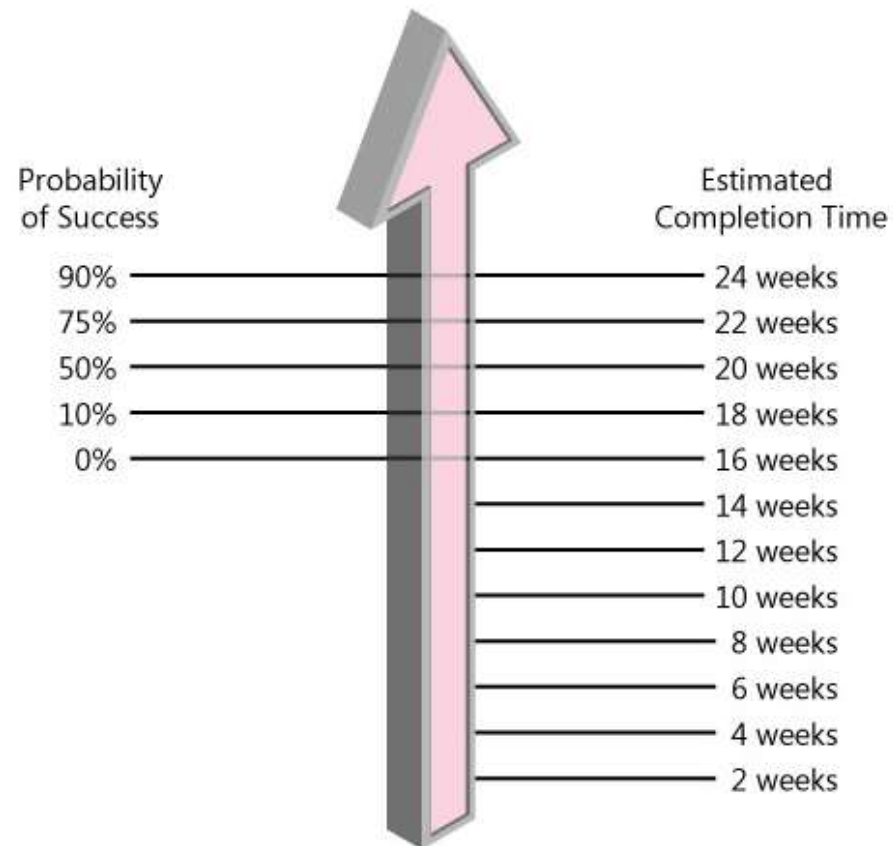
Estimation as probability statements



Single-point estimates assume 100% probability of the actual outcome equaling the planned outcome
UNREALISTIC!

Project outcomes: There is a limit to how well a project can go but no limit to how many problems can occur
REALISTIC!

Estimation as probability statements



Main issues when estimating

- ▶ System complexity
 - ▶ Non-repeatable work
- ▶ Bias toward underestimation
 - ▶ Overestimation -> Parkinson's Law
- ▶ Estimates treated as goals
 - ▶ Manipulated estimates to achieve the goals
- ▶ Inaccurate information about team capabilities
 - ▶ Missing or unused historical data
- ▶ Unclear requirements
 - ▶ Stakeholders with different understandings
 - ▶ Hidden assumptions

Steps for of a Sound Estimate

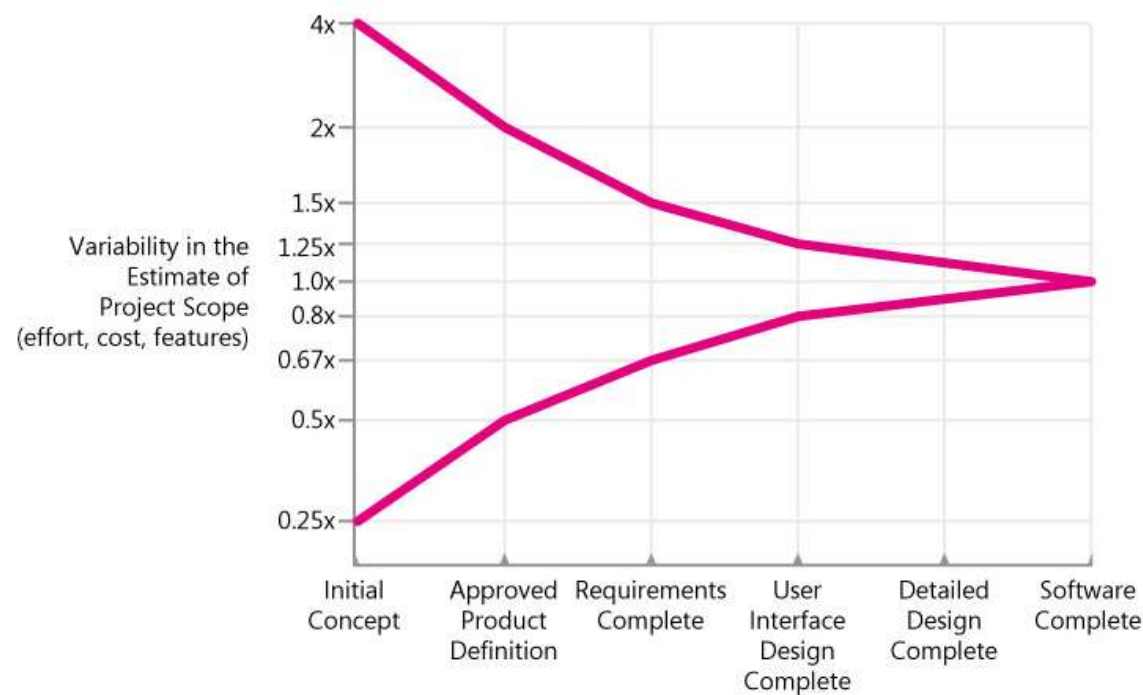
- ▶ Divide the work into items:
 - ▶ Work Breakdown Structure (WBS)
 - ▶ Product Breakdown Structure (PBS)
 - ▶ Task list
- ▶ Estimate each item
 - ▶ Effort
 - ▶ Time
 - ▶ Cost
- ▶ Write assumptions used in the estimates
- ▶ Reach team consensus that the estimate is realistic

How to estimate?

- ▶ Estimation by Analogy
 - ▶ Affinity estimation
- ▶ Parametric estimation
 - ▶ COCOMO, COCOMO II, Function Points
- ▶ Expert judgement
 - ▶ Wideband Delphi
 - ▶ Planning Poker

When to estimate?

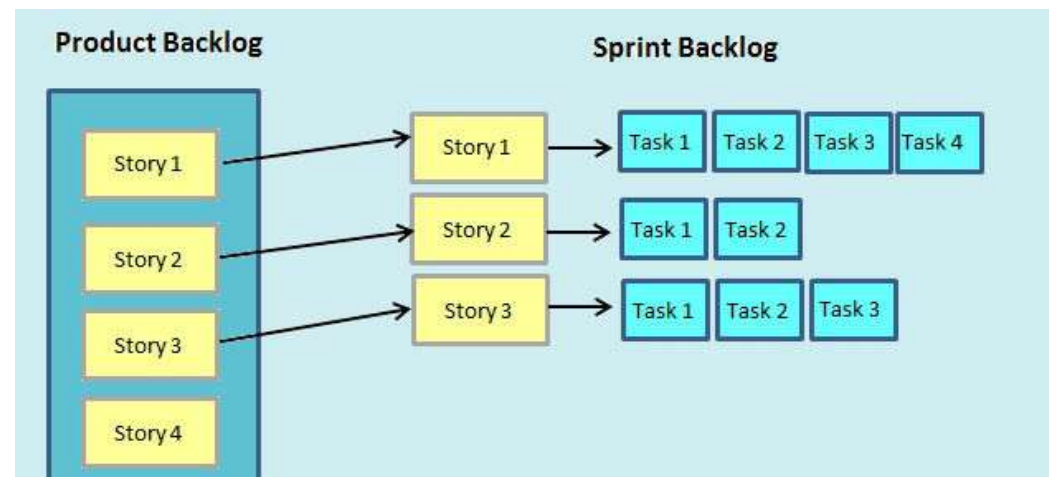
► The Cone of Uncertainty



Source: *Software Estimation – Demystifying the Black Art*, 2006

Estimating in Agile

- ▶ **Big and distant work:**
 - ▶ Artifact: **Product Backlog**
 - ▶ Content: **User Stories**
 - ▶ Technique: **Affinity Estimation**
 - ▶ Measure: **Story Points / T-Shirt Size**
- ▶ **Small and near work:**
 - ▶ Artifact: **Sprint Backlog**
 - ▶ Content: **Tasks**
 - ▶ Technique: **Planning Poker**
 - ▶ Measure: **Hours (Fibonacci sequence)**

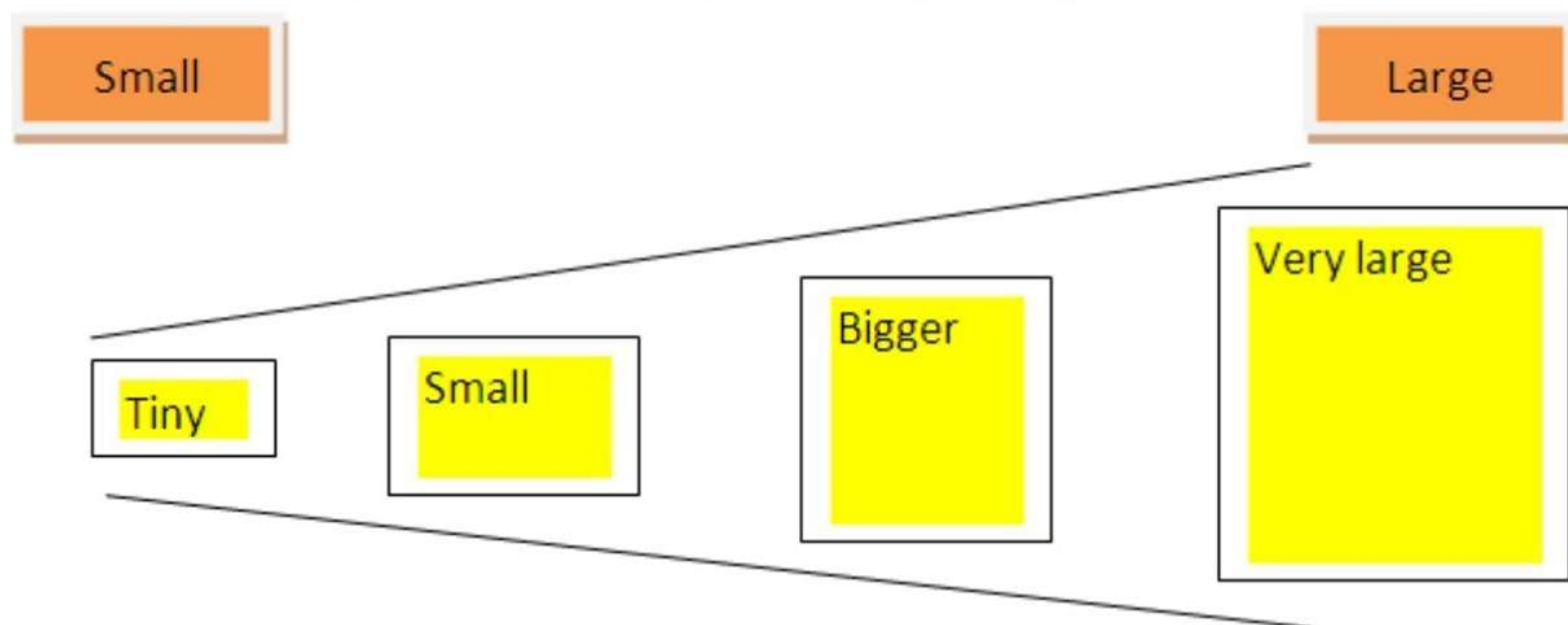


Story points

- ▶ Measure of **relative effort**, not hours
- ▶ Compare items based on **complexity**, **risk**, and **work size**
- ▶ Larger number -> more effort needed
- ▶ Helps with velocity tracking and Sprint planning
- ▶ Provide a common understanding of the work involved.

- ▶ Estimate the story points using based on:
 - ▶ The Fibonacci sequence (0, 0.5, 1, 2, 3, 5, 8, 13, 20, 40, 100)
 - ▶ T-Shirt Size (XS, S, M, L, XL)
 - ▶ ...

Affinity Estimation



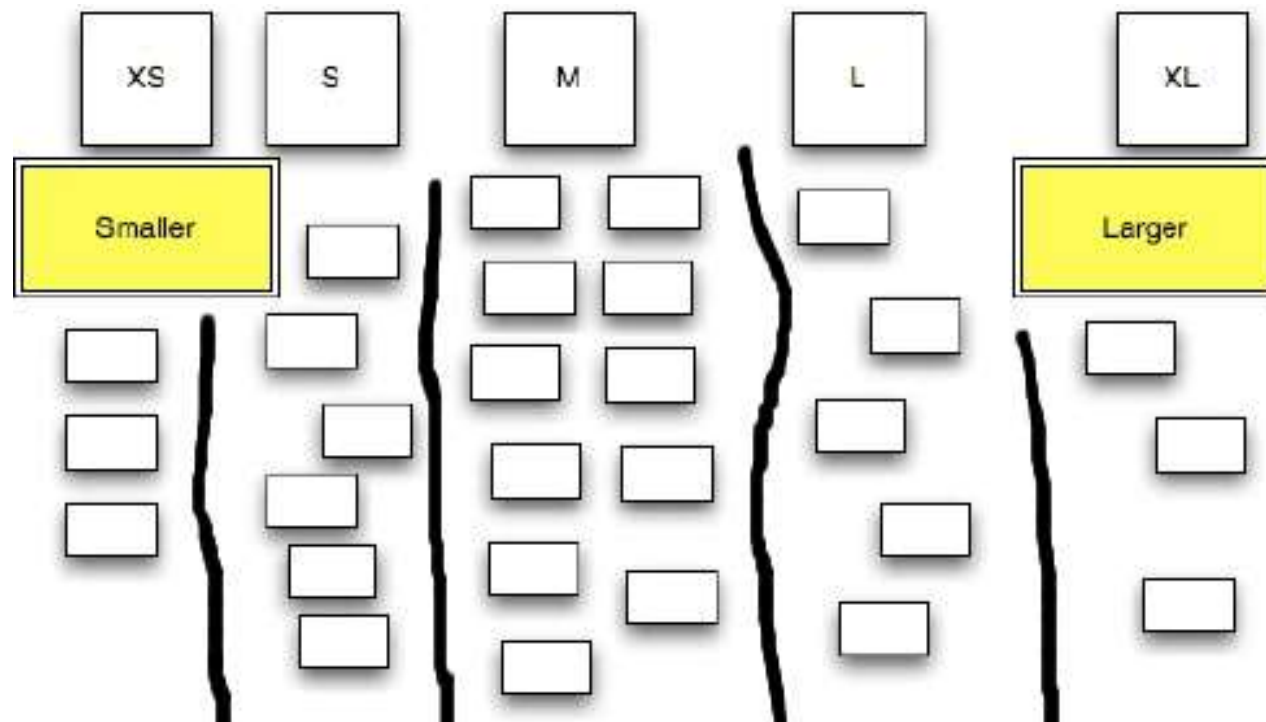
How to estimate – Affinity Estimation

1. Place two posts on the Wall (small and large).
2. Team members will be asked to come to the Wall with a subset of the product backlog.
3. Team members will size each item relatively to the others on the wall.
 - ▶ Silent step.
4. Product Owner/Stackholders present only to provide clarity on some issue.
5. Place all the items on the Wall.
6. After that, all members would look to the Wall and validate if they agree on the estimation.
7. They may change the place of the user story after some debate.
8. Estimation ends when no more user stories are changed in the Wall.

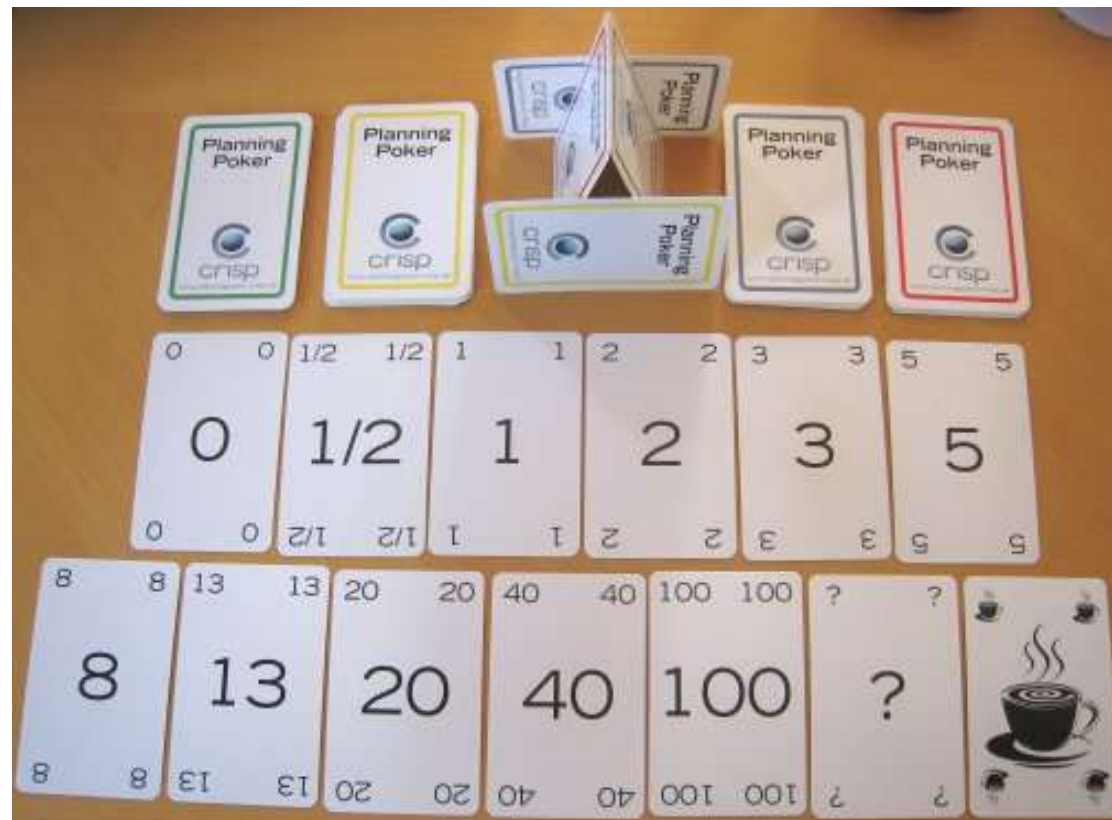
How to estimate – Affinity Estimating

9. Create columns

- ▶ Relative buckets.
- ▶ T-shirt sizes



How to estimate – Planning Poker



How to estimate – Planning Poker

1. Form a group of no more than 10 estimators and a moderator. The Product Owner cannot be an estimator.
2. Each estimator gets a deck of cards with the modified Fibonacci sequence: 0, 0.5, 1, 2, 3, 5, 8, 13, 20, 40, 100 and ?.
3. The moderator reads the description of the task (or user story). The Product Owner answers questions from the estimators.
4. Every estimator selects an estimate card and places it face down on the table. After all estimates are in, the cards are flipped over.
5. If the estimates vary widely, the owners of the high and low estimates discuss the reasons for diverging. All estimators should participate in the discussion, 3 minutes max.
6. Repeat from step 4 until the estimates converge.

Fibonacci Sequence

THE FIBONACCI SEQUENCE

Each number is the sum of the two that precede it.

0 1 1 2 3 5 8 13 21

$$0 + 1 = 1$$

$$1 + 1 = 2$$

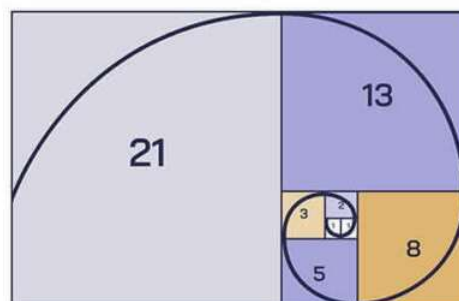
$$1 + 2 = 3$$

$$2 + 3 = 5$$

$$3 + 5 = 8$$

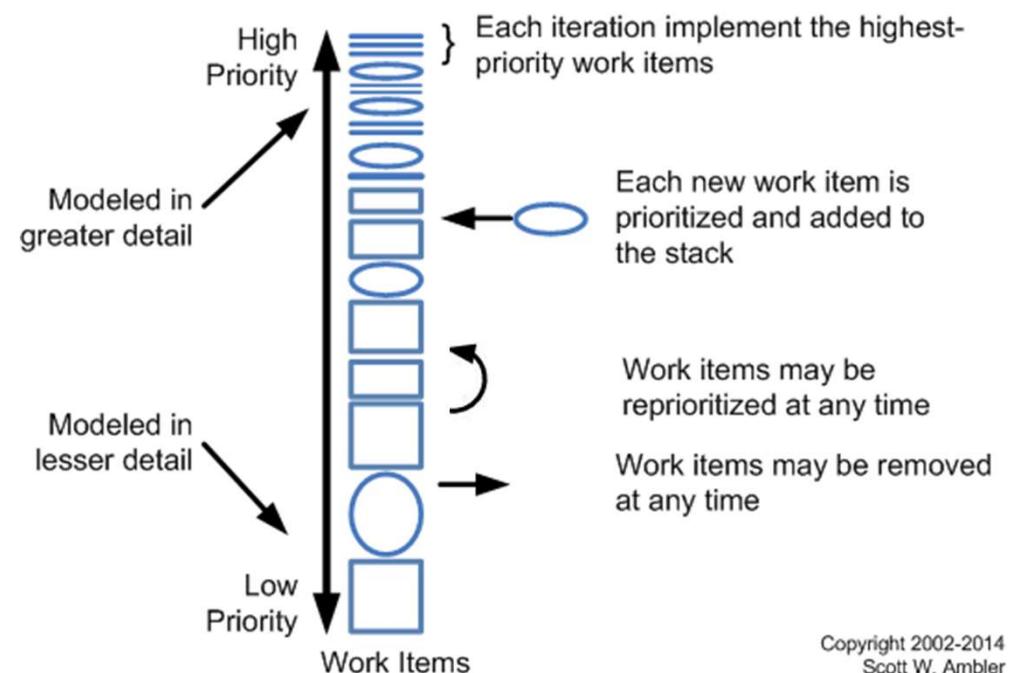
$$5 + 8 = 13$$

$$8 + 13 = 21$$



Software Prioritization

- ▶ In fixed-time projects it is vital to understand the relative importance of the work to be done
- ▶ Focus first on features with highest value and impact

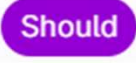
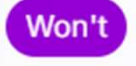


Copyright 2002-2014
Scott W. Ambler

Prioritization with MoSCoW

- ▶ **M**ust Have
 - ▶ there is no point in implementing a solution that does not meet this requirement
- ▶ **S**hould Have
 - ▶ important but not vital, although painful to leave out, the solution is still viable
- ▶ **C**ould Have
 - ▶ wanted or desirable but less important, has less impact than “should have” if left out
- ▶ **W**on't Have this time
 - ▶ not be delivered for now, maybe later; helps keeping the focus at this timeframe on the more important requirements

Size and Priority Labels for the Product Backlog Issues in GitLab

Size Estimation		Priority	
Small		Must have	
Medium		Should have	
Large		Can have	
Extra Large		Won't have	



Further reading

- ▶ Andrew Stellman and Jennifer Greene, *Applied Software Project Management*, O'Reilly, 2006. [Section 3]
- ▶ Whitney Vige, What are story points? Six easy steps to estimate work in Agile, (<https://asana.com/resources/story-points>)
- ▶ Mountain Goat Software website, *Planning Poker*, <https://www.mountaingoatsoftware.com/agile/planning-poker>
- ▶ Many online tools (ex: <https://planningpokeronline.com/>)
- ▶ Agile Modeling, Prioritized Requirements: An Agile Core Practice (<https://agilemodeling.com/essays/prioritizedrequirements.htm>)

9. Acceptance Criteria

Acceptance Criteria

- ▶ Define clear conditions for a User Story to be considered “done”
 - ▶ Ensure shared understanding between team and stakeholders
 - ▶ Written in simple, testable statements (usually Given-When-Then)
 - ▶ Basis for testing and validating features
-
- ▶ **Acceptance criteria complement User Stories**

A Checklist for Writing Acceptance Criteria

- ▶ Clear to everyone involved
 - ▶ Can be tested or verified
 - ▶ Either passes or fails (cannot be 50% completed, for example)
 - ▶ Focus on the outcome, not how the outcome is achieved
 - ▶ As specific as possible (fast page load speed vs. 3-second page load speed)
- ▶ Scrum Alliance, <https://resources.scrumalliance.org/Article/need-know-acceptance-criteria>

Given/When/Then testable statements

- ▶ **Scenario:** (name of the behavior)
 - ▶ **Given** a context
 - ▶ **When** an event happens
 - ▶ **Then** there is an outcome
 - ▶ **And** (may be used to continue any of the 3 previous statements)

Example:

- ▶ **User Story:** As a user, I want to reset my password so I can access my account if I forget it.
- ▶ **Acceptance Criteria:**
 - ▶ **Scenario 1:** User receives a reset link by email when requesting a password reset
 - ▶ **Scenario 2:** Link expires after 24 hours
 - ▶ **Scenario 3:** New password must meet security rules (min. 8 chars, 1 number, 1 symbol)
 - ▶ **Scenario 4:** User can log in with the new password immediately

-
- ▶ **Scenario 1:** User receives a reset link by email when requesting a password reset
 - ▶ **Given** a registered user requests a password reset
 - ▶ **When** they submit their email address
 - ▶ **Then** they receive a reset link via email
 - ▶ **And** the link opens the “enter new password” window

 - ▶ **Scenario 2:** Link expires after 24 hours
 - ▶ **Given** a reset link was generated
 - ▶ **When** more than 24 hours have passed
 - ▶ **And** the user opens the link
 - ▶ **Then** they get a “the link is no longer valid” message

-
- ▶ **Scenario 3:** New password must meet security rules (min. 8 chars, 1 number, 1 symbol)
 - ▶ **Given** a user sets a new password
 - ▶ **When** the password does not meet the security rules (min. 8 chars, 1 number, 1 symbol)
 - ▶ **Then** the system rejects the password and displays an error message

 - ▶ **Scenario 4:** User can log in with the new password immediately
 - ▶ **Given** a user successfully resets their password
 - ▶ **When** they log in with the new password
 - ▶ **Then** the system grants access to their account

Further Reading

- ▶ Alexsoft, Acceptance Criteria for User Stories: Purposes, Formats, Examples, and Best Practices (<https://www.altexsoft.com/blog/business/acceptance-criteria-purposes-formats-and-best-practices/>)
- ▶ Scrum Alliance, Everything You Need to Know About Acceptance Criteria, (<https://resources.scrumalliance.org/Article/need-know-acceptance-criteria>)

10. Sprint Planning and Sprint Backlog

Backlog Refinement

- ▶ **Product Backlog (reminder)**
 - ▶ Prioritized list of work to be performed to achieve the Product Goal
 - ▶ Can contain any type of work (user stories, tasks, bug fixes, spikes, etc.)
 - ▶ Starts high-level
 - ▶ Prioritized and estimated (Story points, T-Shirt size, etc.)

- ▶ **PO refines PB**
 - ▶ Clarify US, split, estimate, re-order

Sprint Planning

- ▶ Meeting to initiate the Sprint
 - ▶ Prepared and managed by the PO
 - ▶ Other stakeholders may attend
 - ▶ Timeboxed – max. 8 hours for 1 month Sprint

- ▶ Address:
- 1. **Why** is this Sprint valuable?
 - ▶ Create **Sprint Goal**
- 2. **What** can be Done this Sprint?
 - ▶ Select Product Backlog Items (**PBI**)
- 3. **How** will the chosen work get done?
 - ▶ Plan the work necessary, create the **Sprint Backlog**

1. **Why** is this Sprint Valuable?

- ▶ PO proposes how to increase product value (aligned with release plan)
- ▶ Team defines the **Sprint Goal** together
 - ▶ Goal gives focus and direction for the Sprint
 - ▶ Scope may change, but goal stays the same

2. What can be done this Sprint?

- ▶ Team selects **Product Backlog Items (PBI)** for the Sprint
- ▶ Based on capacity and velocity
 - ▶ **Team capacity** = number of team members × effective hours per week × number of weeks in the Sprint
 - ▶ Note: effective hours = realistic number of hours available for focused project development work
 - ▶ e.g. 75% * 4 hours per week = 3 hours)
 - ▶ **Velocity** = number of story points per sprint
 - ▶ e.g. have a rough idea of the effort to develop the PBI: convert story points into hours (ex: S:0-2h; M:3-5h; L:6-10h; XL:11-18h)
 - ▶ Check if the selected US can be finished. Adjust as necessary.
- ▶ Only Developers decide what they can deliver

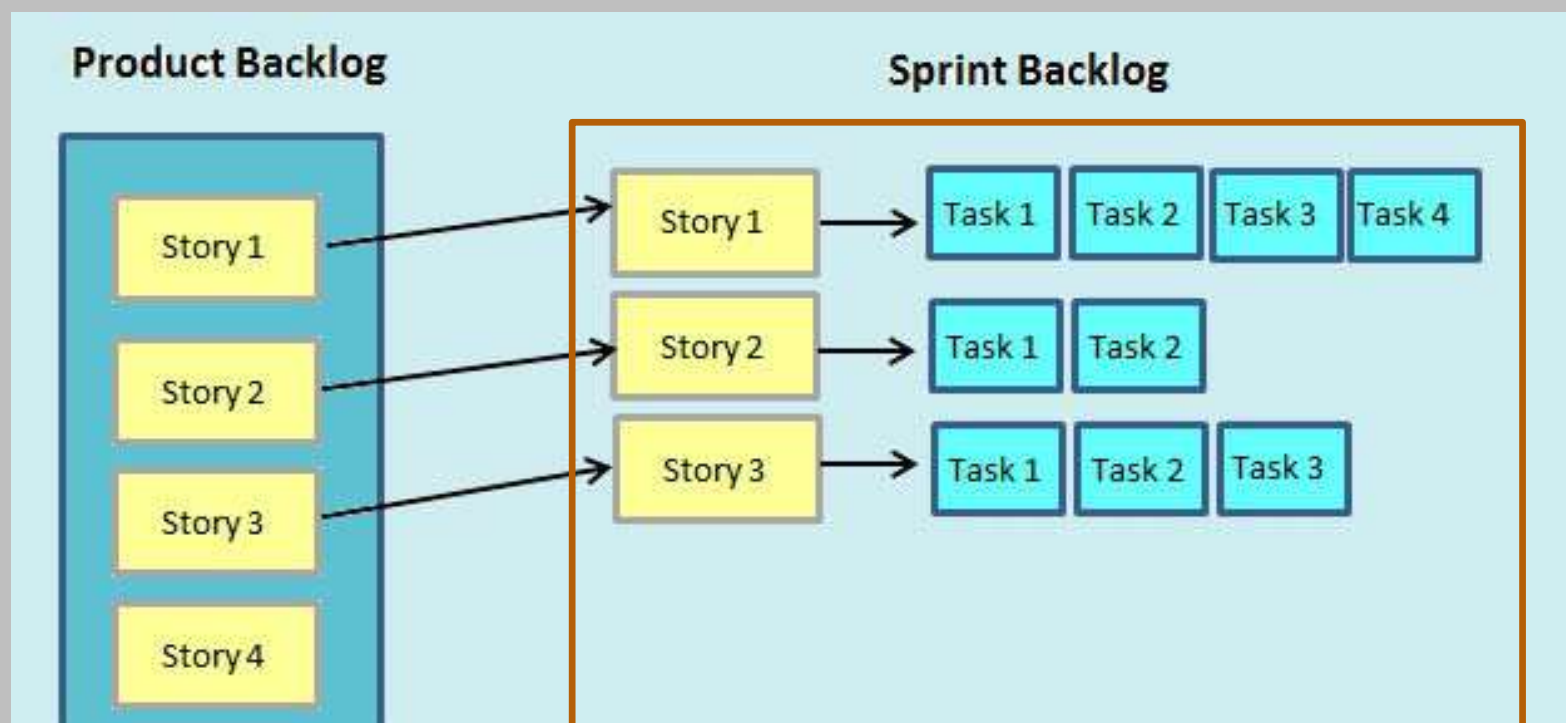
3. **How** will the work get done?

- ▶ Developers plan how to complete the work
- ▶ Break PBIs into smaller tasks
- ▶ Add Acceptance Criteria for each US
- ▶ Estimate the effort for each Task (in hours)
 - ▶ Use e.g. Planning Poker
- ▶ Verify if the Tasks can be done in the sprint. Review if necessary

README – Sprint Plan

- ▶ Sprint Goal: *what's the goal for this sprint*
- ▶ Dates: from XX/Oct to XX, 2 weeks
- ▶ Team capacity = XX hours (2 Weeks * X Team Elements * Y Hours/Week)
- ▶ Roles:
 - ▶ Product Owner: name
 - ▶ Scrum Master: name
 - ▶ QA Engineer: name
- ▶ Sprint Backlog:
 - ▶ US01: As ... I want ... so that ... (link to GitLab) – [XL size]
 - ▶ Task01: Some task (link to GitLab) – [2 hours]
 - ▶ ...
- ▶ Estimated Effort:
 - ▶ 1XL+3M (story points)
 - ▶ 35 hours
- ▶ Analysis: *short analysis of the planning meeting*
- ▶ Sprint board : (link to file with Sprint Backlog Board)

Sprint Backlog board



Product and Sprint Backlog boards

BacklogPB

4

Task15 - Research Unit Tests

spike

#5

US05 - As a teacher, I want to upload course materials so that students can access them anytime.

Must XL

#4

US04 - As a student, I want to search for courses by name or topic so that I can quickly find what interests me.

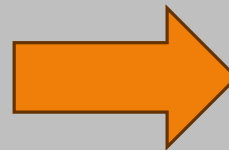
L Should

#3

US08 - As a student, I want to receive an email confirmation when I enroll in a course so that I know my registration was successful.

L Should

#6



BacklogS3

3

Task15 - Research Unit Tests

spike

#5

US05 - As a teacher, I want to upload course materials so that students can access them anytime.

Must XL

#4

US04 - As a student, I want to search for courses by name or topic so that I can quickly find what interests me.

L Should

#3

Sprint Backlog

Mockup

Acceptance
Criteria

Tasks

US05 - As a teacher, I want to upload course materials so that students can access them anytime.

Open Issue created 51 minutes ago by Joao Cunha

Upload File

Description

Select File

OK Cancel

Acceptance Criteria

AC1 - The teacher can upload files in common formats (PDF, DOCX, PPTX, ZIP) Given I am logged in as a teacher When I open the upload materials page Then I should see fields to enter a description AND choose a file (PDF, DOCX, PPTX, ZIP).

Edited 6 minutes ago by Joao Cunha

👍 0 💡 0 😊 Add design Create merge request

Child items 3

- Design the "Upload Material" form (title, file, description). #7 Open
- Implement file upload logic and server storage. #8 Open
- Add validation and upload progress feedback #9 Open

Assignee Edit
None - assign yourself

Labels Edit
BacklogS3 Must XL

Milestone Edit
None

Dates Edit
Start: None
Due: None

Time tracking +
Add an [estimate](#) or [time spent](#).

1 Participant

Task effort
estimation

Edit time estimate

Set estimated time to complete this task.

Estimate

3h

Enter time as a total duration (for example, 1mo 2w 3d 5h 10m), or specify hours and minutes (for example, 75:30).

[How do I estimate and track time?](#)

Cancel Remove Save

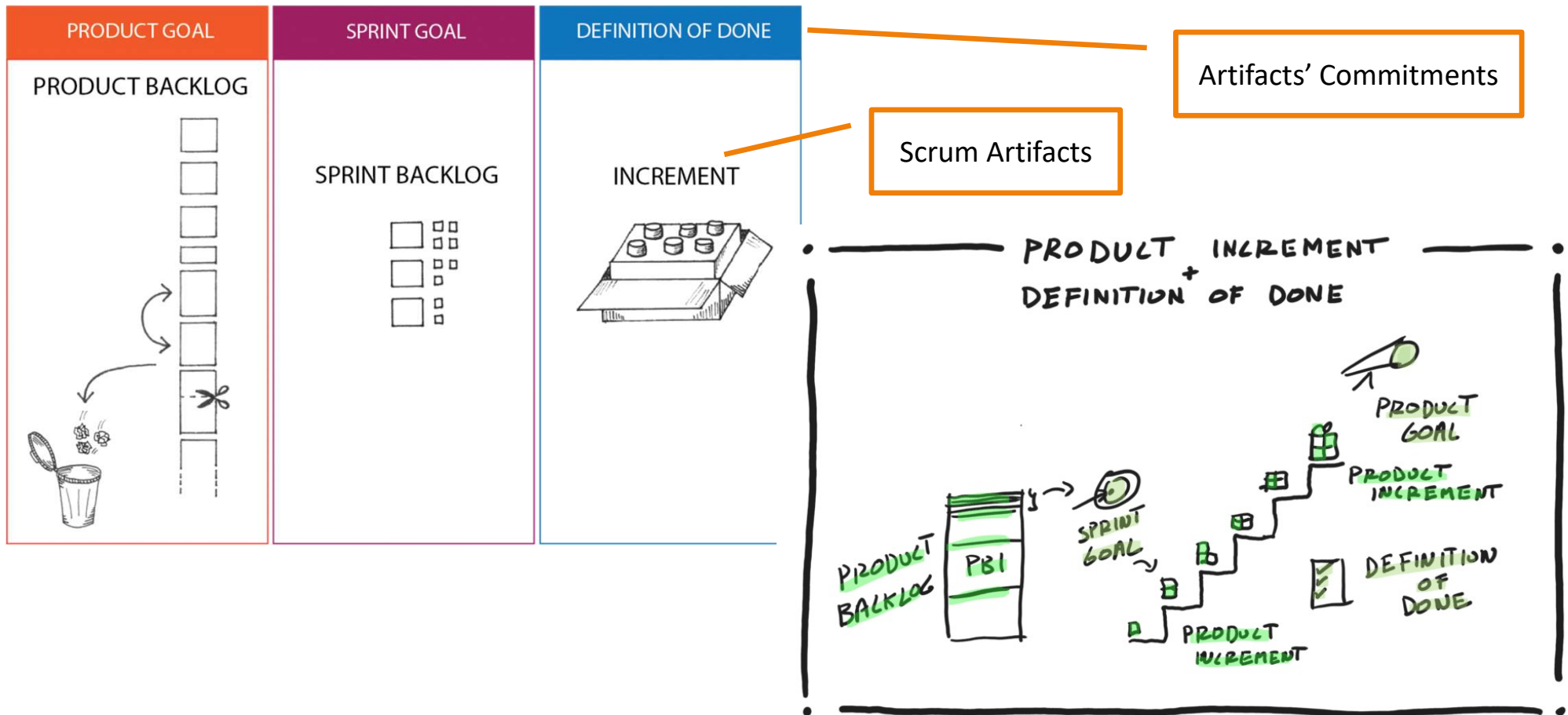
Further reading

- ▶ Ken Schwaber & Jeff Sutherland, The Scrum Guide - The Definitive Guide to Scrum: The Rules of the Game, November 2020
(<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>;
<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Portuguese-European.pdf>)

11. Product Increment and Definition of Done

Product Increment

- ▶ A working, usable piece of software that adds value (may be released or not)
- ▶ Concrete progress toward the Product Goal
- ▶ Must be complete, tested, and integrated (increments build on previous work)
- ▶ Each Sprint delivers at least one Increment
- ▶ Only work meeting the **Definition of Done** counts
- ▶ Ensures transparency on progress toward the Product Goal



<https://www.5dvision.com/post/whats-new-in-the-scrum-guide-2020/>

Definition of Done

- ▶ A shared understanding of what it means for work to be complete
- ▶ Ensures quality, transparency and consistency across Increments
- ▶ Defined by the Scrum Team (or the organization)
- ▶ Work not meeting the DoD cannot be released or counted as part of the Increment

README - Example of DoD

1. All tasks closed
2. Tests passed (unit, integration, UI)
3. Acceptance criteria met
4. Code reviewed by two team members (👍 👍) and merged to QA
5. US accepted by QA Engineer and code merged to Main
6. US moved to Closed

Further reading

- ▶ Product Increment and the Definition of Done in a Nutshell
(<https://www.kaizenko.com/product-increment-and-the-definition-of-done-in-a-nutshell/>)
- ▶ Ken Schwaber & Jeff Sutherland, The Scrum Guide - The Definitive Guide to Scrum: The Rules of the Game, November 2020
(<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>;
<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Portuguese-European.pdf>)

12. Project Control: Sprint and Daily Scrum

Managing Projects

- ▶ Planning
 - ▶ Define tasks, estimate, schedule, assign resources
- ▶ Monitoring
 - ▶ Keep track of the progress and evaluate performance (compare actual status with the plan)
- ▶ Control
 - ▶ Change scope, schedule, reassign resources, reestimate

What to look at?

- ▶ Is the project on schedule?
 - ▶ How much **time** has elapsed?
- ▶ ~~Is the project on budget?~~ not in Scrum
 - ▶ ~~How much did it **cost** so far?~~
- ▶ Is the work being accomplished as planned?
 - ▶ How much of the **planned work** has been realized?

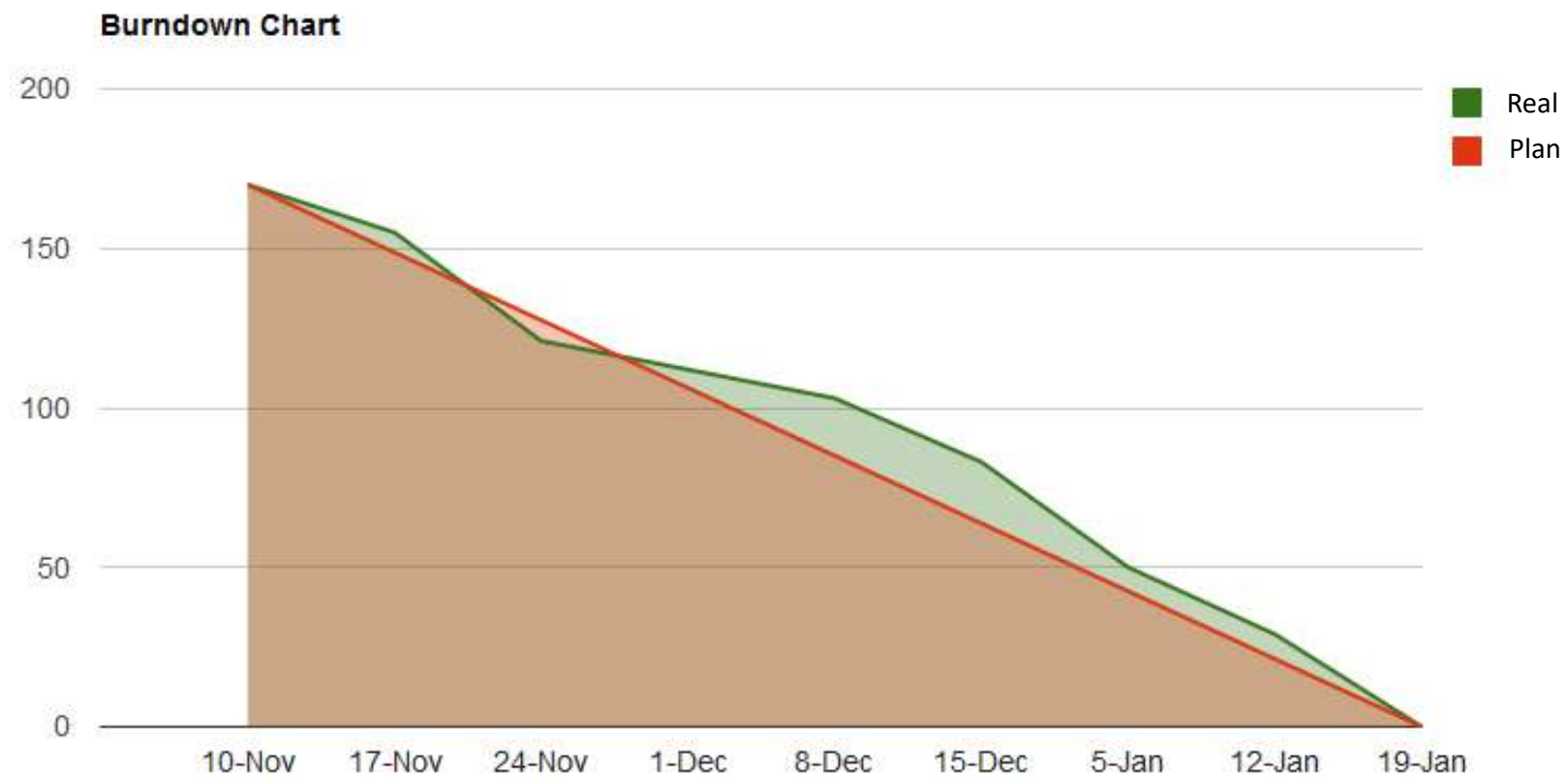
Burndown chart

- ▶ A **burndown chart** is a graphical representation of work left to do versus time.
- ▶ Every task has an estimated work effort
 - ▶ Every day the remaining work for each task is updated
- ▶ It is useful for predicting when all the work will be completed.
- ▶ However, the quality of the burndown chart depends on the accuracy of the tasks estimated effort
- ▶ Also, the burndown chart does not deal with the actual work

How burndown chart works

- ▶ The work is planned until the team capacity
 - ▶ # of team members × # of days in the iteration × # of hours per day (conservative – e.g. 6 hours)
- ▶ The ideal daily burndown is:
 - ▶ team capacity / # of days in the iteration
- ▶ Every day, the work left to be done for the project iteration is calculated:
 - ▶ if a task is completed, its remaining effort is zero
 - ▶ if a task has been started, but not completed, either:
 - ▶ its remaining effort is updated
 - ▶ its remaining effort is left unchanged (all or nothing)
- ▶ If a new task is added, its estimated effort is added to the chart

Burndown chart example



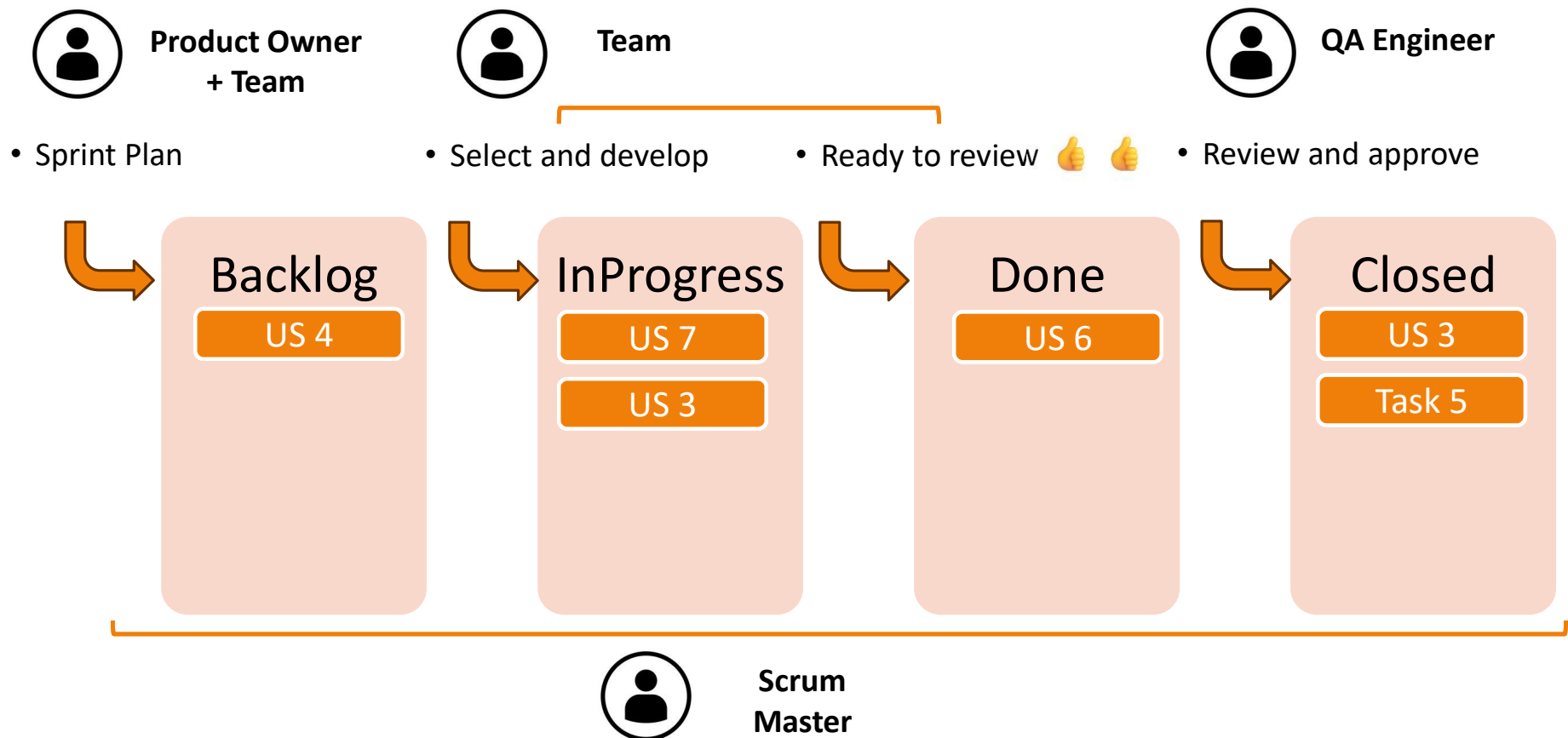
Sprint

- ▶ The heartbeat of Scrum: a fixed-length event (1 month or less)
- ▶ Goal: deliver a usable, valuable Increment toward the Product Goal.
- ▶ Scope: may be clarified or renegotiated with the Product Owner as work emerges.
 - ▶ No changes are made that would endanger Sprint Goal
- ▶ A time-boxed container for all other Scrum events
 - ▶ **Sprint Planning -> Daily Scrums -> Development Work -> Sprint Review -> Sprint Retrospective**

Daily Scrum

- ▶ Also known as Daily Standup Meeting
- ▶ Purpose: inspect progress toward the Sprint Goal
 - ▶ May adapt the Sprint Backlog, adjusting the upcoming planned work.
- ▶ Short meeting with Developers
 - ▶ Daily, 15 minutes, same place, same time
- ▶ Structure and techniques are selected by the Developers
 - ▶ Focus on the progress toward the Sprint Goal
 - ▶ Produces an actionable plan for the next day of work
- ▶ Daily Scrums:
 - ▶ Improve communications
 - ▶ Identify impediments
 - ▶ Promote quick decision-making
 - ▶ Eliminate the need for other meetings

Sprint Progress Tracking



Write Estimates and Real Effort

Edit time estimate

Set estimated time to complete this task.

Estimate

Enter time as a total duration (for example, 1mo 2w 3d 5h 10m), or specify hours and minutes (for example, 75:30).

[How do I estimate and track time?](#)

Cancel

Remove

Save

Add time entry

Track time spent on this issue. [How do I track and estimate time?](#)

Time spent

Spent at (optional)



Example: 1h 30m

Summary (optional)



Cancel

Save

Time tracking



Spent 1h

Est 3h

[Time tracking report](#)

Further reading

- ▶ Ken Schwaber & Jeff Sutherland, The Scrum Guide - The Definitive Guide to Scrum: The Rules of the Game, November 2020
(<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>;
<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Portuguese-European.pdf>)

13. Sprint Review, Sprint Retrospective

Sprint Review

- ▶ Purpose: to inspect the outcome of the Sprint and determine future adaptations.
- ▶ Meeting With Scrum Team and Stakeholders:
 - ▶ At the end of the sprint
 - ▶ The Scrum Team presents the results of their work
 - ▶ Attendees discuss progress toward the Product Goal
 - ▶ Attendees review what was accomplished and what has changed
 - ▶ Attendees collaborate on what to do next
 - ▶ The Product Backlog may be adjusted to meet new opportunities
 - ▶ It is a working session, not just a presentation.
- ▶ The Sprint Review is timeboxed, max. of 4 hours for a one-month Sprint

Sprint Retrospective

- ▶ Purpose: to plan ways to increase quality and effectiveness.
- ▶ Meeting with the Scrum Team:
 - ▶ Inspects how the last Sprint went with regards to individuals, interactions, processes, tools, and their Definition of Done
 - ▶ The Scrum Team discusses what went well, what problems it encountered, and how those problems were (or were not) solved.
 - ▶ The Scrum Team identifies the most helpful changes to improve its effectiveness
- ▶ The Sprint Retrospective concludes the Sprint. It is timeboxed, max. 3 hours for a one-month Sprint.

Further reading

- ▶ Ken Schwaber & Jeff Sutherland, The Scrum Guide - The Definitive Guide to Scrum: The Rules of the Game, November 2020
(<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-US.pdf>;
<https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Portuguese-European.pdf>)

14. Software Version Control

“Software Project Management”
João Cunha, ISEC (jcunha@isec.pt)

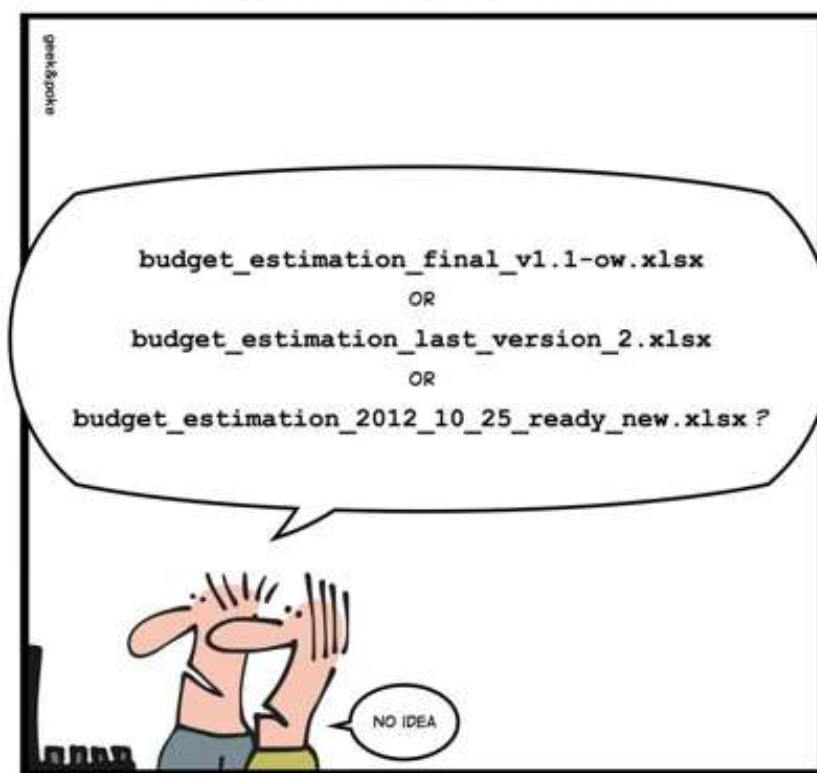


 Octoverse 2024

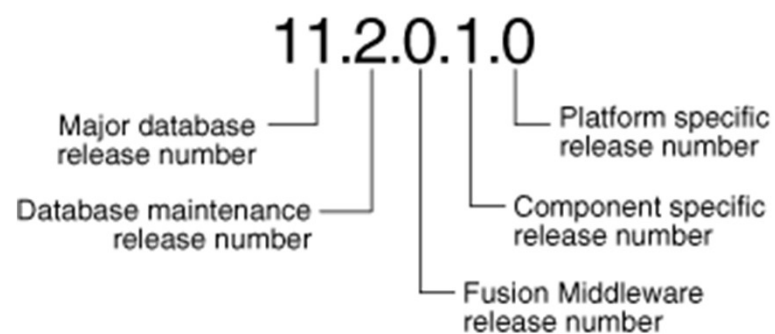
The background is a dark blue gradient with a faint grid of dashed lines. Several glowing lines in magenta, green, blue, and orange run diagonally across the frame. A bright green and blue light streak is visible in the upper right quadrant.

Why version control?

SIMPLY EXPLAINED



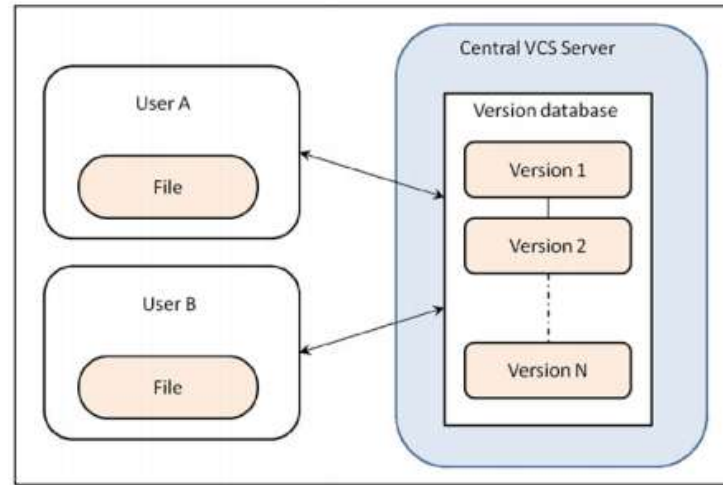
VERSION CONTROL



Configuration Management

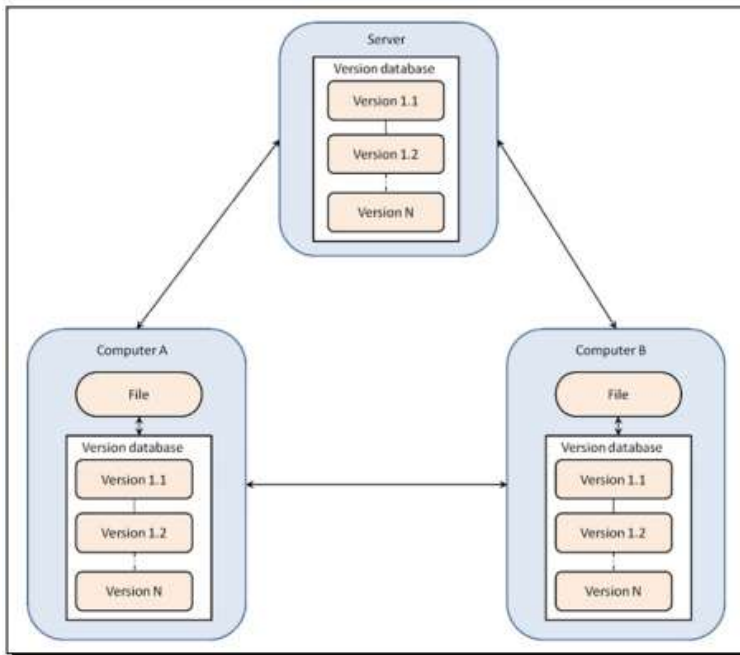
- ▶ Configuration Management (CM) is the discipline that ensures that the state of the software at any given time is known and reconstructable. This refers both to software under development and released.
- ▶ **Version control** (or revision control, or source control) is the management of documents or programs, keeping track of all the changes.
- ▶ **Release control** manages the different releases of product lines, having a common set of files, and different build environments.

Centralized Version Control System



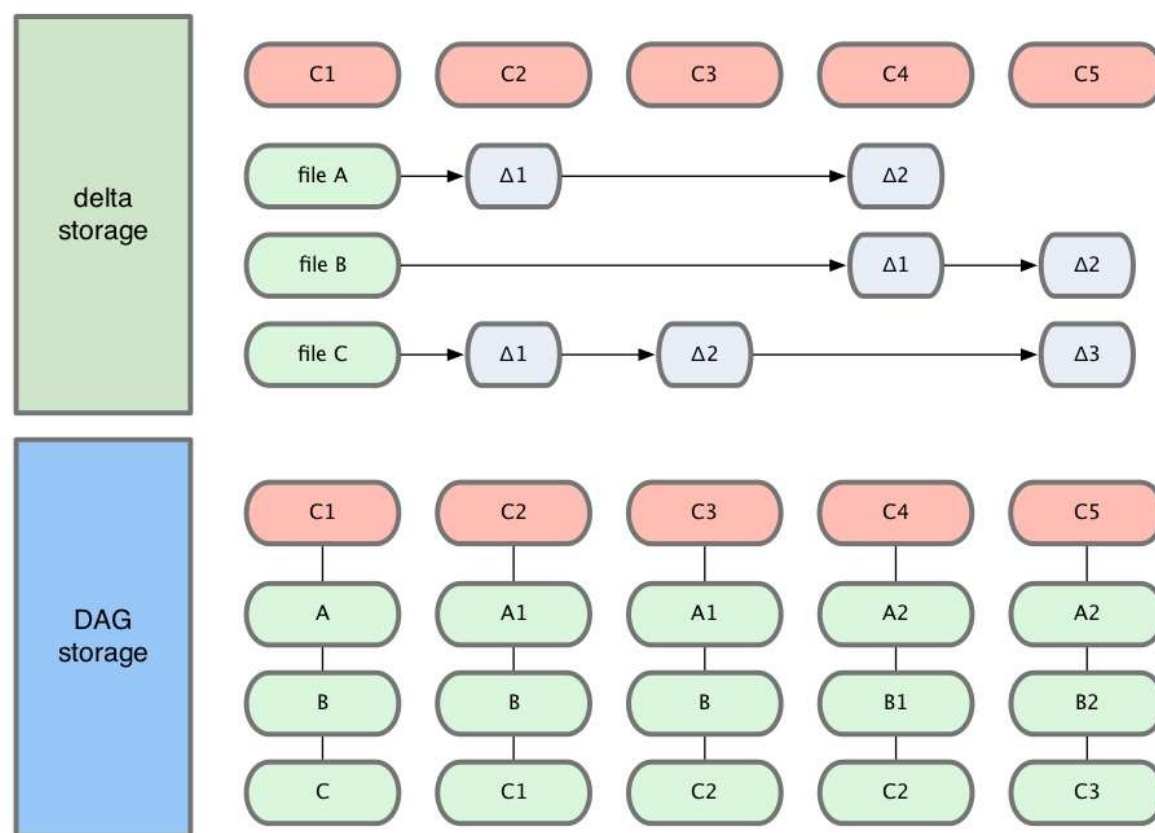
- ▶ Only the last version of the files are retrieved.
- ▶ Offers visibility on what other people are working towards.
- ▶ Any changes made to the files are automatically shared with other individuals as well.

Distributed Version Control System



- ▶ Stores the entire history of the files on each and every machine locally
- ▶ Syncs the local changes made by the user back to the server whenever required so that the changes can be shared with others.
- ▶ Makes local changes without any concern of full-time connectivity to the server
- ▶ Does not rely on a single copy of files stored in the server

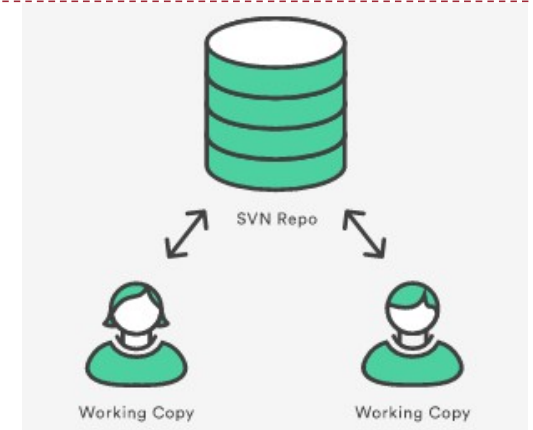
Storage sytem



SVN vs Git

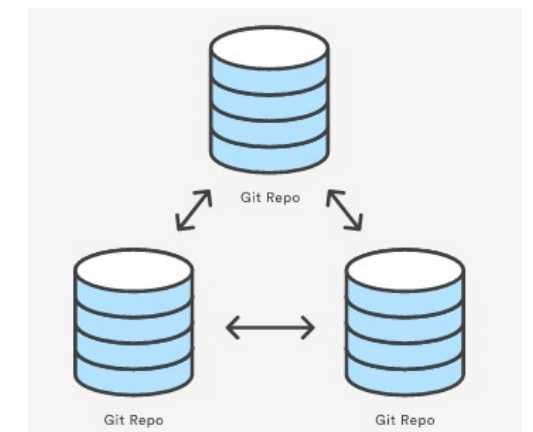
▶ Apache Subversion (SVN)

- ▶ Open-source project, founded in 2000 by CollabNet. Released in 2004
- ▶ Centralized
- ▶ Delta storage



▶ Git

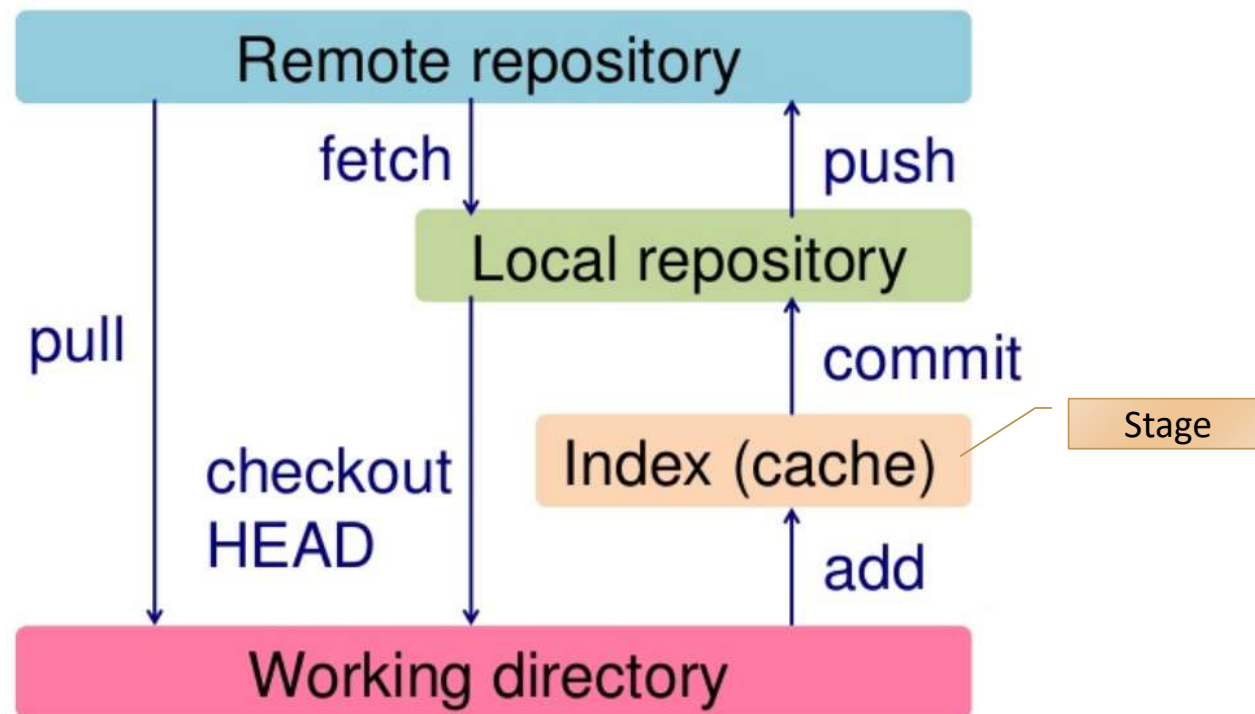
- ▶ Created by Linus Torvalds in 2005 for development of the Linux kernel, as open-source
- ▶ Distributed
- ▶ DAG storage





by [Randall Munroe](#), from [xkcd](#)

Git



Git: Working on a repository

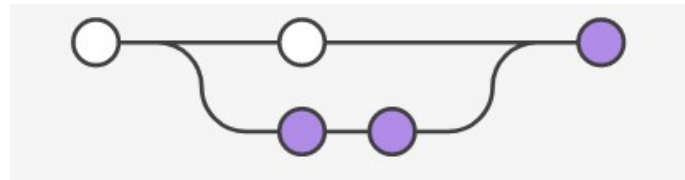
- ▶ Setup a repository
 - ▶ **git init**: initialize a new repository
 - ▶ **git clone**: copy a repository
 - ▶ **git config**: configure repository
- ▶ Save changes in files
 - ▶ **git add**: adds a change in the working directory to the staging area
 - ▶ **git commit**: commits the staged snapshot to the project history
- ▶ Inspect the repository
 - ▶ **git status**: displays the state of the working directory and the staging area
 - ▶ **git log**: command displays committed snapshots
- ▶ Undo changes
 - ▶ **git checkout**: fetch an old version of a file or the entire working directory from a commit

Collaborating - Syncing

- ▶ **git remote**: create, view, and delete connections to other repositories
- ▶ **git fetch**: imports commits from a remote repository into your local repo
- ▶ **git pull**: **fetch** the remote copy of the current branch + **merge** it into the local copy
- ▶ **git push**: transfer commits from your local repository to a remote repo.

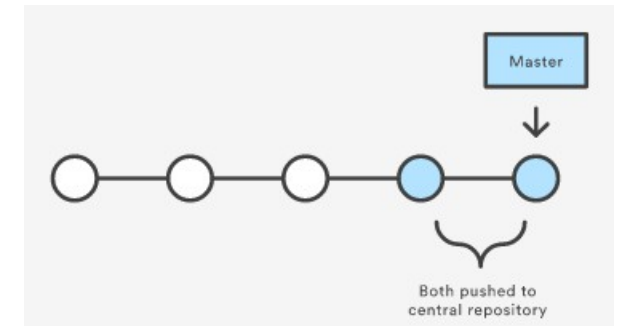
Branches

- ▶ **git branch:** create, list, rename, and delete branches
- ▶ **git checkout:** updates the files in the working directory to match the version stored in that branch
- ▶ **git merge:** take the independent lines of development created by git branch and integrate them into a single branch.

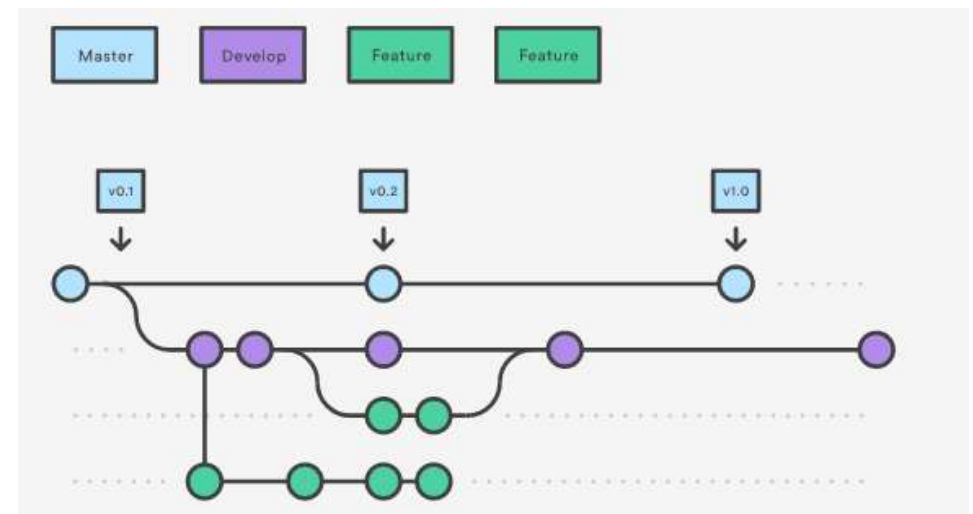


Workflows

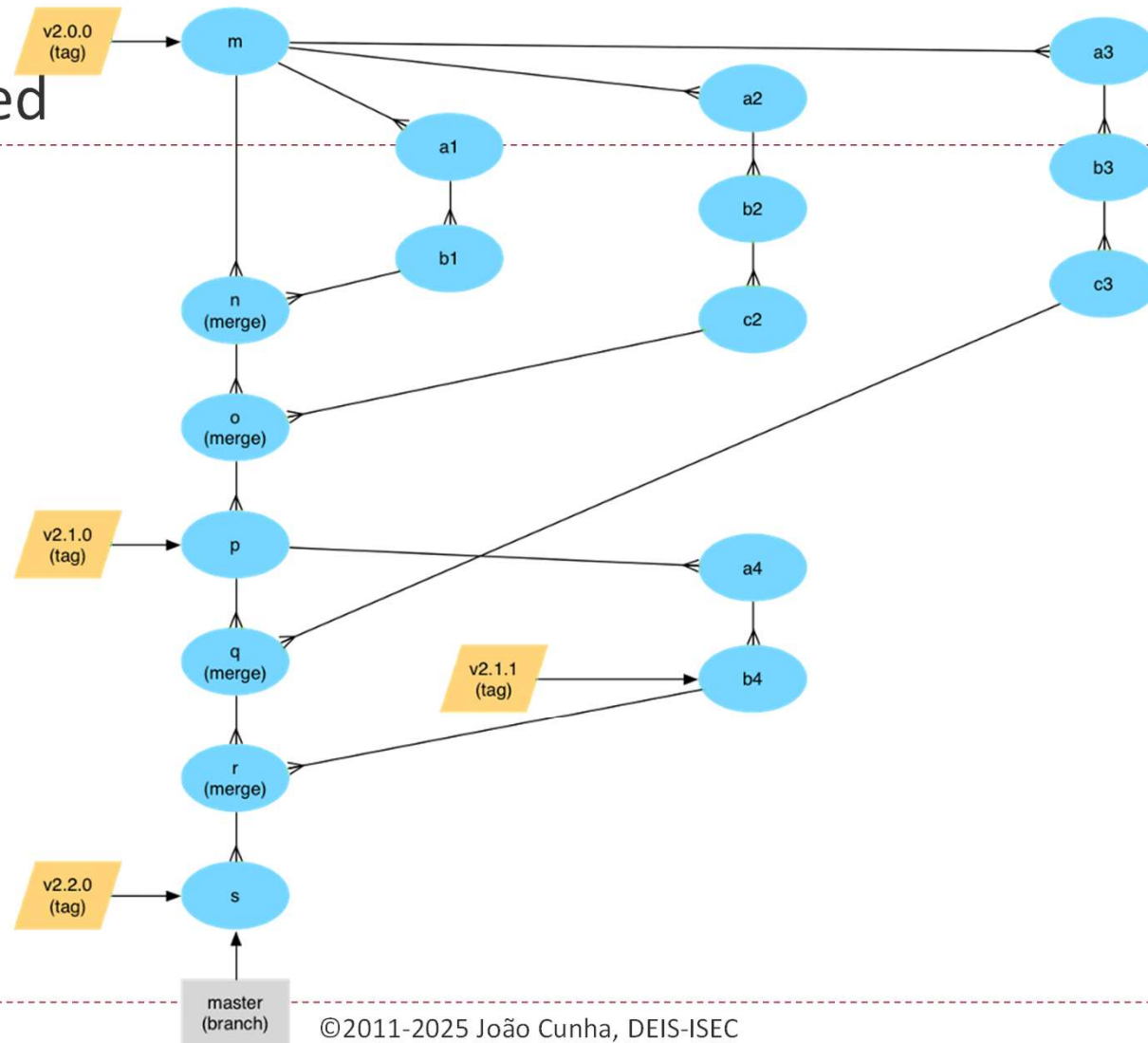
► Centralized workflow



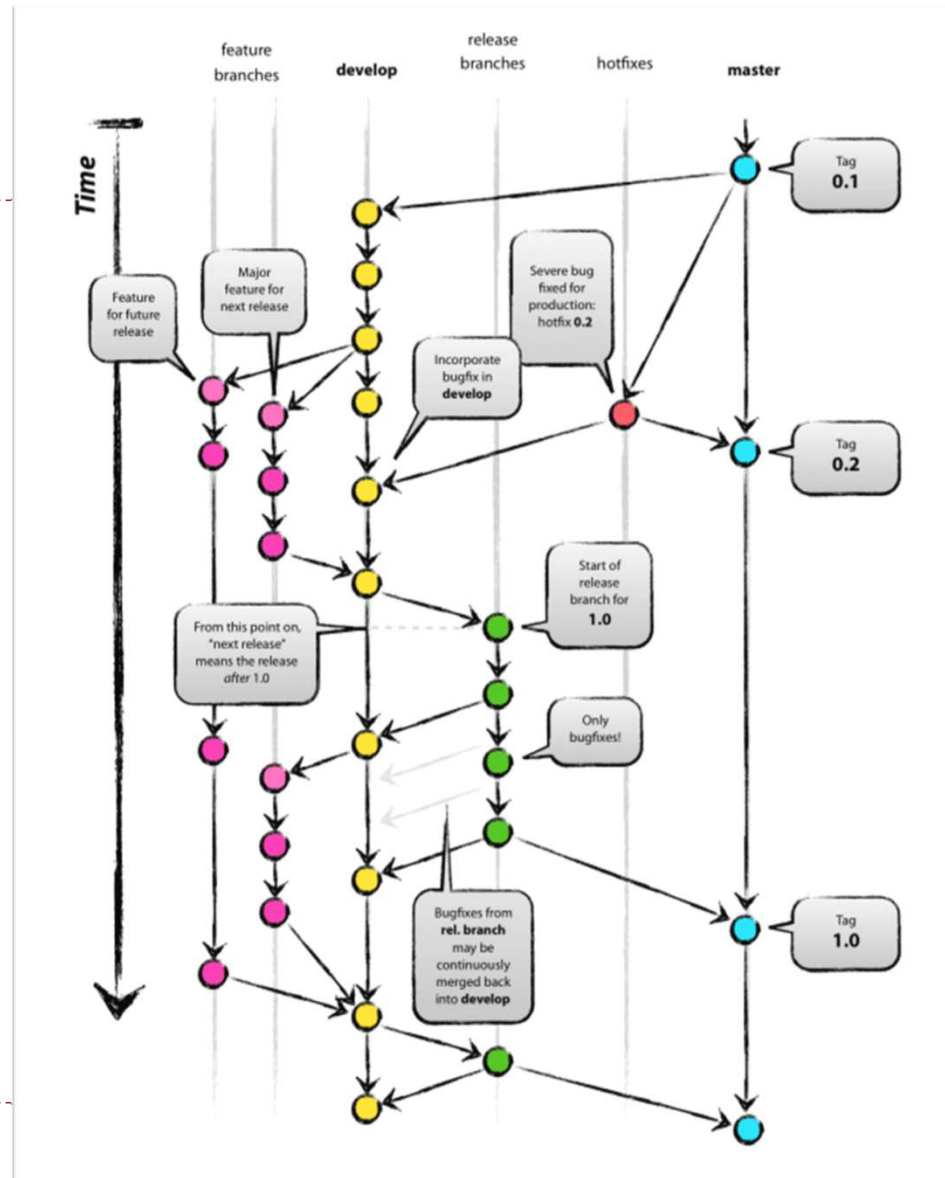
► Feature-based workflow



Feature-based

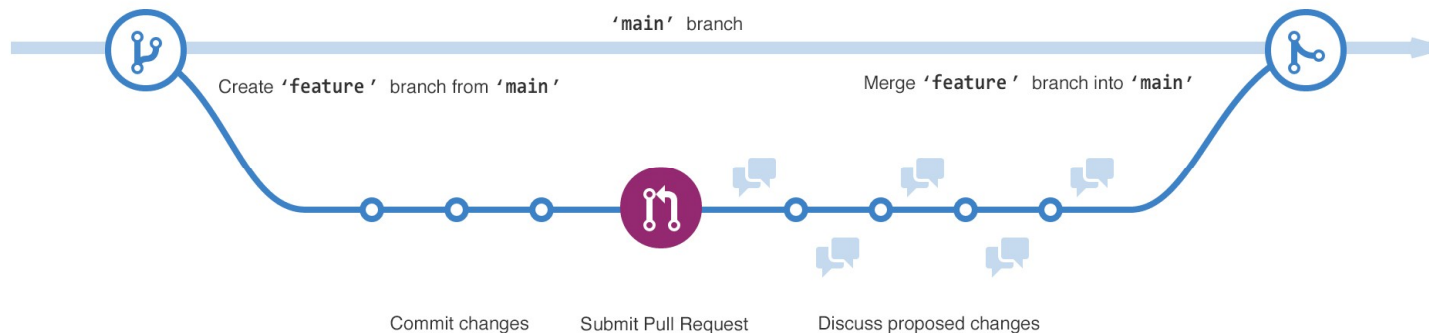


Gitflow workflow

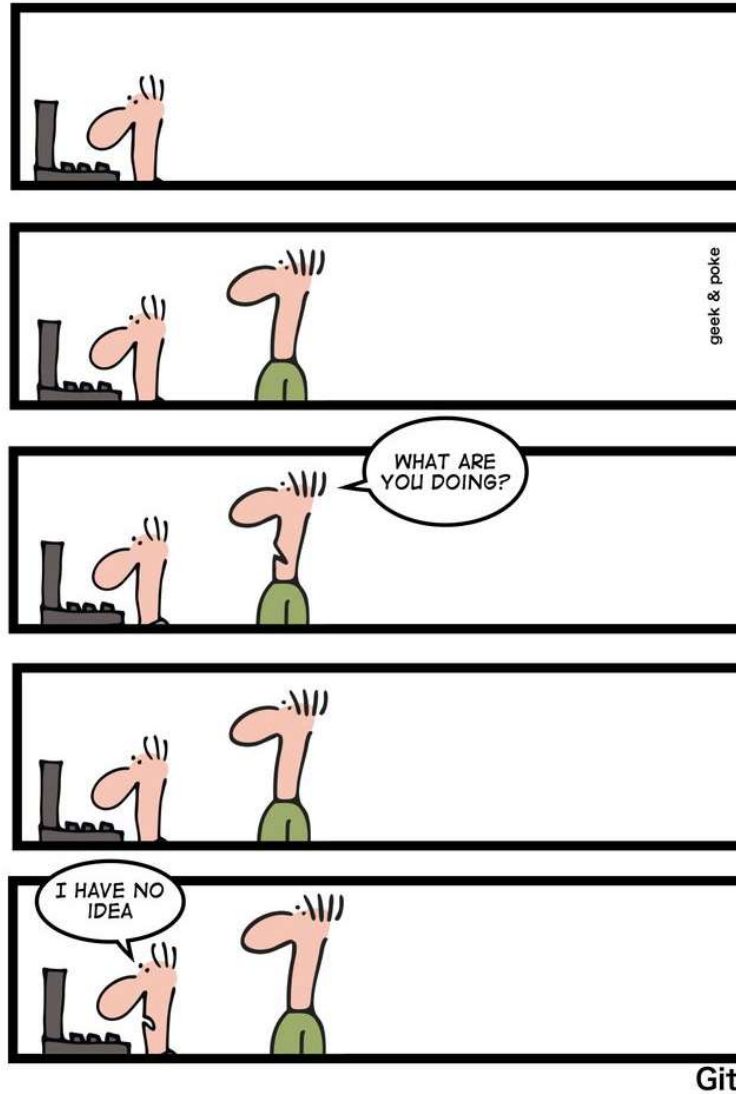


Pull Request

1. A developer creates the feature in a dedicated branch in the local repo.
2. The developer pushes the branch to a public repository.
3. The developer files a pull request via the repository.
4. The rest of the team reviews the code, discusses it, and alters it.
5. The project maintainer merges the feature into the official repository and closes the pull request.



SIMPLY EXPLAINED



Further Reading

▶ GitHub

- ▶ <https://guides.github.com/activities/hello-world/>
- ▶ <https://octoverse.github.com>

▶ Git

- ▶ <http://nvie.com/posts/a-successful-git-branching-model/>
- ▶ <https://www.atlassian.com/git/tutorials>

▶ GitKraken

- ▶ <https://www.gitkraken.com/learn-git>

▶ See some Cheat Sheets

- ▶ <https://education.github.com/git-cheat-sheet-education.pdf>
- ▶ <https://about.gitlab.com/images/press/git-cheat-sheet.pdf>

Some slides adapted from A. Stellman & J. Greene, “Applied Software Project Management” and Ian Sommerville, “Software Engineering”

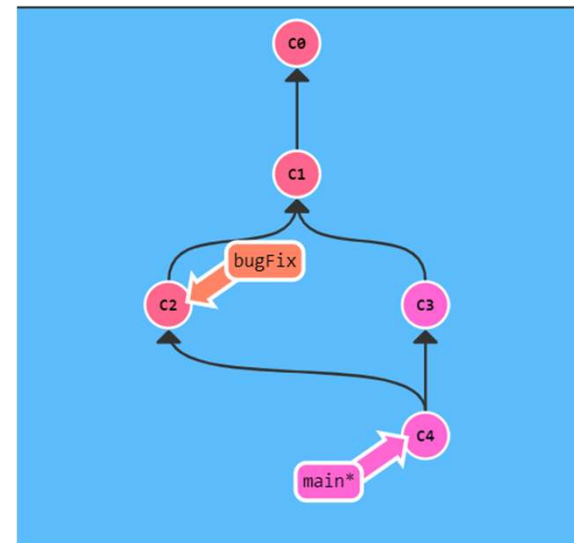
Learn GIT

Oh My GIT!



► <https://ohmygit.org/>

Learn Git Branching



► <https://learngitbranching.js.org/>

15. Team Management

“Software Project Management”
João Cunha, ISEC (jcunha@isec.pt)

What is a team?

- ▶ A team is a small number of people, with complementary skills, who are committed to a common purpose, performance goals, and approach for which they are mutually accountable.

What is an effective team ?

- ▶ An effective team has certain characteristics that allow the team members to function more **efficiently** and **productively**
- ▶ They develop ways to share leadership roles and ways to share accountability for their work products, shifting the emphasis from the **individual** to **several individuals** within the team

Characteristics of an effective team

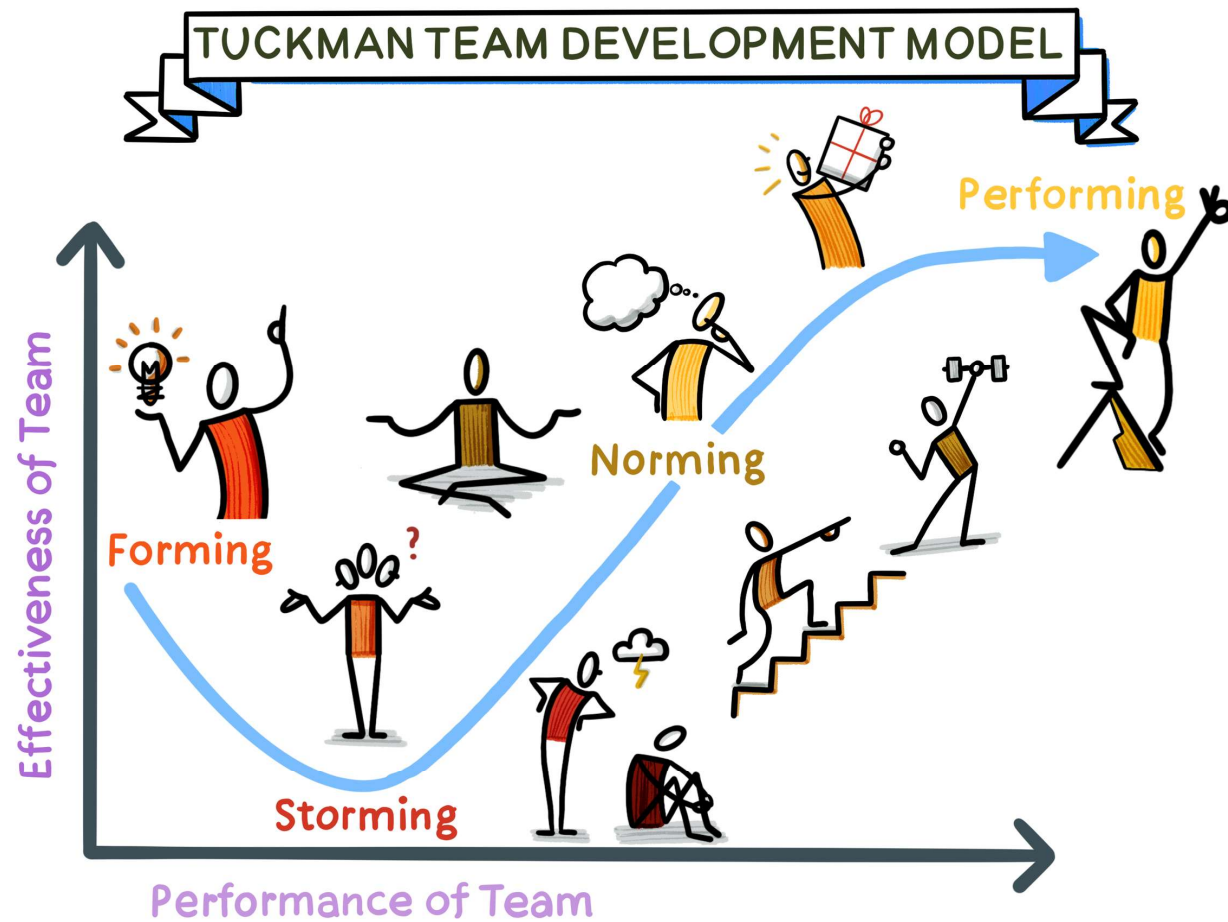
- ▶ Team members share leadership roles
- ▶ They develop their own scope of work
- ▶ The team schedules work to be done and commits to taking time allotted to do work
- ▶ They develop tangible work products
- ▶ They are mutually accountable for work product
- ▶ Individual performance is based on achieving team products
- ▶ Problems are discussed and resolved by the team

Characteristics of an effective team

- ▶ Teams accept **responsibility** and not "blame" one another for team mistakes, nor should they spend useless time in personal justifications.
- ▶ They celebrate their successes **together**, just as they identify the special performances and contributions of each individual.
- ▶ Good teamwork is based on **prideful humility**: pride in the outcomes of the team and a humility that assumes that other team members may well have a level of expertise that can add value to the outcome.
- ▶ Good teams look upon first time mistakes as opportunities for **learning**, rather than criticism and punishment. They are, however, tough on repetitive errors, just as they encourage appropriate risk and innovation.
- ▶ Good teams **honor** the contribution that each member makes to the total work of the team

Characteristics of an effective team

- ▶ **Take responsibility** — don't assign blame or make excuses.
- ▶ **Celebrate success** and recognize individual contributions.
- ▶ **Show pride and humility** — value others' expertise.
- ▶ Treat **mistakes as learning**, not failure — but avoid repeating them.
- ▶ **Respect and value** every team member's contribution.



Introduction: Tuckman's Model

- ▶ Proposed by Bruce Tuckman in **1965**
- ▶ Describes how **teams evolve and mature** over time.
- ▶ Widely used in **project management**, education, and organizational development.
- ▶ Applies to **any team-based activity** — from student projects to professional teams.
- ▶ **Key Idea:** Effective teamwork develops through identifiable, predictable stages.

The 5 Stages

- ▶ **Forming**
 - ▶ Team members get to know each other and understand the task.
 - ▶ Lack of clarity about the team's purpose and individual roles.
- ▶ **Storming**
 - ▶ Conflicts and different opinions emerge as roles take shape.
- ▶ **Norming**
 - ▶ The team finds its rhythm, establishes trust, and defines norms.
 - ▶ There is clarity about individual roles.
- ▶ **Performing**
 - ▶ The group has a clear strategy and shared vision.
 - ▶ Members work efficiently and collaboratively toward shared goals.
- ▶ **Adjourning (added in 1977)**
 - ▶ The team completes its work, reflects and prepares to disband.
 - ▶ Ensuring members leave with a sense of accomplishment and readiness for future teamwork.

Further reading

- ▶ Andrew Stellman and Jennifer Greene, *Applied Software Project Management*, O'Reilly, 2006. [Section 10.4]
- ▶ West Chester University, Tuckman's Stages of Group Development [<https://www.wcupa.edu/coral/tuckmanStagesGroupDevelopment.aspx>]

Some slides adapted from A. Stellman & J. Greene, “Applied Software Project Management” and Sumit Malhotra, “Creating Effective Teams”

16. Software Quality and Testing

“Software Project Management”
João Cunha, ISEC (jcunha@isec.pt)

Quality in Agile

- ▶ **Agile View of Quality:**
 - ▶ “Working software over comprehensive documentation”
 - ▶ Quality as built-in, not inspected at the end
 - ▶ Continuous feedback and adaptation
 - ▶ Shared responsibility (team owns quality, not just QA)
- ▶ **Quality Across the Agile Lifecycle**
 - ▶ Definition of Done (DoD)
 - ▶ Definition of Ready (DoR) -> to be addressed
 - ▶ Continuous integration & delivery (CI/CD) -> to be addressed
 - ▶ Short feedback loops (sprints, reviews, retrospectives)

Software testing

- ▶ Purpose of testing: to show that a program does what it is intended to do and to discover program defects before it is put into use.
- ▶ The program is executed using artificial data.
- ▶ The results of the test runs are checked for errors, anomalies or information about the program's non-functional attributes (e.g. performance).
- ▶ Tests can reveal the presence of errors NOT their absence.

Testing levels

1. Unit Tests

- ▶ **Verify** individual modules in isolation (methods, classes, etc.)
- ▶ May use stubs/mocks for external dependencies
- ▶ Ex: **Junit**, Mockito

2. Integration Tests

- ▶ Verify communication between modules
- ▶ Modules are integrated one by one – **incremental integration** ≠ ~~big-bang integration~~

3. Acceptance Tests

- ▶ **Validate** user requirements or business logic
- ▶ Based on user stories and acceptance criteria
- ▶ **Manual** or automated (Selenium, Serenity, TestFX)

Code Quality Practices

1. **Static Code Analysis** (automated reviews)

- ▶ Detect bugs, smells, and vulnerabilities early
- ▶ Tools: **SonarCloud**, SpotBugs, Checkstyle
- ▶ Integrate in CI/CD pipeline -> to be addressed

2. **Code Reviews**

- ▶ Peer review before merge (Merge Requests)
- ▶ Check for clarity, maintainability, and design consistency
- ▶ Encourage knowledge sharing

3. **Pair Programming**

- ▶ Continuous peer review in action
- ▶ Promotes shared ownership and real-time feedback

Software reviews vs testing

▶ **Code Review (Static Verification)**

- ▶ Analyze code without execution
- ▶ Manual (peer review) or tool-based
- ▶ Detect logic errors, style issues, and design flaws early

▶ **Testing (Dynamic Verification and Validation)**

- ▶ Run code with test data
- ▶ Observe actual behavior and results

▶ **Both are complementary**

- ▶ Reviews can check conformance with specifications but not conformance with the customer's real requirements
- ▶ Tests can check real behavior and non-functional aspects

Verification vs Validation

- ▶ **Verification** – “Are we building the product right?”
 - ▶ Ensures the system **meets specified requirements**
 - ▶ Focus on process and correctness
 - ▶ Techniques: **reviews**, walkthroughs, inspections, **static analysis**, **unit testing**
- ▶ **Validation** – “Are we building the right product?”
 - ▶ Ensures the system meets user needs and expectations
 - ▶ Focus on fitness for purpose
 - ▶ Techniques: **acceptance tests**, **user evaluation**, prototypes, demos
- ▶ Verification ensures **consistency**, Validation ensures **value**.

Quality of Unit Tests

- ▶ Test quality matters more than quantity. Must avoid redundant or weak tests
- ▶ **Test coverage** measures how much code is executed by tests (statements, branches, paths)
- ▶ High coverage helps reveal untested code
- ▶ **coverage $\neq$ correctness**
- ▶ Aim for meaningful, maintainable, and automated tests.

Acceptance Criteria & Acceptance Tests

- ▶ Acceptance criteria define when a user story is complete — clear, testable conditions agreed with the Product Owner.
- ▶ Acceptance tests verify those criteria, confirming the product meets user needs.
- ▶ Documenting test cases or checklists from the criteria improves traceability, consistency, and transparency.

Example of Acceptance Criterion & Acceptance Test

► User Story:

- **As a** registered user, **I want to** reset my password, **so that** I can access my account if I forget it.

► Acceptance Criterion 1:

- **Given** I am on the login page **and** click “Forgot password”, **When** I enter my registered email **and** submit, **Then** I should receive an email with a password reset link.

► Acceptance Test Case 1:

Step	Action	Expected Result
1	Open the login page	Login form is displayed
2	Click “Forgot password”	Password reset form appears
3	Enter a valid registered email and click “Submit”	Confirmation message appears
4	Check the email inbox	Reset email received with valid link
5	Click the reset link	Password reset page opens successfully

Usability testing

- ▶ Usability testing refers to evaluating a product or service by testing it with representative users.
- ▶ Typically, during a test, participants will try to complete typical tasks while observers watch, listen and take notes.
- ▶ The goal is to identify any usability problems, collect qualitative and quantitative data and determine the participant's satisfaction with the product.

Usability test plan

- ▶ Should include:
 - ▶ **Scope** - Indicate what you are testing. E.g. the navigation and content.
 - ▶ **Purpose** - Identify the concerns, questions, and goals for this test. Example: "Can users navigate to important information from the home page?"
 - ▶ **Participants** - Number and types of participants
 - ▶ **Scenarios** - describe the stories and context behind why a specific user or user group comes to your application. They note the goals and questions to be achieved and sometimes define the possibilities of how the user(s) can achieve them on the application. Example: You are traveling to Bragança for your job next week and you want to check on the amount you can be reimbursed for meals and other expenses
 - ▶ **Metrics** (e.g., successful completion rates, error rates, time on task)

Further reading

- ▶ Andrew Stellman and Jennifer Greene, *Applied Software Project Management*, O'Reilly, 2006. [Section 8.1]
- ▶ Paul Ammann and Jeff Offutt, *Introduction to Software Testing*
- ▶ Some slides adapted from A. Stellman & J. Greene, “Applied Software Project Management” and Ian Sommerville, “Software Engineering”

17. Unit Test - JUnit

“Software Project Management”
João Cunha, ISEC (jcunha@isec.pt)

Why create a test suite?

▶ How to test your code

1. Do ad hoc testing (running whatever tests occur at the moment)
2. Build a test suite (a thorough set of tests that can be run at any time)

▶ Disadvantages of a test suite

- ▶ It's a lot of extra programming
 - ▶ However, a good test framework can help quite a lot
- ▶ No time to do all that extra work
 - ▶ Experiments repeatedly show that test suites reduce debugging time more than the amount spent building the test suite

▶ Advantages of a test suite

- ▶ Reduces total number of bugs in delivered code
- ▶ Makes code much more maintainable and refactorable

JUnit

- ▶ JUnit is a unit testing framework for the Java programming language.
- ▶ It is an instance of the xUnit architecture for unit testing frameworks
- ▶ JDK
 - ▶ JUnit 4, requires Java 5 or higher.
 - ▶ JUnit 5, requires Java 8 or higher.
 - ▶ JUnit 6, Java 17 and Kotlin 2.1 baseline – released September 30, 2025

Terminology

- ▶ A **test fixture** sets up the data (both objects and primitives) that are needed to run tests
 - ▶ Example: If you are testing code that updates an employee record, you need an employee record to test it on
- ▶ A **unit test** is a test of a *single* class
- ▶ A **test case** tests the response of a single method to a particular set of inputs
- ▶ A **test suite** is a collection of test cases
- ▶ A **test runner** is software that runs tests and reports results
- ▶ An **integration test** is a test of how well classes work together
 - ▶ JUnit provides some limited support for integration tests

@Test

- ▶ A test method is annotated with **@Test**, takes no parameters, and returns no result
- ▶ All the usual **assertXXX** methods can be used
 - ▶ **@Test**

```
void sum() {  
    assertEquals(15, program.sum(array));  
    assertTrue(program.min(array) > 0);  
}
```

Assert methods

- ▶ Each assert method has parameters like these:
 - ▶ message, expected-value, actual-value (JUnit 4)
 - ▶ expected-value, actual-value, message (JUnit 5)
- ▶ Assert methods dealing with floating point numbers get an additional argument, a tolerance
- ▶ Each assert method has an equivalent version that does not take a message

example: Test Class

```
public class Calculator {  
    public int multiply(int a, int b) {  
        return a * b;  
    }  
}
```

```
class CalculatorTest {  
    private final Calculator calc = new Calculator();  
  
    @Test  
    void testMultiply() {  
        assertEquals(20, calc.multiply(4, 5));  
    }  
}
```

Assert methods examples

- ▶ `assertEquals(Object expected, Object actual)`
 - ▶ Asserts that expected and actual are equal. Uses `equals()` method
- ▶ `assertTrue(Boolean test)`
 - ▶ Asserts that the supplied condition is true
- ▶ `assertFalse(Boolean test)`
- ▶ `assertNull(Object object)`
 - ▶ Asserts that object is null.
- ▶ `assertNotNull(Object object)`
- ▶ `assertSame(Object expected, Object actual)`
 - ▶ Asserts that expected and actual do refer to the same object. Uses `==` operator
- ▶ `assertNotSame(Object expected, Object actual)`

@BeforeAll and @AfterAll (JUnit5)

- ▶ If you wish, you can declare *one* method to be executed *just once*, when the class is first loaded
- ▶ This is for expensive setup, such as connecting to a database
 - ▶ @BeforeAll

```
public static void setUpClass() throws Exception {  
    // one-time initialization code  
}
```
- ▶ If you wish, you can declare *one* method to be executed *just once*, to do cleanup after all the tests have been completed
 - ▶ @AfterAll

```
public static void tearDownClass() throws Exception {  
    // one-time cleanup code  
}
```

@BeforeEach and @AfterEach (JUnit5)

- ▶ You can define one or more methods to be executed before each test; typically, such methods initialize values, so that each test starts with a fresh set
 - ▶ @BeforeEach

```
public void setUp() {  
    program = new MyProgram();  
    array = new int[] { 1, 2, 3, 4, 5 };  
}
```
- ▶ You can define one or more methods to be executed after each test; typically, such methods release resources, such as files
 - ▶ @AfterEach

```
public void tearDown() {  
}
```

Exceptions

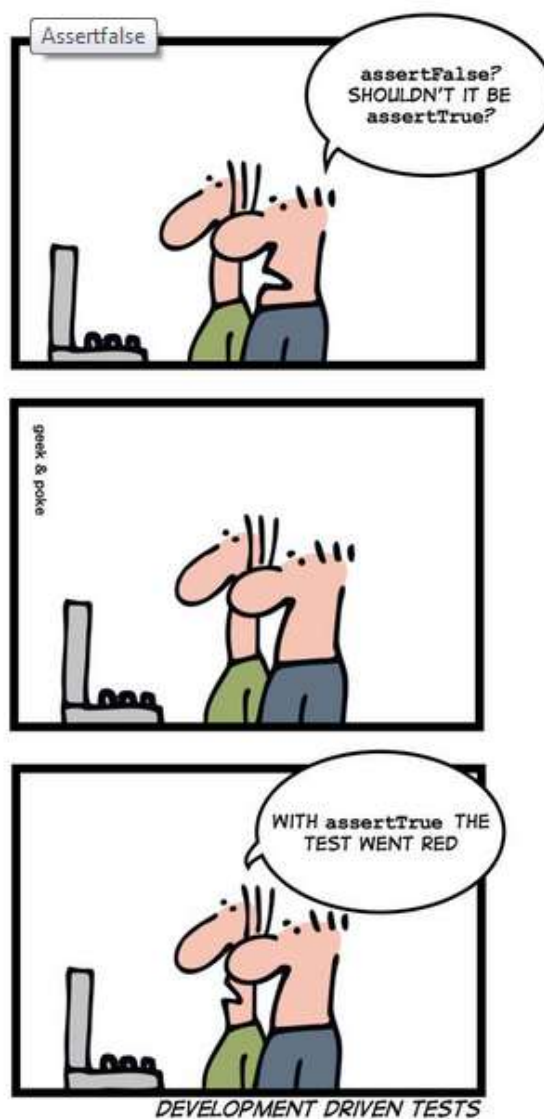
► JUnit 5

- `assertThrows(Class<T> expectedType, Executable executable)`

```
@Test
public void whenDerivedExceptionThrown_thenAssertionSucceeds() {
    Exception exception = assertThrows(RuntimeException.class, () ->
    {
        Integer.parseInt("1a");
    });

    String expectedMessage = "For input string";
    String actualMessage = exception.getMessage();

    assertTrue(actualMessage.contains(expectedMessage));
}
```



More Information

▶ JUnit

- ▶ <https://junit.org/>
- ▶ <https://docs.junit.org/5.6.2/user-guide/index.pdf>
- ▶ Tutorials
 - ▶ <https://www.guru99.com/junit-tutorial.html>
 - ▶ <https://www.vogella.com/tutorials/JUnit/article.html>

Some slides adapted from A. Stellman & J. Greene, “Applied Software Project Management” and Ian Sommerville, “Software Engineering”

18. Risk Management

“Software Project Management”
João Cunha, ISEC (jcunha@isec.pt)

Risk Plan

- ▶ A risk plan is a list of all risks that threaten the project, along with a plan to mitigate some or all of those risks.
 - ▶ The team members brainstorm potential risks
 - ▶ The probability and impact of each risk is estimated
 - ▶ A risk plan is constructed

-
- ▶ Project Risk is an uncertain event or condition that, if it occurs, has a positive or negative effect on, at least, one project objective, such as time, cost, scope or quality [PMBOK Guide 2004].
 - ▶ RISK is the possibility of suffering loss

Risk management

- ▶ Risk management is concerned with identifying risks and drawing up plans to minimise their effect on a project.
- ▶ A risk is the possibility that some adverse circumstance will occur
 - ▶ **Project risks** affect schedule or resources;
 - ▶ **Product risks** affect the quality or performance of the software being developed;
 - ▶ **Business risks** affect the organisation developing or procuring the software.

The risk management process

1. Risk identification

- ▶ Identify project, product and business risks;

2. Risk analysis

- ▶ Assess the likelihood and consequences of these risks;

3. Risk planning

- ▶ Draw up plans to avoid or minimise the effects of the risk;

4. Risk monitoring

- ▶ Monitor the risks throughout the project;

1. Risk identification

- ▶ Team activity
- ▶ A checklist of common risks may be used to identify risks in a project
 - ▶ Technology risks.
 - ▶ People risks.
 - ▶ Organisational risks.
 - ▶ Requirements risks.
 - ▶ Estimation risks.

Threshold of Success (ToS)

- ▶ The minimum set of conditions that must be met for the project to be considered successful
- ▶ If the team fails to meet **even one** of the conditions, then the project is a **failure**
- ▶ Just define 2-3 **SMART** goals.
 - ▶ **S**hort/**S**pecific, **M**easurable, **A**chievable, **R**elevant, and **T**ime bound.

How to create a ToS

- ▶ First create a picture of failure, then convert failure into success.

Ex:

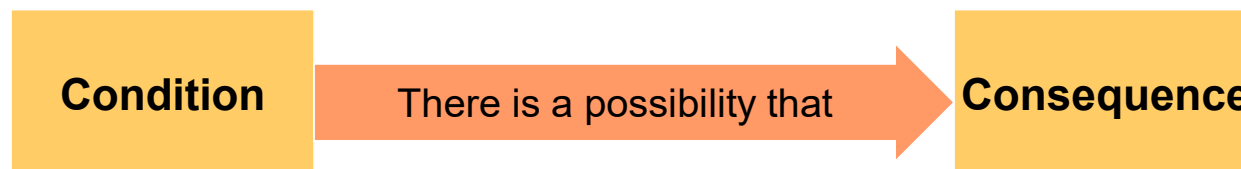
- ▶ The project fails if:
 - ▶ Essential features are not ready by the first release date.
 - ▶ Team members are dissatisfied or bored with their jobs.
- ▶ The project is successful if:
 - ▶ By the first release date, all "Must Have" features are implemented and pass acceptance tests with no known critical defects.
 - ▶ All team members give average score of 5 or better on a job satisfaction survey taken quarterly.

Risk Statement

- ▶ Risks are captured in statements to:
 - ▶ Help understand those issues/concerns that might stop the project from being successful, i.e, fail to meet the ToS.
 - ▶ Provide consistency in how the information is displayed within and across projects.
 - ▶ Understand and follow project concerns.
 - ▶ Better communicate concerns to project stakeholders
- ▶ Statements create a concise description of issues or concerns that can be understood and acted upon.

Guidelines

- ▶ Consider and capture those **fact-based conditions (true now!)** that are causing concern for a potential loss to the project;
- ▶ followed by a brief description of the **potential negative consequences** of these conditions.



What does a good statement look like?

- ▶ **Condition:** a single phrase or sentence that briefly describes the key circumstances, situations, etc., that have caused concern, doubt, anxiety or uncertainty.
- ▶ **Consequence:** a single phrase or sentence that describes the key negative outcome(s) that might result.
- ▶ **Example:**
 - ▶ **Tight schedules are causing people to work too much overtime;** (there is a possibility that this) **might cause project burnout and lower team morale.**

Examples of different risk types

Risk type	condition	consequence
Technology	The database used in the system cannot process as many transactions per second as expected;	performance requirements might not be met
People	Only one team member is expert in swing, and there are many screens to develop;	might delay GUI development, delaying the whole project.
Organizational	Organization has financial problems;	might force reductions in the project budget.
Tools	The code generated by software code generation tools is inefficient;	might compromise software quality.
Requirements	Customers fail to understand the impact of requirements changes;	requested changes might require major design rework .
Estimation	The team has no experience in software estimation;	might case underestimation, delaying the project in an uncontrolled way.

2. Risk analysis

- ▶ Assess probability and impact of each risk.
- ▶ Risk **impact** might be catastrophic(5), serious(3), or tolerable(1).
- ▶ **Probability** may be very low(1), low, moderate, high or very high(5).
- ▶ Prioritize risks according to **probability x impact**

Risk matrix

P x I		Impact				
		Tolerable (1)		Serious (3)		Catastrophic (5)
Probability	Very low (1)	Low (1)	Low (2)	Low (3)	Low (4)	Low (5)
	Low (2)	Low (2)	Low (4)	Low (6)	Low (8)	Medium (10)
	Moderate (3)	Low (3)	Low (6)	Low (9)	Medium (12)	Medium (15)
	High (4)	Low (4)	Low (4)	Medium (12)	Medium (16)	High (20)
	Very high (5)	Low (5)	Medium (10)	Medium (15)	High (20)	High (25)

3. Risk planning

- ▶ Consider the top risks (e.g. with probability x impact ≥ 20) and develop a strategy to mitigate those risks.
- ▶ **Avoidance strategies** (*Plano de Prevenção*)
 - ▶ The probability that the risk will arise is reduced;
- ▶ **Minimisation strategies** (*Plano de Minimização*)
 - ▶ The impact of the risk on the project or product will be reduced;
- ▶ **Contingency plans** (*Plano de Contingência*)
 - ▶ If the risk arises, contingency plans are to deal with that problem;

- ▶ Add actions to:
 - ▶ Product or Sprint Backlog
 - ▶ Sprint retrospective

4. Risk monitoring

- ▶ Assess each identified risks **regularly** to decide whether or not it is becoming less or more probable.
- ▶ Also assess whether the effects of the risk have changed.
- ▶ Each key risk should be discussed at management progress meetings.

Example Risk Plan

- ▶ Threshold of Success
 - ▶ All “Must Have” US are Done by the end of sprint 4.
 - ▶ ...
- ▶ Prioritized Risk List
 - ▶ **Risk4** – Pxl: $4 \times 5 = 20$; In December, students have heavy workload due to other courses; overwork may reduce focus on quality, and delay completion of US 10.
 - ▶ ...
- ▶ Mitigation Actions (risk level ≥ 20)
 - ▶ **Risk4** (avoidance strategy) – **Task4**: run a Spike on US 10 during sprint 3, to ensure the team can confidently complete it in sprint 4
 - ▶ **Risk4** (minimization strategy) – **Task5**: break US 10 into smaller subtasks, where some of which may be classified as “should have”, so that the impact of not finishing US 10 is smaller
 - ▶ **Risk4** (contingency plan) – ...

Example Risk Plan

- ▶ Threshold of Success

- ▶ All “Must Have” US are Done by the end of Sprint 4.
- ▶ ...

- ▶ Prioritized Risk List

- ▶ **Risk5** – Pxl: $5 \times 5 = 20$; Team members are not sufficiently committed, delaying planned deliverables; Lack of accountability may cause Must Have user stories to slip beyond Sprint 4.
- ▶ ...

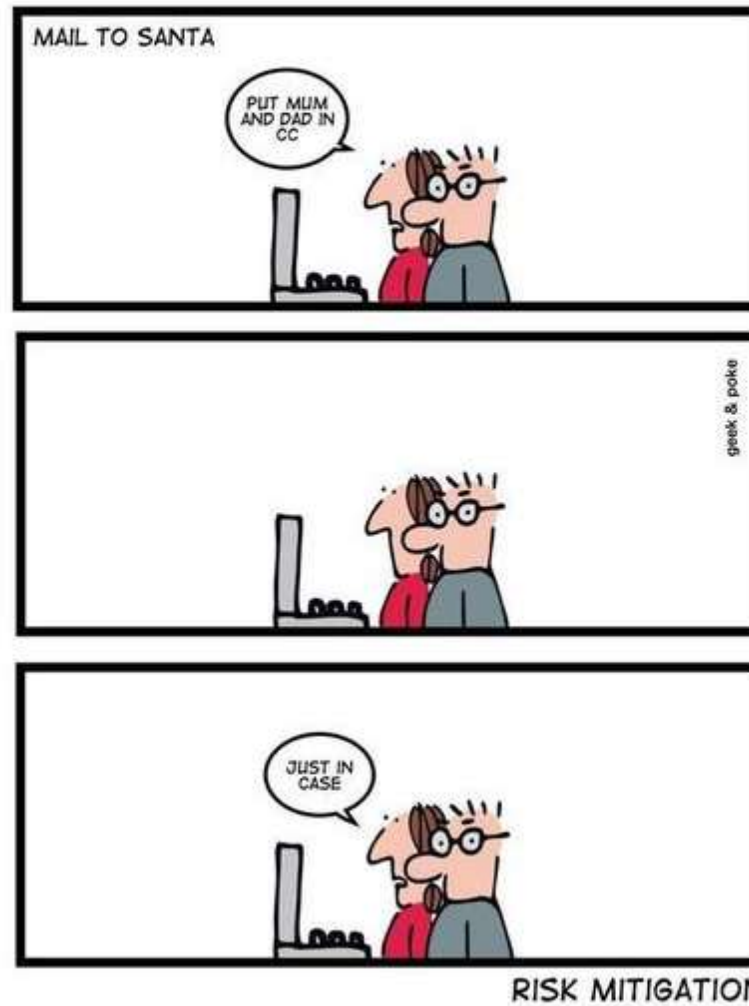
- ▶ Mitigation Actions (risk level ≥ 20)

- ▶ **Risk5** (avoidance strategy) – Retrospective meeting 2 – Action: define and sign a Team Commitment Agreement specifying each member’s responsibilities, expected behaviours, and shared accountability rules, including clear consequences for non-compliance; Responsible: Scrum Master
- ▶ ...

Further reading

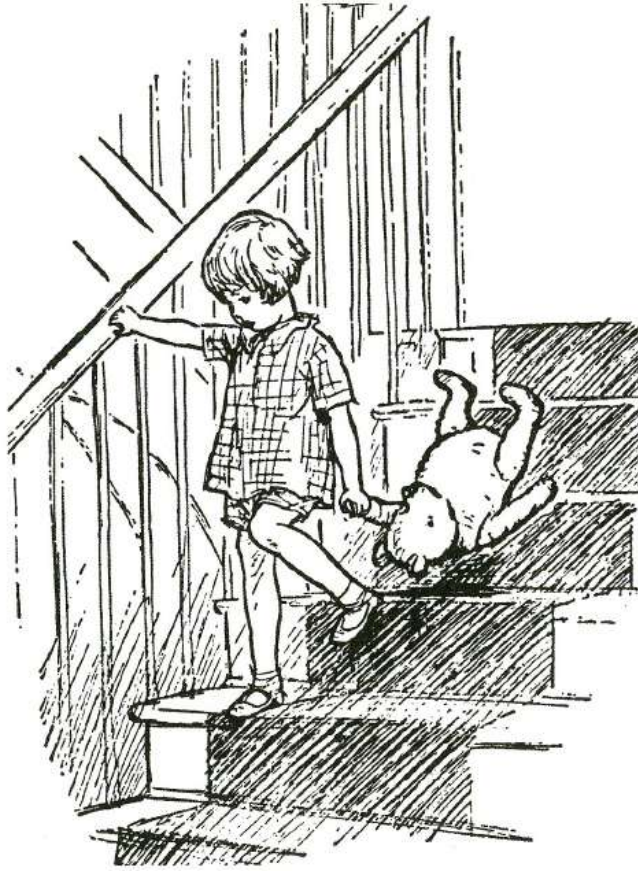
- ▶ Ian Sommerville, *Software Engineering 9*, Pearson Education, 2011. [Section 22.1]
- ▶ Some slides adapted from Ian Sommerville, “Software Engineering”

SIMPLY EXPLAINED



19. Traditional Process Models

“Software Project Management”
João Cunha, ISEC (jcunha@isec.pt)

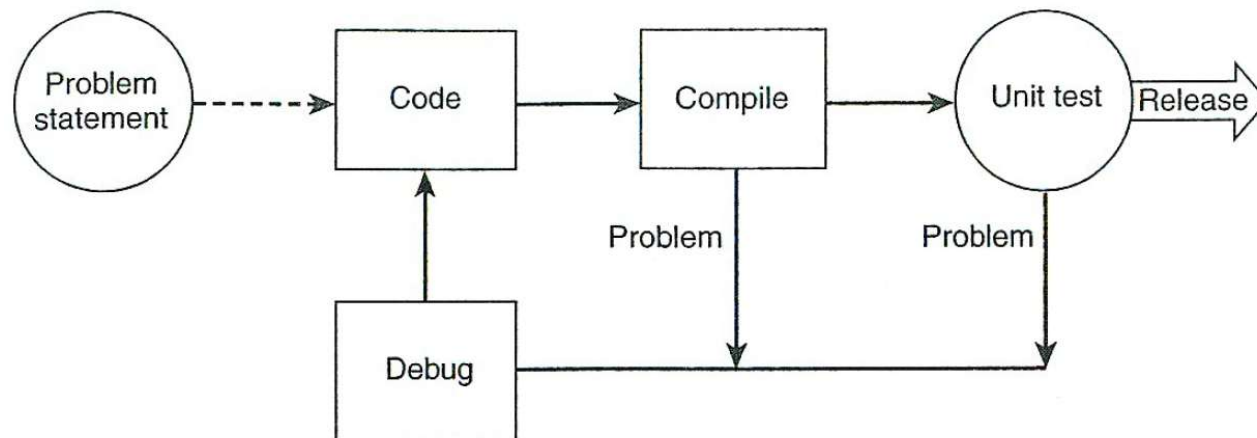


*Here is Edward Bear, coming downstairs now, bump, bump, bump on the
back of his head, behind Christopher Robin.*

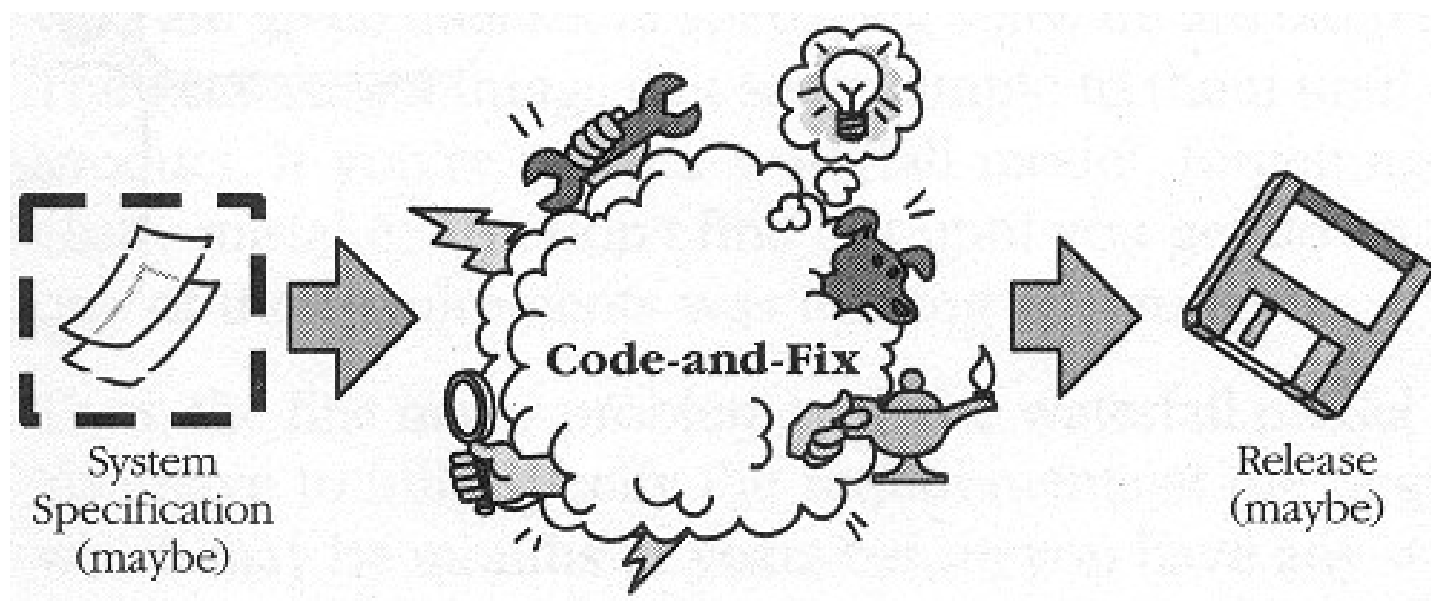
*It is, as far as he knows, the only way of coming downstairs,
but sometimes he feels that there really is another way,
if only he could stop bumping for a moment and think of it.
And then he feels that perhaps there isn't.*

Code-and-fix

1. Discuss the software product
2. Write some code
3. Test to detect defects
4. Debug to find the cause of the defects
5. Correct the defects
6. If not done, go back to 1.

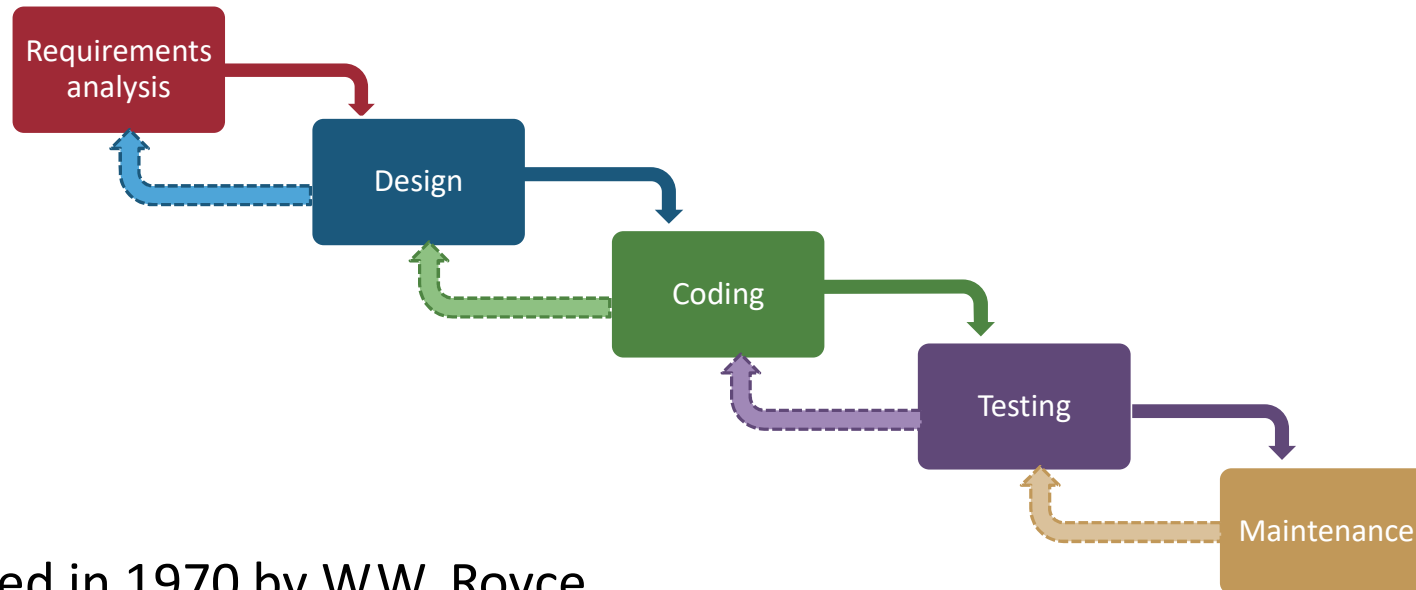


What Code-and-fix really is



[From McConnel]

Waterfall lifecycle



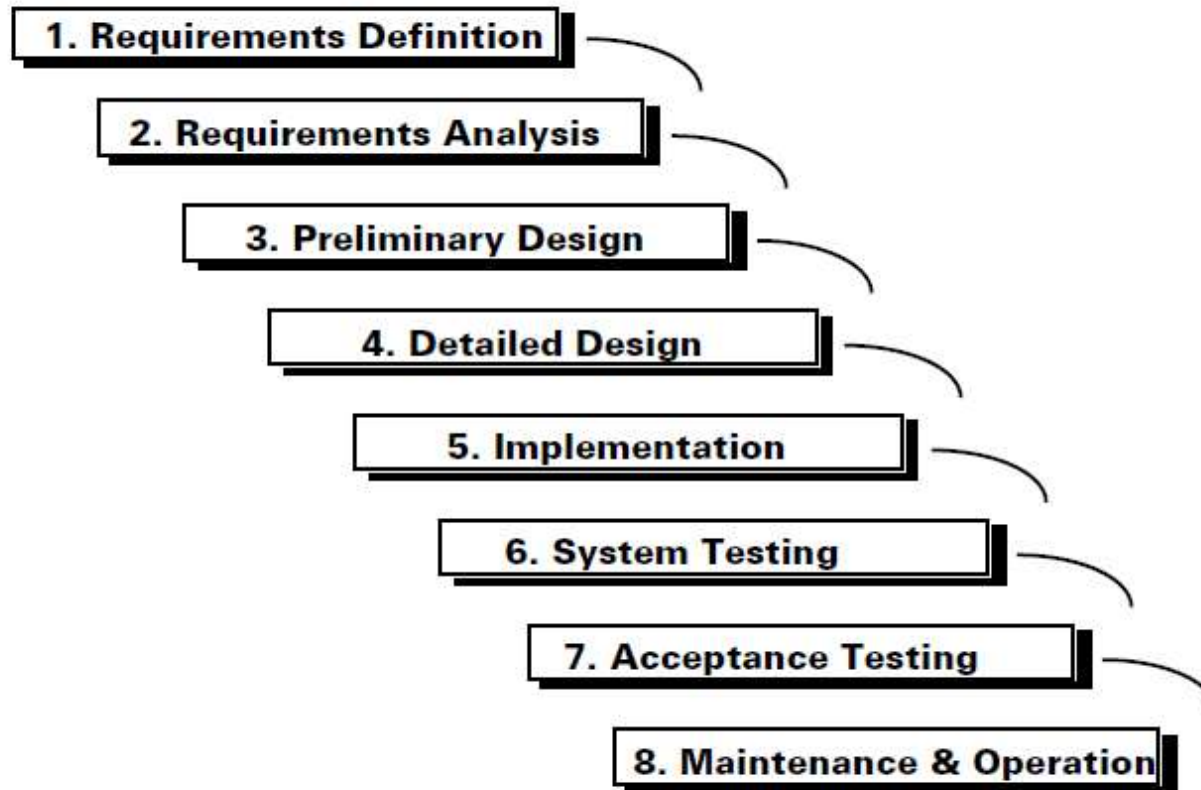
- ▶ Proposed in 1970 by W.W. Royce
- ▶ Models a systematic sequential approach
 - ▶ But real projects rarely follow the sequential flow
 - ▶ When something goes wrong, go back to a previous step (expensive)

Analysis

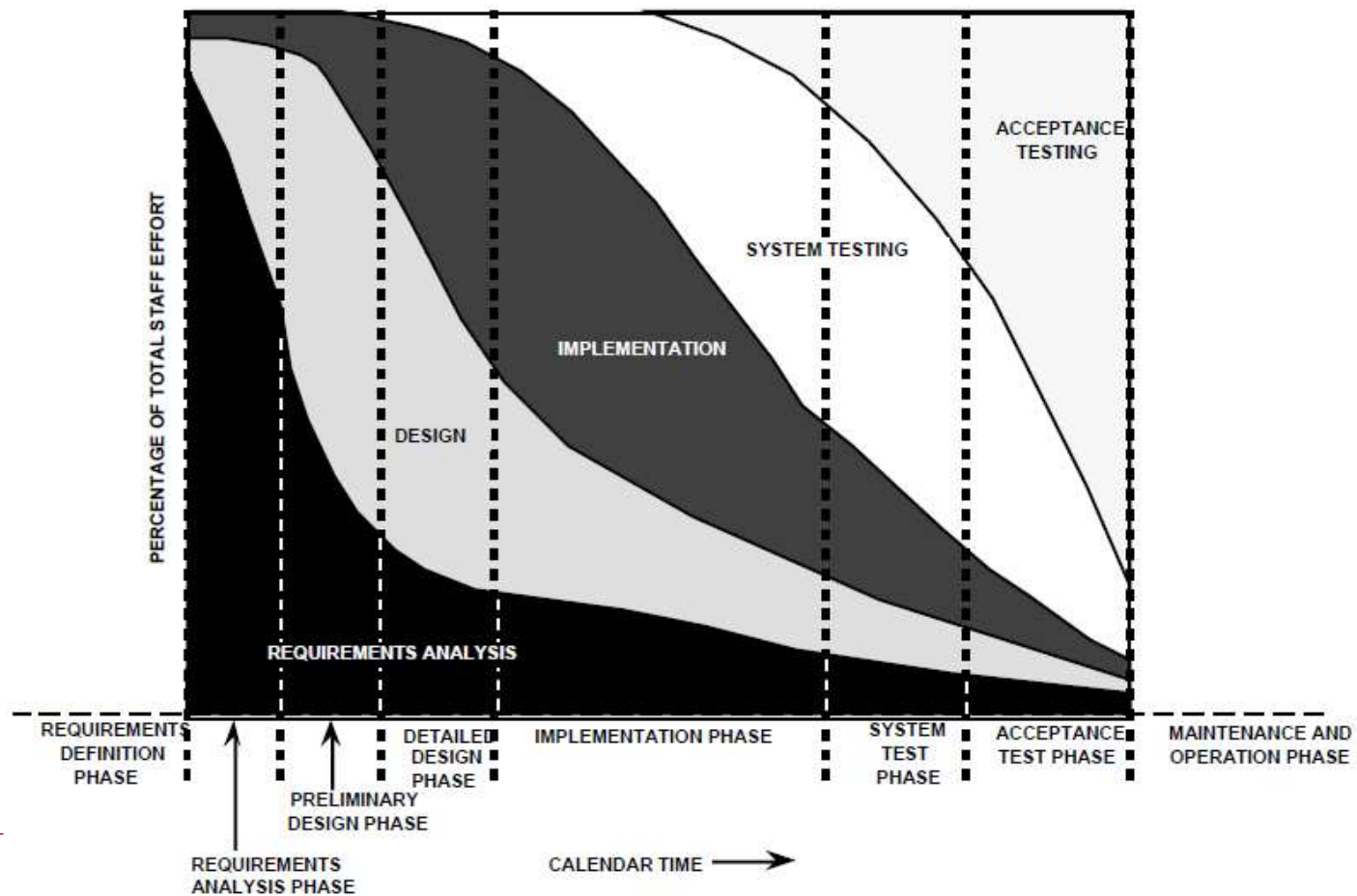
- ▶ Requirements must be specified in the first step
 - ▶ Difficult to accommodate the natural uncertainty at the beginning of the projects
- ▶ Models a systematic sequential approach
 - ▶ Real projects rarely follow the sequential flow
- ▶ The result of each phase is one or more approved documents
 - ▶ Document-driven approach – costly iterations
 - ▶ Prematurely freezing requirements may mean the system doesn't do what the user wants

- ▶ Works well when requirements are well understood, and technologies known

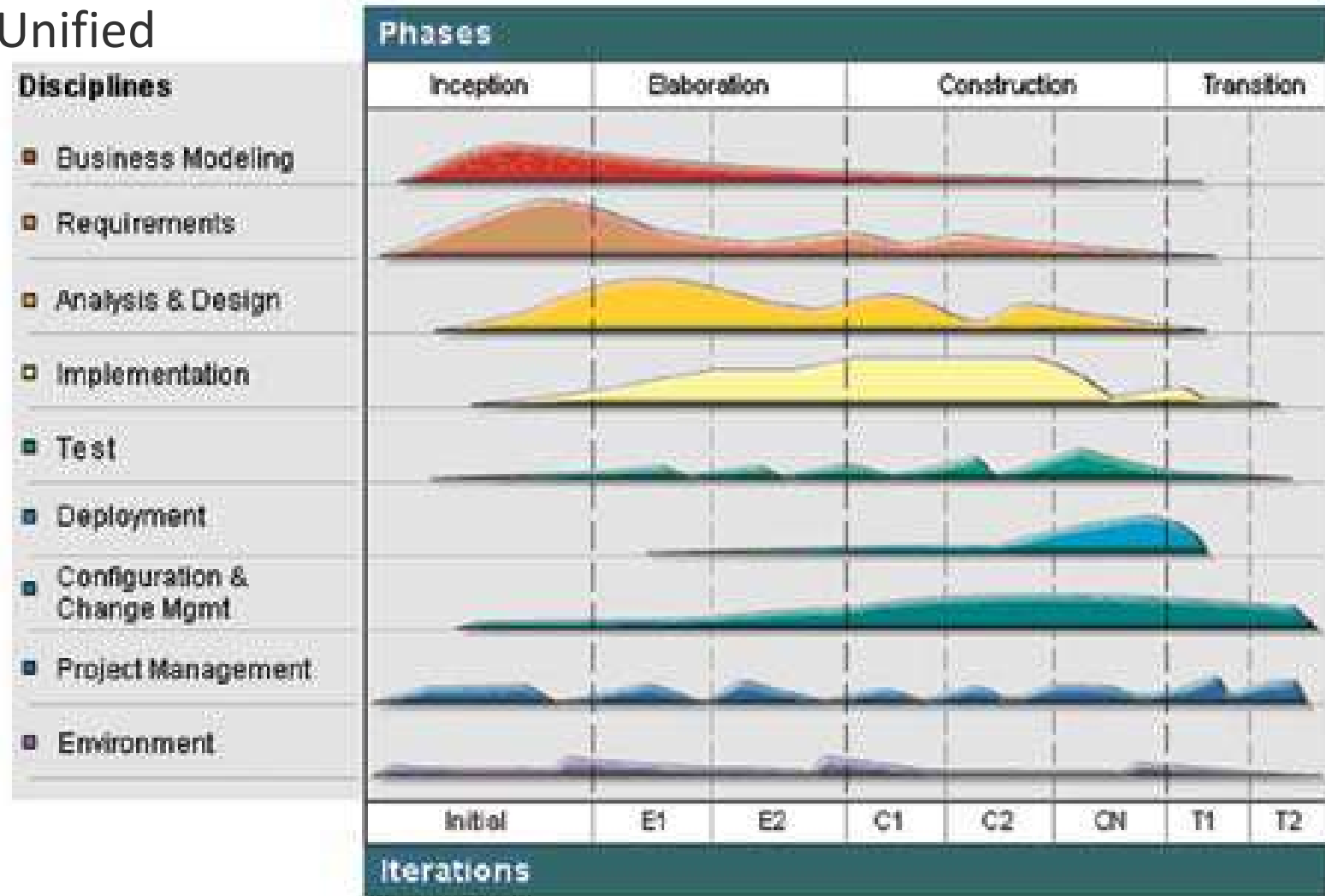
NASA-Software Engineering Laboratory recommended software development life cycle



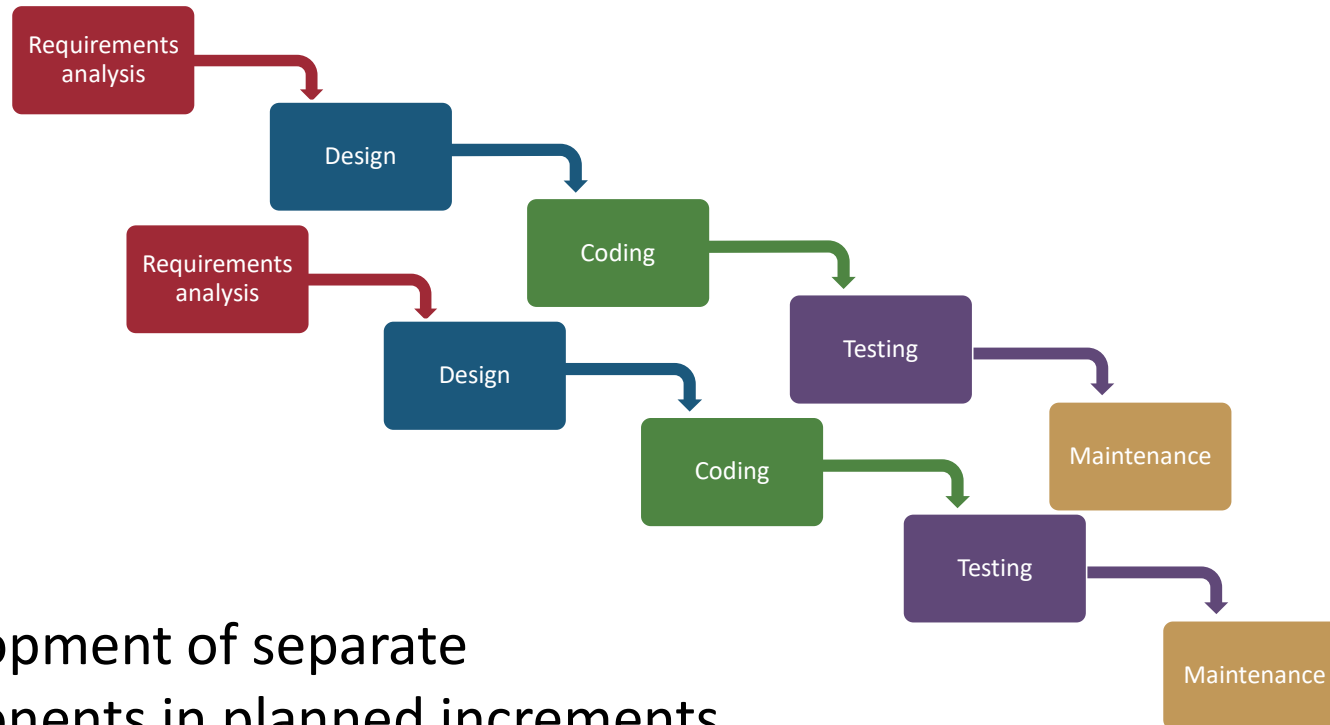
NASA-SEL – Activities by Staff Effort



RUP – Rational Unified Process



Incremental

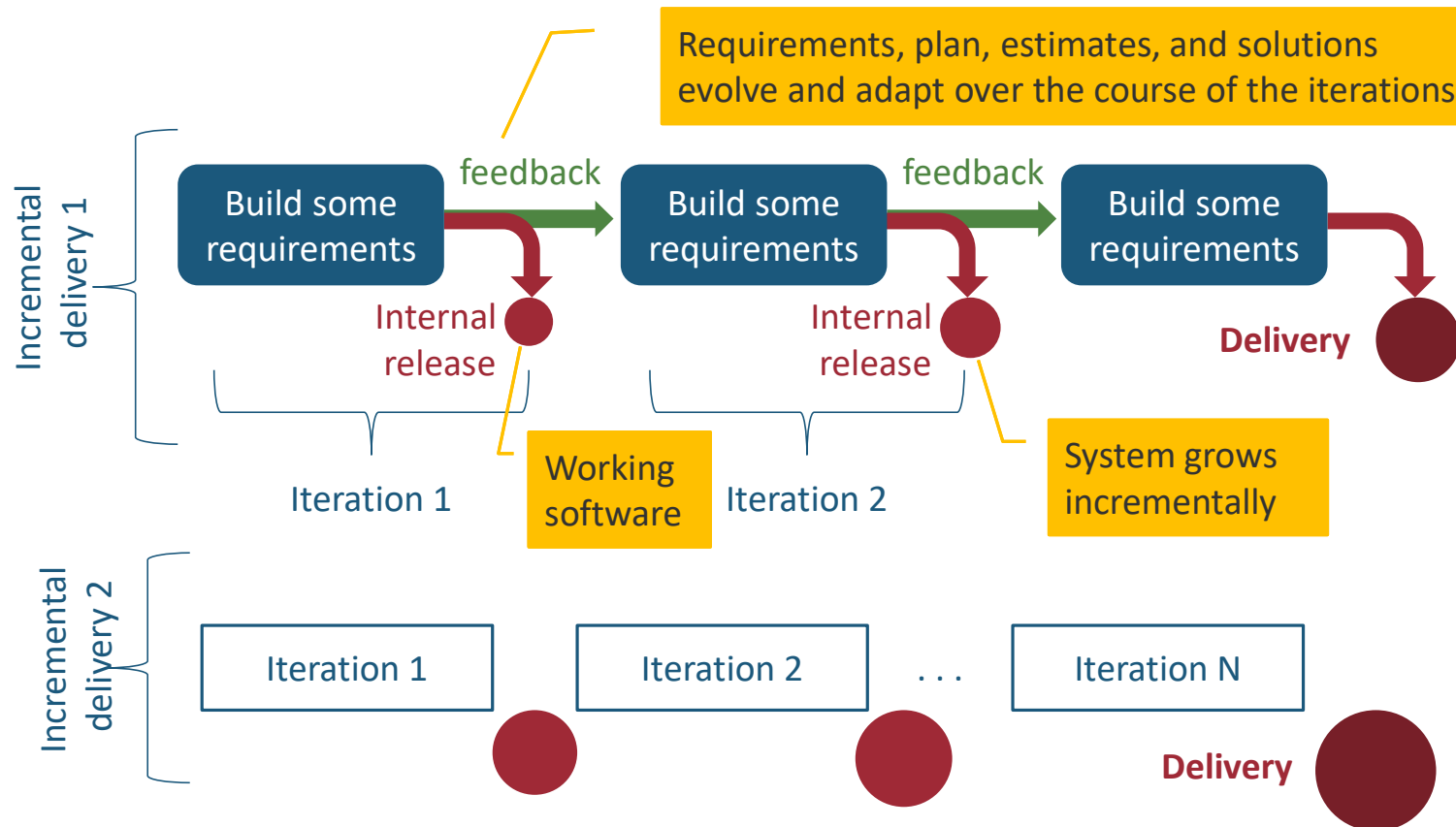


- ▶ Development of separate components in planned increments
 - ▶ Progress and problems get visible earlier

Incremental

- ▶ Development of separate components incrementally (and maybe iteratively)
 - ▶ Risk containment
 - ▶ Large products get decomposed into parts
 - ▶ Problems get visible earlier
 - ▶ Increments are planned
- ▶ Each increment may be released to the customer
 - ▶ Progress gets more visible to the team and client
 - ▶ First increment is the core product

Iterative, evolutionary, adaptive and incremental delivery



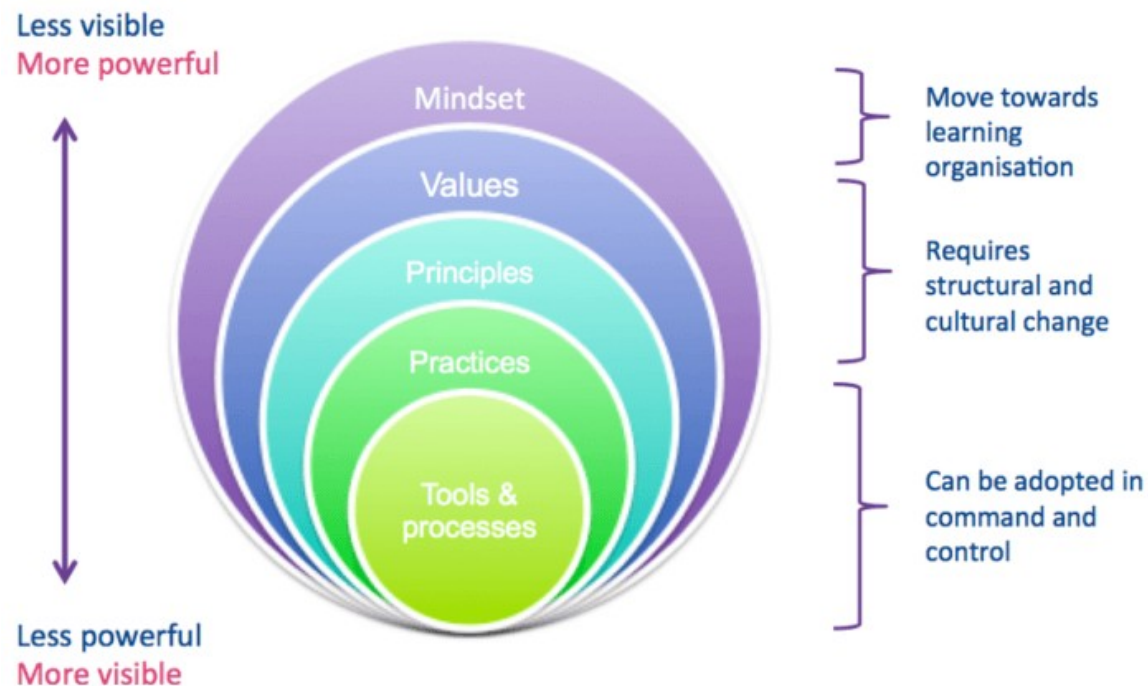
What makes a method Agile?

- ▶ Short iterations
- ▶ Incremental design
- ▶ User involvement
- ▶ Minimal documentation
- ▶ Informal communication
- ▶ Adaptive planning
- ▶ Evolutionary delivery

remember:

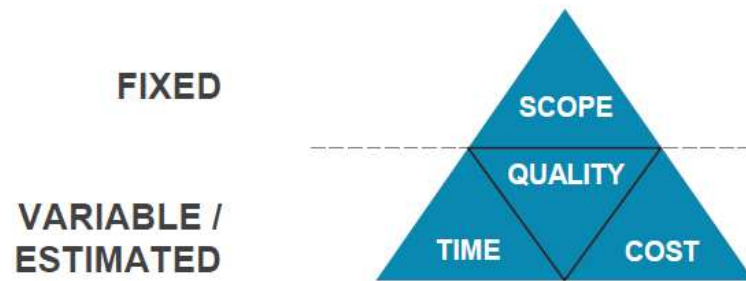
- ▶ **4 core Agile Values**
- ▶ **12 Agile Principles**

Agile is not only tools and processes

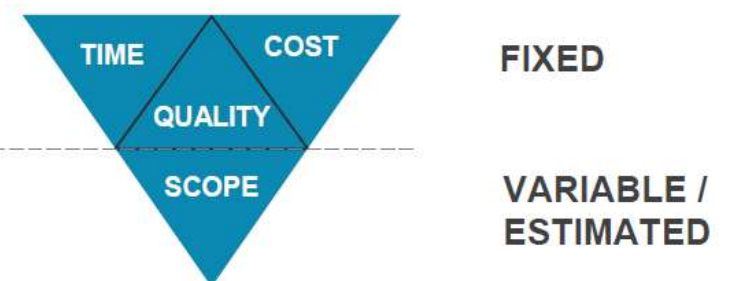


Motivation

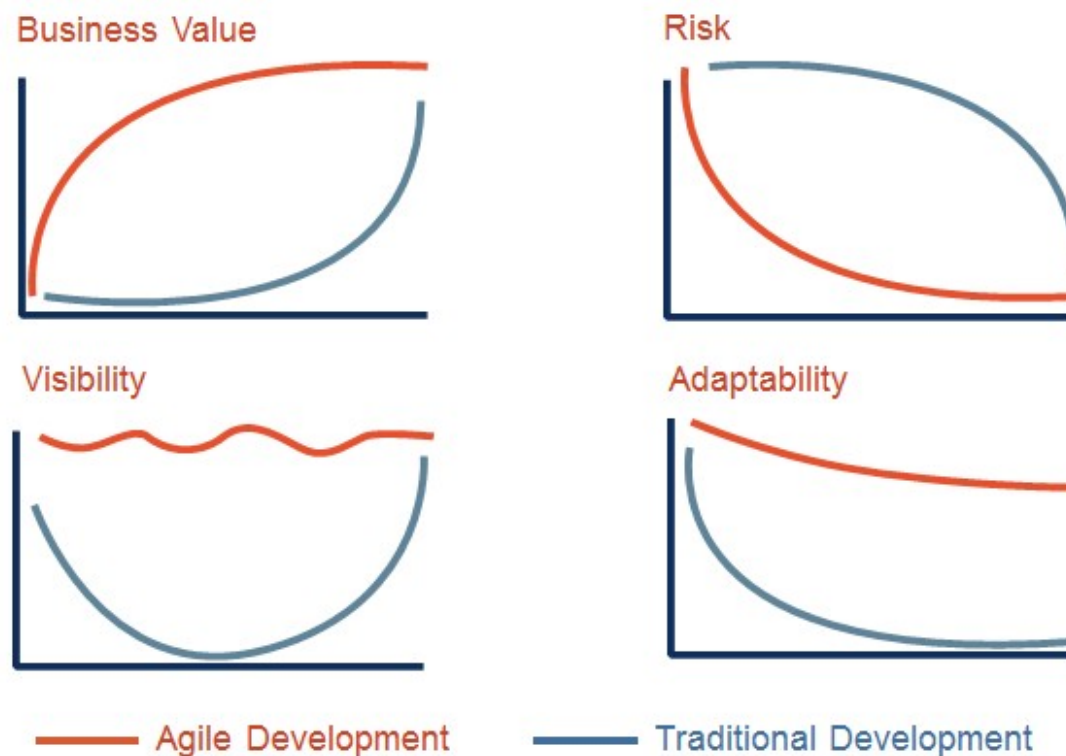
Tradicional



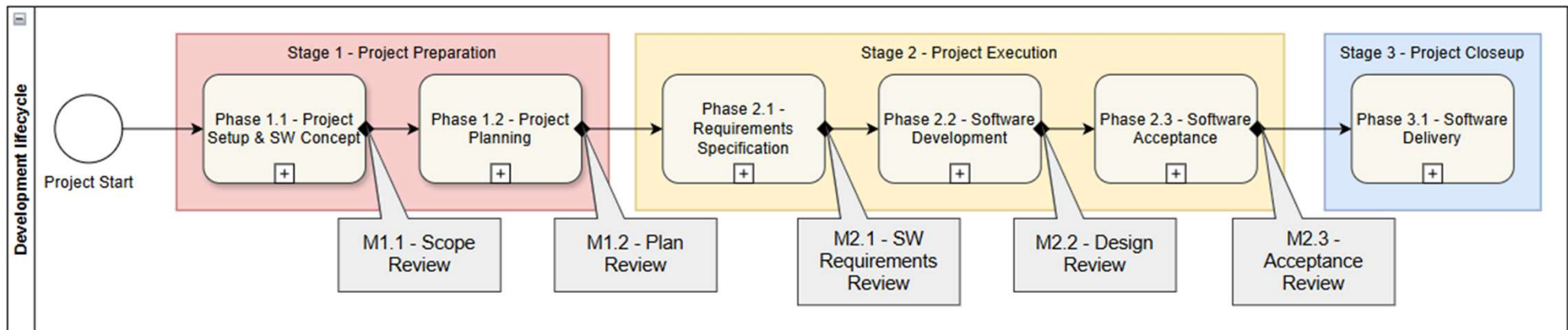
Agile



Motivation



GPS 2022/2023 – Processes

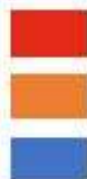


► GPS2223 - SDLC

GPS 2022/2023 - Project Lifecycle

Stages:

1. Project Preparation
2. Project Execution
3. Project Closeup



Phases:

- 1.1 - Project Setup & Software Concept
- 1.2 - Project Planning
- 2.1 - Requirements Specification
- 2.2 - Software Development
- 2.3 - Software Acceptance
- 3.1 - Project Closeup

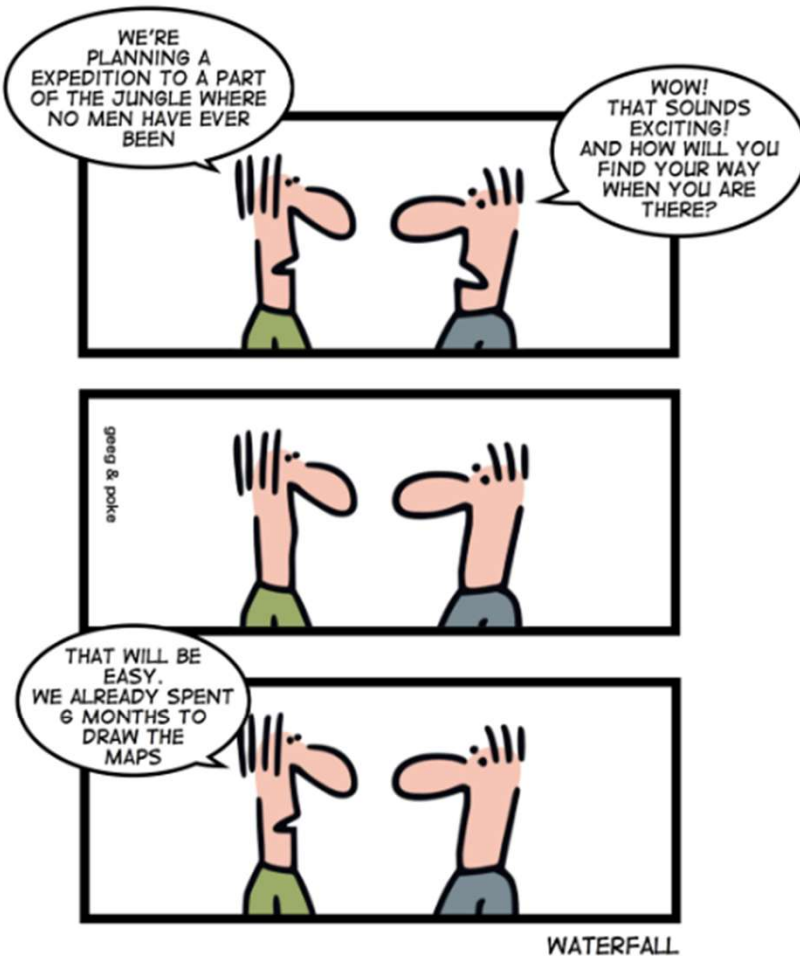
Milestones:



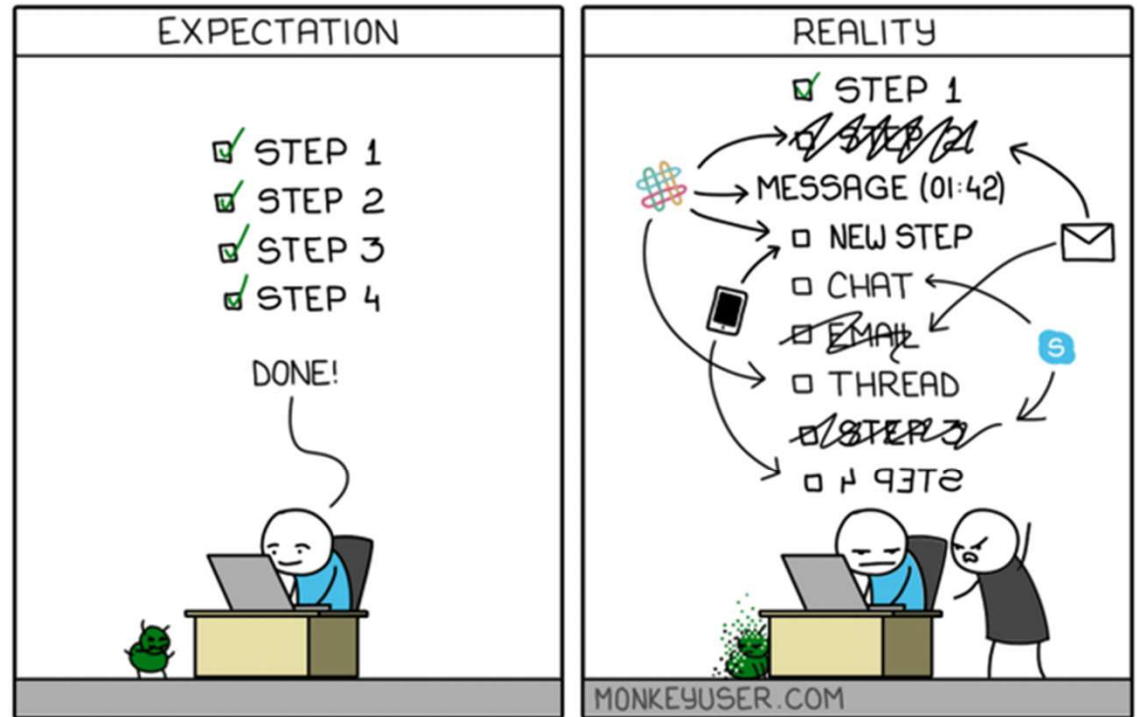
GPS 2022/2023 - Deliverables

Documents				Milestones					
Stage	Phase	ID	Name	M1.1	M1.2	M2.1	M2.2	M2.3	M3.1
1	1.1	D1.1.1	Vision & Scope (VS)	X					
		D1.1.2	Milestone M1.1 Report (M11)	X					
	1.2	D1.2.1	Software Development Plan (SDP)		X				
		D1.2.2	Quality Assurance Plan (QAP)		X				
		D1.2.3	Milestone M1.2 Report (M12)		X				
2	2.1	D2.1.1	Software Requirements Specification (SRS)			X			
		D2.1.2	Risk Plan (RP)			X			
		D2.1.3	Acceptance Test Plan (ATP)			X			
		D2.1.4	Milestone M2.1 Report (M21)			X			
	2.2	D2.2.1	Software Architecture & Design (SAD)				X		
		D2.2.2	Source code				X	X	
		D2.2.3	Milestone M2.3 Report (M22)				X		
	2.3	D2.3.1	Acceptance Test Report (ATR)		X			X	
		D2.3.2	Quality Assessment Report (QAR)		X	X	X	X	
		D2.3.3	Milestone M2.3 Report (M23)					X	
3	3.1	D3.1.1	Post-Mortem Analysis (PMA)						X

SIMPLY EXPLAINED



USER STORY



WATERFALL

SEQUENTIAL ENGINEERING



AGILE

INCREMENTAL DELIVERY



AI BUILD

PROTOTYPE TO PLATEAU



VIBE CODING

NO PROCESS CHAOS

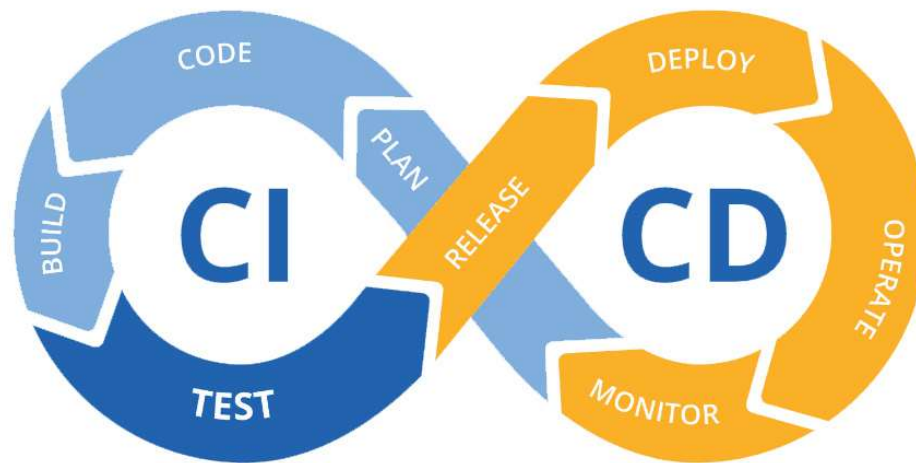


Bibliography

- ▶ Difference between Agile and Waterfall
https://www.youtube.com/watch?v=5RocT_OdQcA
- ▶ Agile Alliance <https://www.agilealliance.org/>
- ▶ Waterfall model definition and example
▶ https://www.youtube.com/watch?v=Y_A0E1ToC_I
- ▶ Waterfall methodology — a complete guide
▶ <https://business.adobe.com/blog/basics/waterfall>

20. CI/CD and DevOps

“Software Project Management”
João Cunha, ISEC (jcunha@isec.pt)



Continuous Integration

- ▶ **Goal:** to automate the integration of code changes from multiple contributors into a shared repository on a frequent and regular basis
- ▶ Key aspects:
 - ▶ **Automated builds** - code changes are automatically compiled and built into executable formats.
 - ▶ **Automated tests** - Code changes are subjected to a suite of automated tests to ensure they do not introduce bugs or regressions.
 - ▶ **Version Control** - All code changes are tracked in a version control system, enabling collaboration and rollback if necessary.
- ▶ **Advantages:**
 - ▶ Fast feedback to developers.
 - ▶ Small, frequent commits reduce merge conflicts.
 - ▶ One single source of truth in version control.

Continuous Delivery

- ▶ **Goal:** to automatically prepare the code for release to production, ensuring it's always in a deployable state.
 - ▶ Every successful build is ready for deployment.
 - ▶ Automated packaging and preparation for staging or production.
 - ▶ Quality enforced through automated checks.
 - ▶ Software always in a releasable state.
 - ▶ Easy rollbacks when necessary.
- ▶ **Manual approval** gate before production

Continuous Deployment

- ▶ **Goal:** to automatically deploy to production when automated tests pass.
 - ▶ Fully automated pipeline from commit to production.
 - ▶ Requires high confidence in automated testing
 - ▶ Safe release strategies: feature flags, canary, blue/green.
 - ▶ Real-time feedback from production environments.
 - ▶ Automatic rollback if deployment fails.
- ▶ **No manual steps** in the release process.

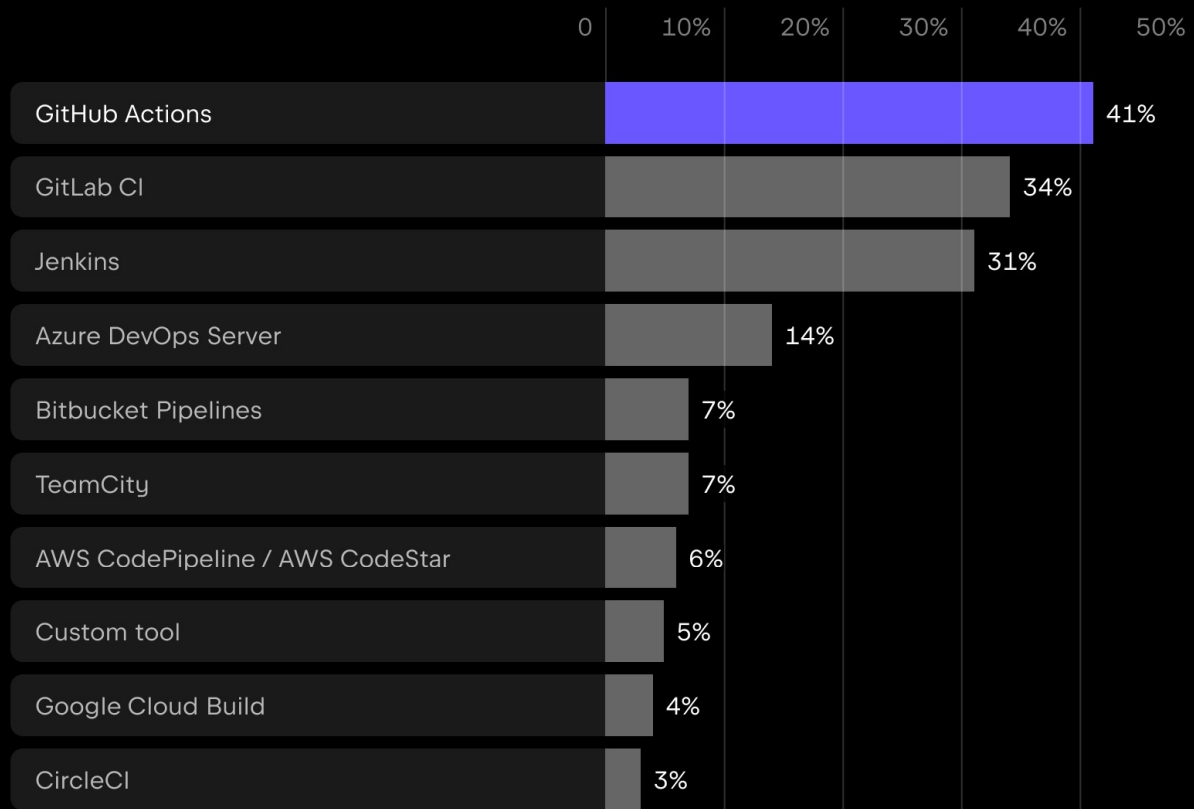
Continuous Delivery vs Continuous Deployment

Aspect	Continuous Delivery	Continuous Deployment
Production deployment	Manual approval required	Fully automated
Release frequency	As needed (on-demand)	Every successful build
Risk tolerance	Lower (manual gate provides control)	Higher (requires robust automation)
Feedback speed	Moderate	Very fast
Required maturity	Moderate	High

CI/CD Pipeline

- ▶ **Automated** sequence of stages like build, testing, packaging
- ▶ Pipeline as code usually described in **YAML** (for example `.gitlab-ci.yml`)
- ▶ **Quality gates**: tests, security checks, validations
- ▶ **Monitoring and logging** integrated into the pipeline
- ▶ **Continuous cycle**: commit, pipeline, deploy
- ▶ Ability to scale jobs, runners and complexity
- ▶ Shorter time between coding and releasing
- ▶ Releases become predictable and repeatable

Which CI/CD tools are currently used in your organization?



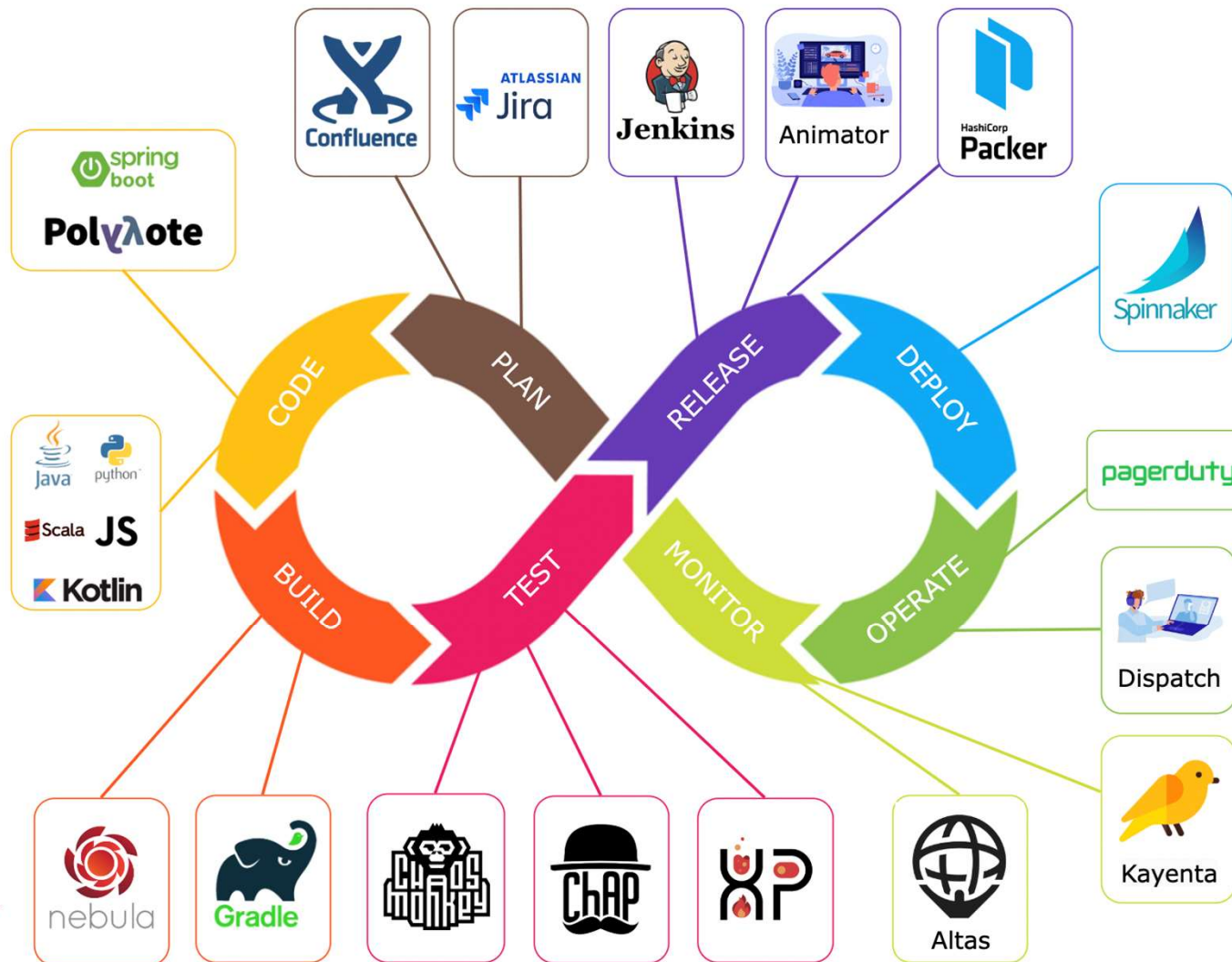
The State of CI/CD in 2025

jb.gg/state-of-cicd



NETFLIX Tech Stack (CI/CD Pipeline)

 blog.bytebytego.com



Netflix CI/CD Pipeline

1. Branching Model

- ▶ Developers work on feature branches
- ▶ Pull requests require automated tests to pass before merging
- ▶ Approved changes get merged into the main branch

2. Build & Test Automation

- ▶ Integrate with Jenkins and Gradle to run automated builds
- ▶ Automated tests (unit, integration, etc.) run as part of the pipeline

3. Deployment Automation

- ▶ Spinnaker coordinates deployments
- ▶ Deployments run on AWS infrastructure

4. Canary Testing

- ▶ New changes are first released to a small subset of users (canary)
- ▶ Monitor behavior and metrics in that subset before a full rollout

5. Monitoring & Alerting

- ▶ Use Atlas and Spectator for real-time observability
- ▶ Alerts are triggered based on predefined thresholds / anomalous behavior

6. Static Code Analysis

- ▶ Tools like SonarQube and Checkstyle for analyzing code quality
- ▶ Checks include code smells, duplications, vulnerabilities
- ▶ If static analysis fails, pull requests are blocked until issues are fixed

<https://medium.com/@seyhunak/learning-best-practices-from-netflix-tech-stack-ci-cd-pipeline-a25a58f46711>

DevOps

- ▶ A combination of software development (dev) and operations (ops)
- ▶ A set of cultural philosophies, practices, and tools that combine software development and IT operations to automate and integrate processes, enabling faster, more reliable software delivery
- ▶ Focus on collaboration, transparency and shared ownership
- ▶ Heavy use of automation and Infrastructure as Code
- ▶ Continuous improvement mindset
- ▶ Monitoring and observability as part of the lifecycle

DevOps Principles

- ▶ Automate everything that is repetitive or error-prone
- ▶ Communicate and collaborate continuously
- ▶ Use feedback loops (testing, monitoring, logs)
- ▶ Infrastructure treated as code for consistency
- ▶ Integrate security early (DevSecOps)
- ▶ Systems designed for resilience and scalability

DevOps Culture and Organization

- ▶ Shared ownership of code and operations
- ▶ Integration of pipelines with agile processes
- ▶ Automation as a mindset, not a feature
- ▶ Continuous learning and improvement
- ▶ Constant communication and open feedback

CI/CD vs DevOps

- ▶ CI/CD is a technical practice
- ▶ DevOps is a cultural and organizational approach
- ▶ CI/CD can exist without DevOps but is weaker
- ▶ DevOps without automation becomes inefficient
- ▶ CI/CD increases speed
- ▶ DevOps increases alignment and quality
- ▶ Both aim at faster and more reliable delivery

Best Practices for Starting with CI/CD

- ▶ Start small: a simple pipeline with build and tests
- ▶ Use short-lived branches and frequent commits
- ▶ Standardize templates for pipelines
- ▶ Automate testing early
- ▶ Keep pipelines fast to ensure quick feedback
- ▶ Document pipeline stages for the team

Further reading

- ▶ <https://www.geeksforgeeks.org/system-design/cicd-pipeline-design-principles>
- ▶ <https://www.atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment>
- ▶ <https://www.refontelearning.com/blog/mastering-ci-cd-best-online-courses-for-devops-engineers>
- ▶ GitLab CI/CD Pipeline Tutorial for Beginners <https://youtu.be/z7nLsJvEyMY>

21. Metrics and KPI

“Software Project Management”
João Cunha, ISEC (jcunha@isec.pt)

Metrics and KPIs

▶ What Are Metrics?

- ▶ Quantitative measures of activities, processes, or outcomes
- ▶ Used to understand performance and quality
- ▶ Examples from daily life (fitness apps, games, sports)

▶ What Are KPIs?

- ▶ Key Performance Indicators
- ▶ A subset of metrics focused on strategic goals
- ▶ KPIs help measure progress toward outcomes, not just activity

- ▶ All KPIs are metrics but not all metrics are KPIs

- ▶ KPIs align with team or business objectives

- ▶ Examples:

- ▶ Metric: Number of tests

- ▶ KPI: Defect leakage after release

Why we measure?

- ▶ Support decision-making
- ▶ Identify problems earlier
- ▶ Improve predictability
- ▶ Improve team communication
- ▶ Reduce waste and rework

Characteristics of good and bad metrics:

▶ Good metrics:

- ▶ Meaningful
- ▶ Traceable
- ▶ Comparable over time
- ▶ Actionable
- ▶ Aligned with team goals
- ▶ Examples:
 - ▶ Defect Leakage (Bugs found after release vs. before release)
 - ▶ Team Satisfaction (Anonymous check)
 - ▶ Test Coverage (Useful to detect untested areas)

▶ Bad metrics:

- ▶ Easy to game
- ▶ Focus on individual performance
- ▶ Cause pressure instead of improvement
- ▶ Lack context
- ▶ Encourage vanity results
- ▶ Examples:
 - ▶ Number of Commits (Encourages spam commits)
 - ▶ Hours Worked (Promotes burnout)
 - ▶ Test Coverage (Encourage useless tests for 100% coverage)

CONTEXT MATTERS

Examples of Metrics - Productivity

- ▶ Concern productivity as a team, not individuals
- ▶ Focus on outcomes completed each sprint
- ▶ **Throughput**
 - ▶ Number of PBIs completed in a sprint
 - ▶ Count all items marked Done in the Sprint Backlog
- ▶ **Velocity** (for forecasting only)
 - ▶ Sum of completed story points per sprint.
 - ▶ Add the story point estimates of all Done items.

Examples of Metrics - Predictability

- ▶ Assess how consistently the team meets its sprint commitments
- ▶ Support short-term forecasting and planning
- ▶ **Sprint/Release Burndown**
 - ▶ Shows remaining work in the Backlog over time, to indicate whether the Goal remains achievable.
 - ▶ Sum remaining work and plot it daily; the slope indicates whether work is burning down as expected
- ▶ **Forecast Accuracy (Planned vs Delivered)**
 - ▶ Degree to which delivered work matches planned work
 - ▶ $\text{Delivered PBIs or points} / \text{planned PBIs or points}$
- ▶ **Work Spillover Rate**
 - ▶ Proportion of sprint work not completed and moved to another sprint.
 - ▶ $\text{Carry-over items} / \text{total planned items}$

Examples of Metrics - Flow

- ▶ Measure how efficiently work moves from “to do” to “done”
- ▶ Useful for identifying bottlenecks and delay
- ▶ Mainly used in Kanban
- ▶ **Cycle Time**
 - ▶ Time from when the team starts working on a PBI until it is Done
 - ▶ Done timestamp (-) In-Progress timestamp.
- ▶ **Lead Time**
 - ▶ Total time from a PBI becoming Ready (or start of sprint) until Done.
 - ▶ Done timestamp (-) Ready timestamp.
- ▶ **Work in Progress (WIP)**
 - ▶ Number of PBIs currently being worked on.
 - ▶ Count items in active workflow columns.
- ▶ **Blocker Time / Blocker Frequency**
 - ▶ Time PBIs remain blocked or number of times blocks occur.
 - ▶ Sum of blocked durations or count block events.

Examples of Metrics - Quality

- ▶ Ensure increments meet the Definition of Done
- ▶ Detect defects early and maintain product integrity
- ▶ **Defect Density**
 - ▶ Number of defects relative to the size of delivered work.
 - ▶ Total defects / unit of work (e.g. PBI, feature, KLOC).
- ▶ **Escaped Defects**
 - ▶ Definition: defects found after the increment is released.
 - ▶ Calculation: count defects reported post-release.
- ▶ **Reopened Issues**
 - ▶ Work items that return to development after being marked Done.
 - ▶ Count of items reopened during or after the sprint.
- ▶ **Definition of Done Compliance**
 - ▶ Degree to which completed PBIs meet all DoD criteria.
 - ▶ Compliant PBIs / total completed PBIs

Examples of Metrics - Team Health

- ▶ Assess sustainability, collaboration, and team cohesion
- ▶ Not productivity measure!
- ▶ **Team Satisfaction**
 - ▶ Short anonymous rating of morale or perceived team health.
 - ▶ Average of 1–5 or 1–10 ratings.
- ▶ **Impediment Resolution Time**
 - ▶ Time needed to remove blockers or impediments.
 - ▶ Impediment closed date (-) opened date.
- ▶ **Knowledge Distribution**
 - ▶ Robustness of the team against knowledge concentration.
 - ▶ Number of people critical to system survival

Examples of Metrics - Value

- ▶ Measure whether the product delivers meaningful outcomes to users and stakeholders
- ▶ Focus on *value* delivered, not team output
- ▶ **Sprint Goal Success Rate**
 - ▶ Percentage of sprints in which the Sprint Goal is fully achieved
 - ▶ Successful sprints / by total sprints
- ▶ **Customer Satisfaction**
 - ▶ User perception of product value or experience.
 - ▶ Average score from satisfaction surveys.
- ▶ **Delivered Value (Business Value Points)**
 - ▶ How much product value (as assessed by stakeholders or the Product Owner) is actually delivered per Sprint or Release
 - ▶ Sum the business value points of items reaching Done in a Sprint or Release.

Which Metrics Are Suitable as KPIs?

▶ **Valid KPIs (aligned with Scrum and empiricism)**

- ▶ Sprint Goal Success Rate (reflects outcome alignment with goals)
- ▶ Cycle Time, Lead Time (reflect flow efficiency and responsiveness)
- ▶ Escaped Defects, Test Coverage (indicate quality outcomes)
- ▶ Impediment Resolution Time (indicate continuous improvement)
- ▶ Delivered Value and Customer Satisfaction (indicate actual impact and stakeholder value)

▶ **Metrics that must NOT be KPIs**

- ▶ Velocity (forecasting only)
- ▶ Throughput (context-dependent, not performance)
- ▶ Individual metrics of any kind
- ▶ WIP (diagnostic only, not performance)

Principles for Using KPIs in Scrum

- ▶ KPIs must support transparency, inspection, and adaptation
- ▶ KPIs must never be tied to evaluation of individuals
- ▶ KPIs must measure outcomes, not output
- ▶ KPIs must strengthen team autonomy, not pressure
- ▶ KPIs are used for learning, not comparison between teams

Best Practices

- ▶ **Avoid Anti-pattern Metrics (counterproductive metrics)**
 - ▶ Lines of code
 - ▶ Hours worked
 - ▶ Number of commits
 - ▶ Bugs found per developer
- ▶ **Choose Metrics for the Right Context**
 - ▶ Match metrics to project goals
 - ▶ Avoid measuring everything
 - ▶ Use a small set of meaningful KPIs
- ▶ **Use Metrics to Make Decisions**
 - ▶ Metrics highlight trends
 - ▶ Spot bottlenecks
 - ▶ Resource planning
 - ▶ Prioritization
 - ▶ Risk management
- ▶ **Use Metrics Ethically**
 - ▶ Avoid micromanagement
 - ▶ Protect psychological safety
 - ▶ Focus on team performance, not individuals
 - ▶ Metrics should guide improvement, not punishment

Further reading

- ▶ The Pennsylvania State University, Identifying relevant measures, <https://opair.psu.edu/planning-handbook/kpi-metrics/>
- ▶ <https://docs.gitlab.com/user/analytics/>
- ▶ <https://www.atlassian.com/agile/scrum/scrum-metrics>
- ▶ <https://www.atlassian.com/agile/project-management/metrics>
- ▶ <https://agilealliance.org/resources/experience-reports/metrics-understanding-flow>
- ▶